

Microarchitecture Analysis

LvYuan

Version 0.01 · Draft · August 24, 2026

Contents

Preface	vi
I Microarchitecture Fundamentals	1
1 Overview	2
1.1 Constraints That Shape Microarchitecture	3
2 Instruction Fetch Unit (IFU)	4
2.1 Front-End Organization	4
2.2 Control-Flow Prediction	4
2.3 Prediction Pipeline and Override	7
2.4 Instruction Supply Path	7
2.5 Queues, Decoupling, and Backpressure	8
2.6 Redirection, Replay, and Recovery	8
2.7 Front-End Performance Limits	8
3 Memory Management Unit (MMU)	10
3.1 Address Translation	10
3.2 Virtual Address Spaces and Protection	10
3.3 TLB Hierarchy and Reach	11
3.4 Address-Space Tags and Maintenance	11
3.5 Page Table Walking	12
3.6 Increasing Translation Coverage	12
3.7 Translation Prefetching	13
3.8 Virtualization and Nested Translation	13
3.9 MMU and Cache Interaction	14
3.10 Hardware–Software Co-Design	14
4 Instruction Decode	15
4.1 Decode Pipeline	15
4.2 Instruction Boundaries and Encoding Cost	15
4.3 Predecode, Alignment, and Queues	16
4.4 Semantic Translation and Internal Operations	16
4.5 Control-Flow Information	17

4.6	Decoded-Operation Caches	17
4.7	Decode Throughput and Energy	17
5	Register Rename	19
5.1	Register Renaming	19
5.2	Architectural and Physical State	19
5.3	Rename Structures	20
5.4	Move Elimination and Special Cases	20
5.5	Recovery and Checkpoints	20
5.6	Register Reclamation	21
5.7	Rename Scalability	21
6	Dispatch	23
6.1	Dispatch and Window Allocation	23
6.2	Coupled Resource Allocation	23
6.3	Dispatch and Issue	24
6.4	Backpressure and Window Balance	24
6.5	Recovery and Thread Sharing	25
7	Issue	26
7.1	Issue and Dynamic Scheduling	26
7.2	Ready-State Tracking	26
7.3	Wakeup and Select	26
7.4	Port Binding and Steering	27
7.5	Scheduling Policy	27
7.6	Replay and Speculative Scheduling	28
7.7	Scheduler Organization and Scalability	28
8	Execution Units (IXU/FPU)	29
8.1	Execution Back End	29
8.2	Latency, Throughput, and Initiation Interval	29
8.3	Integer and Branch Execution	29
8.4	Floating-Point and Vector Execution	30
8.5	Operand Supply and Bypass	30
8.6	Structural Hazards	31
8.7	Physical and Energy Constraints	31
9	Load/Store Unit (LSU)	33
9.1	Load/Store Pipeline	33
9.2	Address Generation and Translation	33
9.3	Load Queue and Store Queue	34
9.4	Memory Dependence Prediction	34
9.5	Store-to-Load Forwarding	35
9.6	Cache Access and Miss Handling	35

9.7	Ordering and Visibility	36
9.8	LSU Performance Limits	36
10	Retirement and Precise State	37
10.1	Retirement as the Architectural Boundary	37
10.2	Reorder Buffer	37
10.3	Precise Exceptions and Interrupts	38
10.4	Architectural Mapping and Resource Reclamation	38
10.5	Stores and Memory Visibility	39
10.6	Recovery and Machine Clears	39
10.7	Retirement Width and Head-of-Line Blocking	40
11	Cache	41
11.1	Cache Fundamentals	41
11.2	Capacity, Associativity, and Line Size	41
11.3	Banks, Ports, and Access Bandwidth	42
11.4	Instruction and Data Cache Differences	42
11.5	Addressing and Hit Path	42
11.6	Write Policies and Store Path	43
11.7	Miss Handling and Non-Blocking Caches	43
11.8	Multi-Level Hierarchies	44
11.9	Shared LLC and NUCA	44
11.10	Cache Coherence	45
11.11	Memory Types and External Agents	45
11.12	Cache Design Balance	46
12	Memory Ordering and Consistency	47
12.1	Memory Consistency as an Architectural Contract	47
12.2	Orders in a Shared-Memory System	47
12.3	Sequential Consistency	47
12.4	TSO and Store Buffers	48
12.5	Relaxed Memory Models	48
12.6	Microarchitectural Enforcement	49
12.7	Fences, Acquire/Release, and Atomics	49
12.8	Litmus Tests and Allowed Outcomes	49
12.9	Multi-Copy Atomicity and Heterogeneous Observers	50
13	Cache Replacement Policies	51
13.1	Replacement Fundamentals	51
13.2	Recency-Based Policies	51
13.3	Insertion and Promotion	52
13.4	Re-Reference Interval Prediction	52
13.5	Reuse Prediction	52
13.6	Beyond Miss Rate	53

13.7 Multi-Level and Shared Caches	53
13.8 Interaction with Prefetching	54
14 Prefetching	55
14.1 Prefetch Fundamentals	55
14.2 Sequential and Stride Prefetching	55
14.3 Correlation and Footprint Prefetching	56
14.4 Indirect and Execution-Based Prefetching	56
14.5 Instruction and Translation Prefetching	57
14.6 Placement and Request Priority	57
14.7 Accuracy, Coverage, and Timeliness	58
14.8 Feedback and Throttling	58
15 Security	60
15.1 Microarchitectural Security	60
15.2 Transient Execution Model	60
15.3 Spectre V1: Conditional Misprediction	61
15.4 Spectre V2: Target Injection	61
15.5 Return Prediction and History Injection	62
15.6 Fault-Driven Transient Execution	62
15.7 Speculative Store Bypass	63
15.8 Microarchitectural Data Sampling	63
15.9 Microarchitectural Side Channels	64
15.10 SMT and Shared State	64
15.11 Mitigation Principles	65
15.12 Mitigation Lifecycle	65
15.13 Security Evaluation	66
II Microarchitectures Application	67
16 AMD Microarchitectures	68
16.1 Zen 4	68
16.2 Zen 5	74
17 Intel Microarchitectures	91
17.1 Skymont	91
17.2 Lion Cove	94
18 Arm Microarchitectures	98
18.1 Cortex-X925	98
18.2 Neoverse N2	102
18.3 Neoverse V2	106

19 NVIDIA Microarchitectures	113
19.1 Vera and Olympus	113
References	123

Preface

本书是一份处理器微架构的入门手册,从通用视角梳理设计概念、工程权衡与实现路径。内容以中央处理器(CPU)为主线。书中既关注通用的微架构组织与性能目标,也纳入业内主流厂商的设计参考与演进脉络。

本书面向已经具备一定计算机组成、操作系统与性能分析基础的读者,希望在这些基础之上,进一步建立对现代处理器微架构的整体认识。这里所说的“整体认识”,并不是孤立地记忆若干术语、参数或厂商命名,而是尝试理解:一项微架构设计为什么会出现,它试图缓解什么瓶颈,又在性能、功耗、面积、时序、复杂度与软件语义之间付出了怎样的代价。

全书采用“先抽象、后实例”的组织方式。基础部分首先讨论前端、译码、重命名、调度、执行、访存、缓存、地址翻译与安全等核心模块,尽量为不同厂商、不同代际之间的比较建立一套统一分析框架;后续内容再结合 AMD、Intel、Arm、Apple 等主流设计,对这些抽象概念在真实产品中的实现方式加以观察。也就是说,本书更强调用一组可迁移的分析视角理解处理器,而不是把具体产品参数当作彼此孤立的知识点逐条罗列。

在写作边界上,本书主要关注微架构层面的组织与权衡。书中会在必要处涉及指令集、操作系统、编译器、互连与内存系统等主题,因为这些因素常常直接影响微架构设计;但本书并不以晶体管级电路实现、EDA 流程、编译器后端优化或应用层调优技巧为主轴。对于 GPU、NPU、DSP 等其他处理器形态,书中也可能在某些章节中简要提及其共性或差异,但整体叙述仍以通用 CPU 为中心。

书中所使用的资料来源以公开文档为主,包括体系结构手册、厂商白皮书、学术论文、技术报告、性能分析资料以及部分经过交叉验证的公开讨论。由于现代商用处理器的许多实现细节并不会被完整披露,因此某些章节在描述具体产品时,难免会存在基于公开证据的归纳与谨慎推断。对此,本书会尽量把重点放在那些较稳定、较可验证的结构性的规律上,而不是对单一未证实细节作过度延伸。

需要特别说明的是,处理器微架构是一个高度动态演化的领域。相同术语在不同厂商、不同代际之间可能对应不同结构,相似参数背后也可能隐藏完全不同的实现方式。因此,阅读本书时,最有价值的收获不应只是记住“某款处理器有几个端口、几级缓存或多少条执行管线”,而应是逐渐形成一种判断能力:当面对新的处理器资料时,能够把零散参数放回更大的设计框架之中,理解它们为何重要、彼此如何约束,以及它们最终会怎样影响真实程序的行为。

Part I

Microarchitecture Fundamentals

Chapter 1

Overview

现代 CPU 微架构(microarchitecture)的分析,既不能停留在 ISA 语义层面,也不应直接陷入某一代产品的参数细节。更稳妥的起点,是先建立一套通用框架:一条指令如何被取出、翻译、解码、重命名、调度、执行、访存以及退休;各个阶段分别受哪些结构限制;不同厂商又为何会在宽度、深度、容量、频率、面积和功耗之间做出不同的取舍。

从抽象层次上看,ISA 定义了软件可见的行为,而微架构决定了硬件如何高效地实现这些行为。以相同的 ISA 为例,不同处理器可以具有完全不同的前端组织、不同大小的乱序窗口、不同的缓存层次以及不同的预测结构。因此,分析微架构时必须把“功能是否正确”和“如何高效实现”区分开来。前者是体系结构语义,后者则属于微架构设计空间。

现代高性能 CPU 通常围绕三个核心目标展开优化:第一是单线程性能,即尽可能降低关键路径上的有效延迟;第二是总体吞吐,即在单位时间内维持更多有用指令在流水线中流动;第三是能效和面积效率,即在给定功耗与硅面积约束下获得尽可能高的性能。许多看似局部的设计选择,例如扩大 BTB、增加调度队列深度、提高缓存带宽、引入更激进的预取器,本质上都服务于这三类目标,只是不同产品的优先级不同。[1]

从数据流角度看,一颗典型的乱序超标量核心可以被理解为若干相互耦合的模块:指令取指单元(Instruction Fetch Unit, IFU)产生预测路径上的指令供给;译码(Decode)把 ISA 编码转化为内部操作;重命名(Rename)与分派(Dispatch)建立依赖并分配在途资源;发射(Issue)与执行单元推进就绪操作;访存单元(Load/Store Unit, LSU)与缓存层次共同完成数据访问;退休(Retire)最终按程序顺序更新体系结构状态。内存管理单元(Memory Management Unit, MMU)并不是这条主线中的单一串行阶段,而是分别与取指和数据访问并行协作,提供地址翻译、权限检查与内存属性。

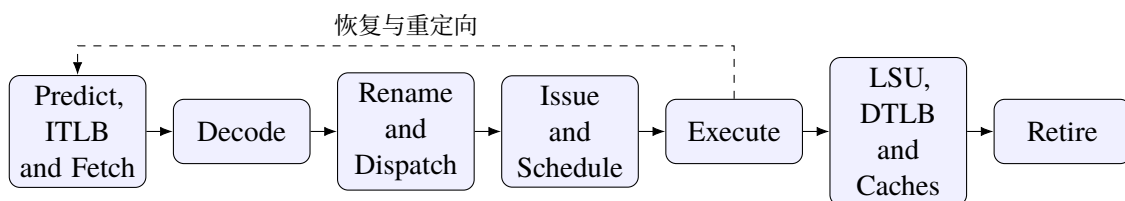


Figure 1.1: 典型乱序核心的主数据流。执行可以乱序推进,但恢复与退休机制必须维持软件可见的顺序语义。

模块化分析并不意味着结构彼此独立。分支预测会改变乱序窗口中的有效工作比例,地址翻译会影响缓存访问关键路径,预取会同时消耗队列、缓存和内存带宽,SMT 则会改变几乎所有共享资源的竞争关系。因此,基础部分在沿主数据流介绍结构的同时,也把缓存替换、预取、一致性与安全等跨模块问题单独讨论。

1.1 Constraints That Shape Microarchitecture

现代微架构是在若干基本约束共同作用下形成的, 其中最重要的是吞吐(throughput)、延迟(latency)、能耗(energy)与面积(area)。一个更大的预测器可能改善前端吞吐, 却也可能增加访问延迟和能耗; 更深的乱序窗口有助于寻找独立工作, 却会扩大唤醒、选择、寄存器和恢复结构。因而, 这四个维度不是设计完成后的评价标签, 而是决定结构能否实现以及应当做到多大的初始条件。

对前端而言, 吞吐表现为单位时间内能否持续提供足够多的有用指令, 延迟则更多表现为分支预测、目标生成、重定向与取指返回路径是否足够迅速。对后端而言, 吞吐取决于 rename、dispatch、issue、execute 与 retire 资源能否长期保持高利用率, 而延迟则更多体现在关键依赖链、唤醒选择、旁路网络以及访存返回路径上。对存储系统而言, 吞吐对应的是缓存、LSU、MSHR 与内存子系统所能维持的并发服务能力, 延迟则体现在 cache miss service path、TLB miss 与 page walk 路径是否会持续暴露给核心。

能耗与面积则决定了这些结构能够被实现到什么程度。更大的表、更长的历史、更高的 associativity、更深的队列, 通常意味着更高的存储成本、更复杂的布线以及更困难的时序收敛; 而更多的访问端口、更激进的预测、更高频参与关键路径的状态位翻转, 又会迅速抬高访问能耗与错误路径成本。也正因为如此, 现代高性能核心里很多结构的关键问题已不是“能不能继续做大”, 而是“继续做大之后, 收益是否仍足以覆盖它带来的延迟、能耗与面积代价”。

微架构分析真正需要判断的, 不是某个结构是否单纯“更大”“更准”或“更深”, 而是它改善了平均吞吐还是关键路径延迟, 收益能否在真实工作负载中持续出现, 以及新增面积、能耗和时序复杂度是否把瓶颈转移到了其它位置。只有把参数放回这些约束中, 结构之间的工程关系才会清晰。

Chapter 2

Instruction Fetch Unit (IFU)

2.1 Front-End Organization

指令取指单元 (Instruction Fetch Unit, IFU) 负责沿预测的控制流持续生成取指地址, 并把正确路径上的指令字节送往译码端。对宽发射乱序核心而言, 后端是否拥有足够多的执行资源并不是唯一限制; 如果前端不能稳定提供有用指令, 调度窗口与执行端口仍会周期性空转。因此, IFU 的核心目标不是追求孤立的峰值取指宽度, 而是提高**有效指令供给率**: 在控制流预测、地址翻译、指令缓存与队列约束下, 尽可能持续地向下游提供最终会被退休的指令。[1]

现代 IFU 可以概括为三条相互耦合的路径。第一条是**下一取指地址生成路径**, 由方向预测器、分支目标缓冲器 (Branch Target Buffer, BTB)、间接目标预测器、返回地址栈 (Return Address Stack, RAS) 以及顺序地址生成逻辑共同构成。第二条是**指令供给路径**, 负责 ITLB 查询、L1 指令缓存访问、取指块对齐和必要的预译码。第三条是**解耦与恢复路径**, 利用取指目标队列 (Fetch Target Queue, FTQ)、指令队列和检查点保存尚未被验证的控制流, 并在预测错误或前端异常发生时撤销错误路径。

这三条路径通常并非严格串行。快速预测结构可能先给出下一取指地址, 较大但较慢的预测结构随后校正结果; ITLB 与 L1 指令缓存可以在地址条件允许时并行访问; 取指队列则把预测、缓存返回和译码之间的短期速度差异隔离开来。由此形成的前端更接近一个带反馈的供给网络, 而不是从 PC 到 Decode 的单向流水线。

2.2 Control-Flow Prediction

控制流预测必须回答两个不同问题: 条件分支是否跳转, 以及发生控制转移时目标地址在哪里。方向预测错误会使前端沿错误路径推进; 目标预测缺失则可能在方向已经判断为跳转时仍无法及时产生下一 PC。两者都会降低有效取指带宽, 但所依赖的历史信息和硬件结构并不相同。

2.2.1 Direction Prediction

方向预测器利用静态分支地址、局部历史、全局控制流历史和路径上下文估计条件分支结果。经典两级预测器把历史记录与模式表分离; gshare 等结构通过混合 PC 与全局历史降低有害别名; TAGE 一类预测器使用多组不同历史长度的表, 以较短历史覆盖常见模式、较长历史捕捉稀疏相关性; 感知器类预测器则以加权相关性处理较长历史。现代设计通常还配置循环预测器或其它专用组件, 以处理主预测器不擅长的分支类型。[2]

预测器容量并不自动转化为更高性能。更长历史和更多表项有助于降低一部分误预测, 却会增加访问延迟、索引与标签开销、训练状态和动态能耗。若复杂预测器无法在取指关键路径内给出结果, 前端就需要采用流水化预测、快速预测与慢速覆盖 (override), 或者在更早的控制流位置预取未来预测结果。因此, 方向预测器的实际价值取决于准确率、覆盖率、访问时序和错误恢复代价的共同作用。

投机历史的维护也是实现的一部分。前端为了连续预测多个尚未执行的分支, 必须暂时把预测结果写入全局历史、路径历史或循环状态; 一旦较老分支被判定为误预测, 这些状态需要恢复到对应检查点。若检查点不足、恢复延迟过长或错误路径更新没有被正确隔离, 预测器本身就可能成为后续错误的来源。

两级预测器有助于理解历史如何进入预测。局部历史记录同一静态分支最近若干次结果, 适合捕捉循环退出或周期性分支; 全局历史记录近期多个动态分支的结果, 适合捕捉分支之间的相关性。局部历史需要为不同分支保存独立状态, 全局历史则容易让无关分支相互干扰。现代预测器通常并不只选择其中一种, 而是用多个组件覆盖不同模式。

预测表中的别名 (aliasing) 是方向预测的重要限制。不同分支或不同历史上下文可能映射到同一计数器, 并以相反方向更新它。加入标签能够减少错误匹配, 却会增加存储与访问延迟; 使用 PC 与历史的 XOR、不同银行采用不同哈希, 或把预测目标改为“是否符合该分支的常见偏置”, 都可以在不完全消除共享的前提下降低有害冲突。

TAGE 的核心不是单纯使用很长历史, 而是同时维护若干具有几何增长历史长度的 tagged tables。基础预测器覆盖没有匹配长历史的情况, 较长历史表只有在标签匹配时才参与预测。这样, 容易预测的分支不必消耗长历史状态, 真正依赖稀疏远程相关性的分支则可以使用更长上下文。表项还需要有用性或替换信息, 以避免长历史表被短暂模式占满。

感知器类预测器把历史结果映射为正负输入, 并计算加权。它能用较紧凑参数表达较长线性相关性, 但加法树、权重更新和阈值判断会增加时序与能耗。对于非线性模式, 单一感知器也可能无法准确分类。因此, “neural predictor”通常只说明预测器使用了某种加权学习思想, 并不能由名称推断具体表结构或性能。

循环预测器则利用循环分支常见的迭代次数规律。若某分支连续多次 taken 后固定出现一次 not-taken, 普通饱和计数器可能在循环退出时反复失误; 循环预测器可以记录当前迭代计数和历史稳定次数, 在置信度足够时直接预测退出点。它对稳定循环有效, 但必须识别迭代次数变化和嵌套循环, 否则错误覆盖主预测器反而会增加代价。

2.2.2 Target Prediction and BTB Organization

BTB 保存控制流指令的位置、类型与目标相关信息, 使前端在尚未完成取指和译码时就能生成跳转目标。它解决的不只是单次目标查询延迟, 还解决宽前端中的**取指 PC 生成带宽**: 当基本块较短、每个取指块包含多个分支时, 前端必须在很少周期内跨越多个控制流边界, 否则即使方向预测完全正确, 也无法持续填满译码器。

一个 BTB 表项可能包含分支标签、目标或目标增量、分支类型、块内位置以及与多分支预测有关的附加信息。直接分支目标通常接近分支自身, 可以保存页内或区域内 delta; 间接分支目标变化较大, 需要结合专用目标预测; call 与 return 还可能触发 RAS 更新。表项格式越丰富, 单条分支的状态成本越高, 有限容量下可覆盖的静态控制流数量越少。

BTB 的组织通常在覆盖率、存储效率和每周目标吞吐之间取舍。按单条分支组织的 Instruction BTB 能精确表示分支, 却需要较多标签和端口; 按取指块组织的 Block BTB 可以在一个表项中记录块内多个分支, 但条目利用率受分支密度影响; Region BTB 通过共享区域地址信息

提高存储效率,但需要额外编码和目标重构。目标去重、页内增量编码和多级 BTB 都是在有限面积下扩大有效覆盖范围的常见方向。[3], [4]

现代 BTB 往往采用层次化组织:靠近取指路径的小型表提供低延迟结果,更大的后级表覆盖较大的代码工作集。一级未命中不一定意味着最终无法预测,但后级结果到达前可能已经产生前端气泡。因而,BTB 容量、层级命中率和覆盖延迟必须结合基本块长度与前端队列深度分析,不能只比较总表项数。

BTB miss 与方向误预测的效果不同。方向误预测通常让前端沿错误路径推进,直到分支执行后恢复;BTB miss 则可能使前端暂时按顺序地址取指,或者等待译码阶段确认这是一条控制流指令并生成目标。前者浪费错误路径工作,后者更常表现为目标生成气泡。性能计数器若只给出总分支误预测率,可能无法反映 BTB 覆盖不足造成的前端损失。

大代码工作集会使得 BTB 容量压力显著上升。服务器程序、解释器和大型运行时包含大量冷分支与共享目标,目标地址高位常具有区域局部性。目标去重、区域基址共享和可变长度编码正是利用这种冗余;但压缩后的表项需要额外重构逻辑,并可能使单次访问返回的分支数量或目标生成率受到限制。

对于一个周期内跨越多个 taken branch 的设计,仅增加 BTB 表项数还不够。前端还需要能够并行产生多个候选目标,或者利用提前一个或多个取指块的预测信息形成多块前瞻。多块前瞻预测的基本价值,是把目标生成从“当前块决定下一块”推进为“当前状态已经准备更远块”,以缓解宽前端中的串行 next-PC 依赖。[5]

多分支预测还必须处理块内顺序。若一个取指块中包含多个预测为 taken 的分支,程序实际只会沿最早被执行的 taken 分支离开当前块;更年轻分支只在更老分支不跳转时才有效。预测器因而不仅要生成多个方向和目标,还要按照块内位置选择最早有效的控制流,并把其后的字节标记为无效。

2.2.3 Indirect Branches and Returns

直接分支的目标通常可由静态指令位置和位移确定,而间接分支的目标来自寄存器或内存值,同一静态分支可能对应多个动态目标。简单的 last-target 预测只适用于目标高度稳定的场景;更复杂的间接预测器会结合全局历史、路径或调用上下文区分不同目标。其限制不仅是预测准确率,还包括候选目标数量、更新延迟和与 BTB 的协同方式。

函数返回具有更强的调用栈规律,因此通常由 RAS 单独预测。调用时压入返回地址,返回时弹出栈顶地址,可以避免把大量返回目标交给一般间接预测器处理。但 RAS 会受到深层递归、异常、上下文切换、错误路径调用和非标准控制流破坏,因此同样需要投机更新与恢复机制。

间接分支预测器通常需要区分同一静态分支在不同上下文中的目标,例如虚函数调用、解释器 dispatch 或 jump table。仅保存最后一个目标会在单态调用点表现良好,却难以处理多态目标;加入全局历史、路径或先前目标可以提高区分能力,但表项数量和训练时间也会增加。目标预测错误通常比普通方向错误更难提前发现,因为真实目标可能依赖较长的寄存器数据流。

RAS 也存在容量和一致性问题。栈深度超过硬件容量时,旧返回地址可能被覆盖;错误路径 call/return 会投机改变栈顶;setjmp/longjmp、异常返回和协程切换等非标准控制流还可能破坏简单的调用-返回配对。实现可通过检查点、循环栈、影子计数或退化到间接预测器处理这些边界情况。

2.3 Prediction Pipeline and Override

低延迟与高准确率预测通常难以由同一结构同时满足。快速一级预测器可以在每周期及时生成 PC, 但容量和历史有限; 较慢的后级预测器拥有更丰富状态, 却可能需要多个周期。前端可以先沿快速结果取指, 后级结果不一致时再覆盖并重定向, 这种组织把“完全等待慢预测器”转换为“多数时候使用快速路径、少数时候支付覆盖代价”。

预测流水化会使更新与使用之间存在距离。当前周期查询的表项可能由更早周期的分支结果更新, 同一表中还可能存在多个在途预测。前端需要把每个预测与对应 FTQ 项、历史快照和分支位置绑定, 才能在分支解析时更新正确表项, 并在误预测时恢复正确历史。

覆盖并不总是等同于完整误预测。若慢预测结果在错误路径尚未进入后端前到达, 前端只需丢弃少量取指块; 若结果到达较晚, 覆盖就会接近一次普通分支恢复。因而, 预测层级的收益取决于后级准确率提高是否足以抵消覆盖频率和覆盖延迟。

2.4 Instruction Supply Path

预测得到下一取指地址后, 前端仍需完成地址翻译、缓存访问和指令组织。ITLB miss、L1 指令缓存 miss、跨缓存行或跨页取指都会打断字节供给。对于具有较大代码工作集的服务器和运行时程序, 目标预测正确并不等于前端能够及时获得目标处的指令; 控制流预测、指令预取、ITLB 与 L1 指令缓存必须作为同一条供给路径分析。

固定长度 ISA 可以较早确定指令边界, 便于把取指字节分配给并行译码槽位; 可变长度或带压缩编码的 ISA 则需要长度识别、跨块拼接和对齐逻辑。前端常在缓存行填入或取指过程中生成预译码元数据, 用于标记指令边界、潜在控制流位置和特殊译码需求, 从而减轻主译码器的关键路径压力。

峰值字节带宽与有效指令带宽并不相同。taken branch 可能使缓存行尾部字节失效, 取指地址可能落在行中部, 多个目标可能分布在不同缓存行或页中。评价前端供给时, 需要同时考虑取指块粒度、基本块长度、对齐损失、ITLB/L1I 命中率以及正确路径字节比例。

部分处理器配置微操作缓存或其它译码后缓存, 使热点代码绕过重复的字节边界识别与主译码过程。该结构缓存的是处理器内部操作, 而不是所有 ISA 都必然采用的通用前端组件。它能降低译码功耗并提高热点路径供给, 但容量、标签与控制流组织仍会限制命中率; 未命中时, 前端必须回到常规 L1I 与 Decode 路径。

L1 指令缓存通常以只读为主, 因此端口和写路径压力小于 L1D, 但它对目标访问延迟和连续带宽极其敏感。取指块可能跨越缓存行, taken branch 也可能要求同周期访问两个不同区域。Bank 化可以提高吞吐, 却需要取指地址在 bank 间分布良好; 代码布局和分支目标对齐会直接影响可用带宽。

指令预取试图沿预测控制流提前获取未来缓存行。Next-line 机制对顺序代码有效, 基于 BTB 或取指轨迹的机制可以跨越 taken branch。预取过远会把错误路径代码带入 L1I, 过近又无法覆盖低层延迟, 因此指令预取的 lookahead 与分支预测深度紧密相关。

自修改代码、代码页权限变化和指令/数据别名要求前端执行规定的一致性维护。即使数据侧已经写入新指令字节, L1I、预译码元数据和操作缓存也可能仍保存旧版本。体系结构通常通过显式 cache maintenance 与 instruction synchronization 指令定义软件责任, 微架构则必须保证这些操作正确清除或失效相关状态。

2.5 Queues, Decoupling, and Backpressure

FTQ 或类似结构保存预测得到的取指片段及其历史检查点,使预测器可以领先于实际缓存访问。指令字节队列与 Decode queue 则吸收 L1I 返回和译码消耗之间的短期差异。适当的解耦允许预测器在缓存短暂阻塞前积累若干未来目标,也允许 Decode 在取指侧短暂抖动时继续工作。

队列只能吸收有限时间的速度差异,不能消除长期带宽不足。若 Decode 持续慢于取指,队列最终填满并向预测端施加反压;若 ITLB 或 L1I miss 使取指长期停顿,队列最终耗尽并使 Decode 饥饿。队列容量还会影响误预测代价:更深的前瞻可以隐藏更多延迟,却也允许错误路径占用更多状态并产生更广泛的缓存和预测器污染。

FTQ 项通常以取指块或基本块为粒度,保存起始 PC、预测分支位置、目标、历史检查点和与缓存请求相关的标识。Decode 不一定与预测器保持逐周期同步:一个 FTQ 项可以等待多个缓存块返回,也可能因为后级预测覆盖而在尚未到达 Decode 前失效。这种组织使控制流管理与字节传输解耦。

队列的粒度会影响状态效率。按指令保存信息精确但表项多,按 fetch block 保存信息紧凑却需要在块内恢复分支位置和有效字节。融合、跨块指令和多分支取指还会使“一个队列项对应多少后端操作”随代码形态变化。

2.6 Redirection, Replay, and Recovery

前端重定向不只来自分支方向错误。BTB 目标错误、返回栈失配、间接目标错误、取指权限异常、ITLB 状态变化以及预译码信息不一致,都可能要求前端取消部分在途工作并重新生成取指地址。不同事件的发现位置不同,因此恢复延迟也不同:越晚发现的错误,通常已经在队列和后端中积累了越多无效操作。

恢复过程至少包括检测错误、选择恢复检查点、修复投机历史、清除错误路径队列项、生成正确 PC,以及重新通过 ITLB、L1I 和 Decode。所谓“分支误预测代价”因此不是单一流水级数量,而取决于分支条件产生时间、重定向路径、正确目标的缓存状态和前端剩余队列内容。

Replay 与完全重定向也应区分。若问题只涉及某个取指块尚未准备好或后级预测结果覆盖了快速预测,前端可能只需重新请求局部工作;若控制流本身错误,则需要撤销所有更年轻路径。能够限制恢复范围的设计可减少无效工作,但需要更精细的队列标记和检查点管理。

错误路径还可能训练方向预测器、BTB、RAS 或指令预取器。部分状态只在分支退休后更新,可以避免污染但训练较晚;部分状态必须投机更新,才能连续预测后续分支,却需要在恢复时修正。哪些状态允许错误路径更新,是准确率、复杂度、能耗与安全共同约束的设计选择。

前端恢复结束的标志不是正确 PC 刚被生成,而是正确路径重新产生可供 Decode 消费的指令。若目标处发生 ITLB 或 L1I miss,重定向之后仍会继续空转。因此,实测恢复代价常随代码布局 and 缓存状态变化,而不是一个完全固定的流水级常数。

2.7 Front-End Performance Limits

IFU 的性能上限由多个串联条件共同决定:预测器能否给出足够多的正确路径,BTB 能否及时生成目标,ITLB 与 L1I 能否提供目标处字节,队列能否吸收短期延迟,Decode 又能否持续消费这些字节。任一环节长期不足,都会使局部峰值宽度失去意义。

因此,前端分析应区分**结构带宽**与**持续有效带宽**。结构带宽描述理想条件下每周期可预测、读取或传递多少工作;持续有效带宽还要扣除误预测、目标缺失、对齐浪费、缓存与翻译缺失、队列

反压以及错误路径工作。现代 IFU 的设计目标, 正是在有限面积和能耗预算内, 使这些损失尽可能少地暴露给后端。

Chapter 3

Memory Management Unit (MMU)

3.1 Address Translation

内存管理单元(Memory Management Unit, MMU)把软件使用的虚拟地址转换为物理地址,并完成权限、内存属性和地址空间隔离检查。地址翻译同时服务于前端取指与后端访存:ITLB 或指令侧页表遍历会影响指令供给,DTLB 与数据侧遍历则直接位于 load/store 的关键路径。MMU 因而不是操作系统语义之外的附属结构,而是连接地址生成、缓存访问、异常处理和虚拟化的微架构子系统。[1]

虚拟地址通常由虚拟页号与页内偏移组成。页表项给出物理页号,并携带读写执行权限、特权级、可缓存性、共享属性和访问状态等元数据。页内偏移在翻译过程中保持不变,因此缓存索引若只使用页内位,可以与 TLB 查询并行;若需要物理页号才能完成索引或标签判断,则访问路径必须等待翻译结果。

翻译并非只在 cache miss 时发生。每次指令取指、load 和 store 都需要一个有效物理地址或等价的已验证翻译结果。TLB 命中把多级页表查询压缩为一次近端查找;一旦 TLB miss,处理器才启动 page walk。因此,MMU 的稳态性能主要由 TLB 命中路径决定,而长尾行为则由 page walk、缺页异常和全系统失效决定。

虚拟内存还允许软件为同一虚拟范围改变后备物理页,或暂时不建立映射。若页表项无效,硬件不能自行完成翻译,而需要产生缺页异常,由操作系统分配页面、调入数据或拒绝访问。缺页处理通常远慢于普通微架构事件;微架构中的稳态优化主要针对映射已经存在时的硬件翻译路径。

3.2 Virtual Address Spaces and Protection

每个进程或虚拟机可以拥有独立的虚拟地址空间。相同虚拟地址在不同地址空间映射到不同物理页,称为 homonym;不同虚拟地址映射到同一物理页,称为 synonym。前者要求 TLB 项包含地址空间标识符(ASID)或虚拟机标识符(VMID),或者在切换时执行失效;后者主要影响以虚拟地址参与索引或标记的缓存结构,因为同一物理对象可能通过多个虚拟名字被访问。

权限检查必须与翻译结果保持一致。处理器可以推测性地发起后续访问,但异常、访问位更新和软件可见结果必须遵守体系结构定义。翻译缓存保存的不只是物理页号,也保存权限与内存属性;修改页表后若旧 TLB 项仍可被使用,就可能同时破坏隔离和缓存语义。

执行权限、特权级和读写权限可能在不同流水位置参与检查。为缩短访问路径,处理器可以并行进行 TLB 查询、缓存索引和部分权限判断,但未经授权的数据不能以可传播形式进入后续执

行。安全章节讨论的 fault-driven transient execution, 正说明权限检查的最终结果正确并不足够, 其生效时机也属于微架构边界。

内存属性同样会随翻译传播。普通可缓存内存、设备内存、不可缓存区域和不同共享属性可能进入不同的缓存与排序路径。TLB 命中若返回错误或过时属性, 即使物理页号正确, 也可能产生错误的顺序或一致性行为。

3.3 TLB Hierarchy and Reach

转换后备缓冲器 (Translation Lookaside Buffer, TLB) 缓存近期页表翻译。高性能核心通常分别配置低延迟 ITLB 与 DTLB, 并使用更大、可能统一的后级 TLB 扩大覆盖范围。一级结构接近取指或 LSU 关键路径, 容量和相联度受到访问延迟与端口能耗约束; 后级结构允许更高容量, 但命中时仍会引入额外周期。

TLB 的有效覆盖范围通常称为 reach, 可近似理解为表项数量与页面大小的乘积。较大页面能显著扩大 reach、减少页表层级与遍历次数, 但会增加内存碎片、迁移粒度和分配约束。较小页面提供更细的保护与管理粒度, 却需要更多 TLB 项和更高的页表访问频率。实际 MMU 因而通常同时支持多种页面大小, 并为不同页大小配置独立或共享的翻译资源。

容量并不是 TLB 的唯一指标。取指和多条 load/store 管线可能在同一周期产生多个翻译请求, 因此端口、bank 冲突、相联查找延迟和 miss 合并能力同样重要。一个容量较大的 TLB 若无法维持所需查询带宽, 仍可能限制宽前端或宽 LSU 的持续吞吐。

一级 TLB 可以采用全相联或高相联组织, 以减少有限表项中的冲突; 后级 TLB 容量更大, 通常需要 set-associative 或 banked 结构控制访问成本。不同页大小可共享同一表, 也可使用独立阵列。共享有助于动态利用容量, 但匹配逻辑必须同时判断页面大小; 分离结构实现清晰, 却可能在某种页大小流量集中时留下其它阵列空闲。

TLB replacement 与 cache replacement 类似, 却有不同代价。驱逐一个翻译项不会直接丢失数据, 但后续 miss 可能触发多次页表访问, 并占用缓存与 walker。大页项覆盖范围更大, 简单按 recency 竞争可能使单个小页流驱逐高价值大页翻译, 因此实现可能对页大小和来源设置不同保留策略。

多线程核心还要决定 TLB 是否按线程标记、动态共享或保留配额。完全共享提高平均 reach, 却允许一个线程的地址流污染另一线程; 静态分区提高隔离, 却降低单线程模式下的可用容量。ASID/VMID 解决正确性标记问题, 并不自动解决容量和带宽公平性。

3.4 Address-Space Tags and Maintenance

ASID 与 VMID 允许不同地址空间的翻译同时驻留, 从而减少上下文切换时的全局失效。但标识符数量有限, 回收和复用仍需要与操作系统维护协议配合。页表修改、权限收紧、页面迁移和虚拟机映射更新都可能要求执行 TLB invalidation, 并确保所有相关核心在继续访问前停止使用旧翻译。

多核系统中的 TLB shutdown 既是正确性机制, 也是性能事件。软件需要识别可能持有旧翻译的执行上下文, 发送失效请求并等待必要确认; 硬件则需要定义失效作用范围、排序语义和与在途 page walk 的交互。更细粒度的失效可减少无关翻译损失, 却需要更复杂的匹配和广播机制。

失效与在途访问之间需要明确排序。页表更新后, 旧 page walk 可能仍在返回结果; 若该结果重新填入 TLB, 就会使刚失效的映射复活。处理器通常需要使用代际标记、请求取消、完成前检查

或序列化规则, 保证失效之后不会继续安装过时翻译。

频繁 shutdown 会破坏 TLB 局部性, 并使多个核心在同步等待中停顿。操作系统因此会尝试批量处理映射变化、推迟不必要失效, 或者利用 ASID 避免上下文切换时清空全部翻译。硬件支持的失效粒度和确认机制会直接影响这些软件策略的成本。

3.5 Page Table Walking

TLB miss 后, 硬件页表遍历器 (Page Table Walker, PTW) 按照体系结构规定的多级页表格式读取页表项。一次遍历可能触发多次缓存访问; 若中间页表项也未命中缓存, 翻译延迟会迅速放大。页表遍历还会占用缓存端口、MSHR、互连和内存带宽, 因此大量 TLB miss 不只是增加单次访问延迟, 也会与普通指令和数据访问竞争共享资源。

现代实现通常配置 page-walk cache 或 MMU cache, 保存非叶级页表项或中间翻译结果。这些结构并不等同于普通 TLB: TLB 缓存最终页翻译, walker-side cache 则减少后续遍历中的中间级内存访问。多个并行 walker、相同页表路径的 miss 合并以及页表访问的缓存优先级, 共同决定 TLB miss 能否与其它工作重叠。

页表遍历过程还必须处理访问位、脏位、异常与权限变化。即使实现对页表读取进行了投机或并行化, 软件可见的异常顺序与更新语义仍须保持精确。

以四级页表为例, walker 从根表基址出发, 用虚拟页号的不同字段依次索引中间表, 最终获得叶级页表项。任一级都可能以大页项提前结束遍历, 也可能因无效项产生异常。中间页表通常具有较强时间局部性: 同一进程相邻虚拟页常共享上层路径, 因此缓存非叶项能显著减少后续 walk 的访问次数。

页表访问使用普通物理内存地址, 因而会进入缓存层次并与 demand load、store 和指令取指竞争。为了避免 walker 完全受普通访问阻塞, 缓存或互连可以提供独立队列或优先级; 但给予过高优先级又可能让大量 TLB miss 反过来伤害正常数据请求。

多个 TLB miss 可以并行遍历, 但它们可能共享相同上层页表项。请求合并与 walker-side cache 能减少重复读取; 若 walker 数量过少, 宽 LSU 会因翻译队列饱和而停顿; 若数量过多, 则可能把缓存和内存带宽消耗在大量页表访问上。并行度的合适范围依赖工作负载页大小与 miss 相关性。

3.6 Increasing Translation Coverage

扩大 TLB 容量是提高覆盖范围的直接方法, 但高相联、多端口的大型 TLB 会增加延迟和能耗。另一类方法是让一个翻译项覆盖多个基础页。大页依赖操作系统显式建立较大映射; TLB coalescing 则利用相邻虚拟页与物理页之间的连续或簇状关系, 把多个页级映射压缩到一个条目中。

聚合翻译必须满足权限、内存属性和失效语义兼容。纯硬件方案可以在页表遍历或 TLB 填入时识别连续关系; 硬件-软件协同方案则由操作系统把连续性编码进页表元数据, 使硬件无需反复推断。现实地址空间通常同时包含连续、簇状和离散映射, 因此多粒度表示比完全替代普通页项更为实际。[6], [7]

coalescing 并不是大页的无成本替代。页面迁移、写时复制、NUMA 重平衡和权限变化都可能缩短可聚合范围; 覆盖粒度越大, 维护与失效也越复杂。其价值应以增加的有效 reach 是否足以抵消额外匹配、元数据和维护成本来判断。

大页与 coalescing 的控制主体不同。大页由操作系统建立显式的大粒度映射,能够减少页表层级并提高 TLB reach,但分配和迁移粒度也同步增大;coalescing 可以保留基础页语义,只在翻译缓存中压缩若干兼容映射,却仍需读取或验证对应页表信息。两者可以共存,并服务于地址空间中不同规整程度的区域。

Clustered translation 利用的并不一定是严格连续映射,而可以是虚拟页与物理页均落在有限范围内的簇状关系。条目保存共享高位和簇内编码,使若干非完全连续页共享标签与元数据。这提高了适用范围,却需要更复杂的匹配、权限验证与失效判断。

翻译覆盖优化的共同原则,是用额外表示复杂度换取更少的 TLB miss。判断其有效性时,既要观察 miss 减少,也要观察一级命中延迟是否增加、后级查找是否变复杂,以及映射变化时需要维护多少范围状态。

3.7 Translation Prefetching

翻译预取试图在需求访问发生前准备未来需要的页翻译或页表元数据。顺序页流可由 next-page 类机制捕捉,重复跨页关系可由相关性预测器学习;一次 demand page walk 还可能暴露同一页表缓存行中的相邻 PTE,从而以较低增量成本发现候选翻译。

翻译预取与数据预取面临相同的准确率、及时性和污染问题,但它竞争的是容量有限的 TLB、walker 与页表缓存。错误预取可能驱逐有用翻译,并增加页表访问流量。因此,实际机制需要置信度、节流和放置策略:候选结果可以进入专用预取缓冲、后级 TLB,或仅提前准备中间页表项,而不必无条件污染一级 TLB。[8], [9]

指令侧与数据侧的翻译流具有不同特征。指令流受控制流与代码布局影响,数据流则更多反映数组、对象和数据结构访问。翻译预取器应使用与目标路径相匹配的信号,而不能简单把同一地址序列算法同时套用在 ITLB 与 DTLB 上。

翻译预取的触发点可以是 TLB miss、TLB hit、page walk 完成或普通预取器准备跨页。只在 miss 后训练能够聚焦昂贵访问,却可能获得较短 lookahead;在 hit 时持续观察页流能更早生成请求,但状态与误取数量更大。将数据预取器的未来页信息传递给 MMU,可以避免 cache-line 预取因缺少翻译而停顿。

预取结果的放置同样重要。直接填入一级 TLB 能缩短 demand 路径,却容易驱逐热点翻译;填入后级 TLB 或专用 buffer 更保守,但 demand 仍需一次额外查找。对 page-walk 中间项的预取则不增加最终 TLB 项,却能缩短未来 miss 恢复时间。

多核共享地址空间可能表现出跨核翻译相关性,一个线程刚访问的页面可能很快被另一线程访问。协作式预取可以利用这种关系,但会增加线程间干扰与安全域共享,因此必须限制传播范围,并避免让一个核心的扫描流污染其它核心 TLB。

3.8 Virtualization and Nested Translation

虚拟化通常引入两阶段翻译:客户机虚拟地址先转换为客户机物理地址,再转换为系统物理地址。若缺乏专门支持,客户机页表遍历中的每一次内存访问都可能再次触发第二阶段翻译,使一次 TLB miss 演化为多次嵌套访问。

现代处理器通过嵌套页表、带 VMID 的 TLB、walker-side cache 和对两阶段中间结果的缓存降低这一成本。此时 MMU 的关键指标不再只是单级 TLB 容量,还包括两阶段命中组合、嵌套遍历并行度以及 guest 与 hypervisor 更新映射时的失效范围。[10]

组合 TLB 项可以直接缓存 guest virtual 到 system physical 的最终结果, 使稳态访问仍表现为一次查找; 但 miss 时需要分别解释两套页表。缓存 guest-physical 到 system-physical 的中间结果, 可以减少 guest page walk 中每次页表读取触发的第二阶段翻译。

两阶段权限需要同时成立。Guest 页表可以允许访问, 而第二阶段映射仍拒绝; 异常应被正确归因给 guest 或 hypervisor。VMID 与失效指令还必须区分两个阶段的作用域, 避免更新一层映射时无谓清除所有另一层缓存结果。

3.9 MMU and Cache Interaction

缓存地址组织与页面大小直接耦合。PIPT 缓存完全使用物理地址, 语义直接但必须等待翻译; VIPT 缓存使用页内位索引、物理标签比较, 可把一级缓存访问与 TLB 查询并行化, 但每路容量和索引位受到最小页面大小约束; VIVT 结构访问迅速, 却必须额外处理 homonym、synonym 与一致性维护。

因此, MMU 与缓存不能被简化为严格串行的两个模块。页面大小、ASID/VMID、缓存相联度、索引方式、页表缓存和失效协议共同决定取指与 load-to-use 路径。分析翻译性能时, 既要观察 TLB miss 本身, 也要观察 page walk 对缓存和内存系统产生的附加流量。

3.10 Hardware–Software Co-Design

虚拟内存是稳定的硬件–软件接口。硬件负责快速查询、页表遍历和权限验证; 操作系统负责地址空间布局、页面大小选择、页表维护、缺页处理和 TLB 一致性。硬件支持的大页、ASID、聚合翻译和精细失效只有被软件正确使用时才能形成收益, 而软件策略也必须服从硬件 TLB 容量、页表格式和维护代价。异构 SoC 与 I/O 设备共享虚拟地址时, MMU-aware DMA 和可扩展翻译共享还会把这一协同扩展到 CPU 之外。[11], [12]

MMU 的评价因而应同时覆盖稳态翻译吞吐和映射变化成本。大页使用率、地址空间切换、页表共享、虚拟化层次与 shutdown 频率都会改变同一组硬件结构的实际表现。其设计目标是在快速命中、较大覆盖、并行遍历、精确保护和可维护性之间取得平衡。

Chapter 4

Instruction Decode

4.1 Decode Pipeline

译码(Decode)位于指令字节供给与乱序后端之间,负责把 ISA 编码转换为处理器内部可重命名、分派和执行的表示。其工作可以分为三个阶段:识别指令边界与格式,提取寄存器和立即数等字段,以及生成一个或多个内部操作。不同实现可能把这些功能分散到预译码、对齐、主译码和微码序列中,因此“译码宽度”只有在说明输入粒度和输出粒度后才具有可比性。[1]

Decode 的持续吞吐受 ISA 编码、前端字节组织和后端接收能力共同限制。定长、字段位置规则的编码便于并行译码;可变长度、前缀较多或语义复杂的编码则需要更多边界识别和例外处理。扩大主译码器宽度会增加并行比较、布线和内部操作生成逻辑,因而常受到频率与能耗约束。

4.2 Instruction Boundaries and Encoding Cost

定长 ISA 的指令起始位置由固定对齐关系确定,取指块可以直接切分为若干候选译码槽位。支持压缩指令或可变长度编码时,前端必须先判断每条指令的长度,处理跨取指块或跨缓存行的残缺指令,再把字节重新排列到并行译码器。代码密度可以降低 L1 指令缓存与取指带宽压力,但更复杂的长度识别会把一部分成本转移到对齐和译码路径。

编码规则性还体现在字段位置和格式数量上。稳定的 opcode、源寄存器和目的寄存器字段有利于并行提取;分散的立即数字段、前缀修饰和大量特殊格式则需要更多组合逻辑。译码器必须在有限周期内生成后端所需的最小充分信息,包括操作类型、源与目的寄存器、立即数、访存宽度、分支位移、异常类别和内部执行属性。

定长编码也不意味着译码成本完全一致。系统指令、原子操作、向量配置和特权访问可能需要查阅额外控制状态,或者被送入专用译码路径。规则编码降低的是边界识别和字段定位成本,而不是保证所有指令都映射为相同数量、相同延迟的内部操作。

可变长度编码通常先并行猜测多个候选起点和长度,再通过前缀与 opcode 规则筛选合法边界。前一条指令长度决定后一条指令起点,因此完全串行判定会限制宽度;实际设计会使用预译码位、分段长度解码或多级对齐,把这一依赖分散到多个周期。

立即数生成也可能位于关键路径。分支位移、访存偏移和高位常量在部分 ISA 中由若干离散字段拼接,并需要符号扩展、移位或与 PC 相加。若分支目标希望在 Decode 附近尽早形成,立即数重构与目标加法就会直接影响重定向延迟。

4.3 Predecode, Alignment, and Queues

许多处理器会在 L1 指令缓存填入或取指过程中生成预译码(predecode)元数据,例如指令边界、长度类别、潜在控制流位置和特殊译码标记。预译码并不替代完整语义译码,而是把重复且适合提前完成的工作移出主译码关键路径。

对齐缓冲和指令队列负责把按 fetch block 或 cache line 返回的字节整理成完整指令。它们处理跨块拼接,并解耦取指与译码的短期速度差异。队列较浅时,L1I 边界、bank 冲突或短暂重定向很快会使 Decode 饥饿;队列较深时,前端可以吸收更多抖动,但错误路径也可能占用更多容量。

预译码元数据可以与指令缓存数据一起存储,也可以保存在独立阵列。与数据一起存储读取直接,但会增加每行宽度;独立存储便于更新和压缩,却需要保证两者索引与失效同步。若代码被修改,只失效字节而保留旧边界信息同样会产生错误。

Alignment buffer 不仅拼接跨块指令,还要删除 taken branch 之后的无效字节,并根据本周期可用译码槽位选择一段连续指令。多个预测基本块在同周期供给时,缓冲区可能需要把物理上不连续的字节组织成逻辑连续的内部序列。

Decode queue 的反压方向与取指队列相反。后端停顿时,译码后的操作无法继续进入 Rename, Decode queue 逐渐填满;队列满后主译码器停止,最终再让取指侧减速。队列能平滑短期速度差异,却不能修复长期 Rename 或 Dispatch 带宽不足。

4.4 Semantic Translation and Internal Operations

一条架构指令不一定对应一个内部操作。简单算术和访存指令可能直接映射为单个微操作(micro-op, μop);复杂指令可能被拆分为多个操作,或者进入微码控制的序列。反过来,若相邻指令能够被安全地视为一个调度实体,前端也可能通过融合减少后端压力。

拆分(cracking)会使内部操作数多于架构指令数,因此峰值“每周期译码指令数”不能直接等价为后端分派带宽。融合则存在不同层次:宏融合可以在译码前后把相邻架构指令组合,微融合可以让地址生成与数据操作在部分流水阶段共享一个表示。评价这些机制时,应明确融合在何处发生、在哪一阶段重新展开,以及退休计数采用架构指令还是内部操作。

内部操作格式决定 Rename、scheduler 和执行端需要携带多少信息。格式过于紧凑会把译码或解释工作推迟到后端,格式过于丰富则增加队列宽度、寄存器和传输能耗。Decode 因而不仅是语义转换,也是在前端复杂度与后端表示成本之间划分边界。

复杂指令可以由硬连线译码直接展开,也可以由微码序列器生成一串内部操作。硬连线适合高频、长度固定的展开,延迟低但控制逻辑随指令类型增加;微码适合低频复杂语义和可更新流程,却需要切换前端来源,并可能暂时占用主译码带宽或专用注入端口。

微码序列中的内部操作仍需参与重命名、调度和退休,某些临时值则可以使用不可见的内部寄存器。异常发生在序列中间时,处理器必须保证软件看到的是整条架构指令规定的精确语义,而不是已经执行一半的内部步骤。

宏融合通常要求特定相邻指令组合、对齐和控制流条件。例如,比较与条件分支可以共享部分语义,并减少进入后端的操作数;若中间存在目标边界、前缀或异常语义差异,融合可能被禁止。融合命中率因而受编译器代码生成和指令布局影响。

4.5 Control-Flow Information

控制流指令的类型和静态位移可以在预译码或主译码阶段得到确认, 但这不等于分支方向已经解析。方向预测器和 BTB 在更早阶段给出推测路径; Decode 主要负责确认指令是否确为条件分支、调用、返回或间接转移, 并生成后端执行该控制流操作所需的信息。

若早期预测把普通指令误识别为分支, 或者遗漏了真实控制流, 译码阶段可以触发前端校正。直接分支的静态目标也可能在这一阶段形成, 但条件是否成立通常仍需等待执行单元计算。必须区分**控制流类型确认、目标地址生成与方向结果解析**, 否则容易把 Decode 的作用扩大为完整分支验证。

Decode 还可能确认 call/return 类型, 从而校正 RAS 的投机行为。若 BTB 把普通间接跳转错误分类为 return, 前端可能使用错误的栈顶目标; 译码确认后需要修复类型状态, 并重新引导取指。此类错误与方向误预测不同, 来源是目标元数据或控制流类型不一致。

对于没有 BTB 命中的直接分支, Decode 获得完整指令后可以计算静态目标, 并把新条目训练进 BTB。首次遇到的控制流因此常在 Decode 才被识别, 后续执行才有机会使用早期目标预测。

4.6 Decoded-Operation Caches

微操作缓存或操作缓存保存已经译码的热点指令, 使命中路径绕过字节边界识别和主译码器。它可以降低前端能耗, 并在主译码宽度较窄或编码复杂时提高平均供给能力。然而, 该结构还需要处理控制流组织、标签、失效、自修改代码和与 LII 的一致性; 容量不足或代码路径频繁变化时, 前端仍需依赖常规译码路径。

操作缓存的输出宽度、主译码宽度和后端分派宽度可能不同。较宽的缓存读取只有在下游队列能够接收时才会形成收益; 若 Rename 或 Dispatch 长期受限, 额外输出只会增加反压和访问能耗。

操作缓存的组织粒度可以是单条指令、固定块或动态 trace。按静态块组织容易与 PC 索引对应, 但 taken branch 会留下块内无效操作; trace 组织可以把跨基本块的常见路径存为连续序列, 却需要路径标签和更复杂的填充、替换与失效。[13]

操作缓存通常保存已经确定的内部格式, 因此与特权状态、模式位或微码版本相关的指令需要特别处理。微码更新、控制寄存器变化或代码页修改可能要求失效相关条目, 避免继续使用旧的译码结果。

命中路径与主译码路径还需在合流点保持顺序。如果操作缓存和 LII 同时返回相邻 PC 的工作, 前端必须选择唯一来源, 并保证 FTQ、异常和预测元数据与内部操作正确对应。

4.7 Decode Throughput and Energy

Decode 的结构上限由每周期可识别的指令数或可生成的内部操作数描述, 持续吞吐还要扣除边界浪费、复杂指令、微码序列、缓存未命中和下游反压。固定的峰值宽度不能说明不同指令组合下的实际供给, 因为简单指令、融合指令和复杂拆分指令对译码资源的占用不同。

译码逻辑具有较高切换活动, 尤其在可变长度 ISA 和宽并行设计中。预译码、操作缓存、分簇译码器以及对空闲路径的时钟门控, 都是降低平均能耗的常见方法。其基本权衡是: 把更多状态与复杂度放入辅助结构, 以减少主译码器被频繁激活的次数。

Decode 的微架构位置最终由三组带宽关系决定: 取指侧能否稳定提供完整指令, 主译码路径能否及时转换这些指令, 后端又能否持续接收内部操作。任一关系长期失衡, 都会使其它阶段的

名义宽度无法转化为有效吞吐。

分簇译码器可以把宽输入划分给多个较窄 decoder, 降低单个逻辑块复杂度。其代价是指令需要按槽位或类型分配, 某一簇遇到复杂指令时可能阻塞同批次其它槽位。若各簇支持的指令集合不同, 还会出现结构性不对称。

复杂指令率、融合率和操作缓存命中率共同决定平均内部操作产出。相同的架构指令吞吐下, 更多 cracking 会增加 Rename 与 ROB 压力; 更高融合率则减少后端项数, 却可能使一个融合操作在执行或退休时重新占用多个资源单位。

Chapter 5

Register Rename

5.1 Register Renaming

寄存器重命名(register renaming)把 ISA 定义的体系结构寄存器名映射到数量更多的物理寄存器。体系结构寄存器描述软件可见状态,却会在程序中被反复覆盖;物理寄存器保存尚未退休的多个值版本,使处理器能够区分“使用同一名字”和“依赖同一数值”这两种关系。[14]

真正的数据依赖是写后读(read after write, RAW):消费者必须等待生产者生成值。读后写(WAR)与写后写(WAW)主要来自有限寄存器名字的重复使用。每次目的寄存器写入若获得新的物理目标,WAR 与 WAW 就不会继续约束执行顺序,调度器只需追踪物理标签之间的 RAW 关系。

例如,若程序依次执行对逻辑寄存器 x1 的读取、写入和再次写入,顺序机器必须保护所有名字关系。Rename 可以把两次写入分别映射到物理寄存器 p17 与 p23,较老读取仍指向此前版本,较年轻消费者则指向最近定义。这样,后一次写入不必等待更早读取完成,两次写入之间也不再形成真实依赖。

重命名只消除名字相关,不能突破真实数据流。如果第二条指令读取第一条刚产生的值,即使逻辑名字已经转换为物理标签,消费者仍必须等待对应生产者就绪。扩大物理寄存器数量能够容纳更多版本,却不会缩短 RAW 依赖链。

5.2 Architectural and Physical State

同一个体系结构寄存器在乱序窗口中可能同时存在多个物理版本。较老指令仍可能读取旧版本,较年轻指令则读取最近一次定义产生的新版本。寄存器映射表(Register Alias Table, RAT)给出当前投机路径上的逻辑到物理映射;部分实现还维护独立的退休映射,用于表示最近一次不可撤销的体系结构状态。

物理寄存器文件的容量限制能够同时保留多少未退休值。窗口越深、每周期产生的目的操作数越多,物理寄存器压力通常越大。不过,ROB、scheduler、load/store queue 和物理寄存器文件的容量并不必然相同,其中任何一种资源先耗尽,都可能阻止新指令继续进入后端。

物理寄存器还需要就绪状态。映射建立时,新目标只表示未来结果将写入何处,并不表示数值已经可用;生产者完成后,ready bit 或等价状态被设置,调度器再唤醒消费者。某些实现把 ready 状态放在集中表中,另一些由 scheduler 表项维护副本,两者需要与 replay 和恢复保持一致。

体系结构状态也可能包含整数标志、浮点状态、向量谓词和控制寄存器。频繁产生并参与普通数据流的状态适合重命名;具有全局副作用或序列化语义的控制状态则可能使用专门机制。是否重命名标志位会显著影响整数分支和条件操作之间的假相关。

5.3 Rename Structures

重命名路径通常包含映射表、空闲列表(free list)、旧映射记录和恢复检查点。对每条指令,硬件首先读取源寄存器当前对应的物理标签;若指令产生目的值,则从空闲列表分配新物理寄存器,保存被覆盖的旧映射,并更新投机映射表。旧物理寄存器不会立即释放,因为它仍可能被更老消费者读取,或者在当前投机路径被撤销时重新成为有效映射。

宽重命名器还必须处理同周期指令之间的依赖。若本周期较早的指令定义了某个逻辑寄存器,后续指令读取该寄存器时必须看到刚分配的新物理标签,而不是周期开始时的旧映射。这要求在批次内部按程序顺序解析覆盖关系,并提供短延迟旁路。随着宽度增加,映射表端口、空闲寄存器分配和同周期旁路的组合复杂度会迅速上升。

不同寄存器类别可以采用独立映射与物理文件,例如整数、浮点/向量、标志位或掩码寄存器。这样有利于按各自的读写需求组织端口和容量,却会使跨域操作、恢复和资源平衡更加复杂。

Free list 可以用位图、循环队列或分银行结构实现。位图容易并行寻找多个空闲项,但优先级编码会随寄存器数与分配宽度增加;循环队列分配顺序简单,却需要恢复尾指针和处理非顺序回收。多银行物理文件还可能要求分配器在银行间均衡目标,以减少未来读写冲突。

旧映射通常随目的指令保存在 ROB 或独立历史缓冲中。指令退休时,旧物理寄存器被回收;分支恢复时,错误路径上分配的新物理寄存器也必须归还。若恢复只修正 RAT 而没有同步修复 free list,硬件可能泄漏寄存器或重复分配仍在使用的项。

同周期旁路的复杂度与指令顺序有关。一个批次中可能连续多次写同一逻辑寄存器,最终 RAT 只保留最后映射,但中间指令的消费者仍需获得正确版本。Rename 必须在一个周期内构造局部依赖链,并为每条目的记录相应旧映射。

5.4 Move Elimination and Special Cases

若一条 move 只是复制已有寄存器值,重命名阶段可以让目的逻辑寄存器直接引用同一物理值,而不必把操作送入执行单元。这种 move elimination 能减少调度与执行压力,但会引入物理寄存器引用计数或等价生命周期管理问题,因为多个逻辑映射可能暂时共享同一物理值。

零寄存器、常量生成、部分寄存器写入和隐式操作数也会改变重命名规则。实现必须明确哪些操作真正产生新的完整值版本,哪些操作可通过常量标记、合并或专用依赖处理完成。尤其在允许部分寄存器更新的 ISA 中,新值可能依赖旧值的一部分,不能简单视为完全独立的新定义。

Move elimination 若让多个逻辑寄存器共享同一物理寄存器,回收时不能只依据某一映射被覆盖。实现可以维护引用计数、别名链或在后续写入时重新复制。引用计数更新本身又会增加 Rename 与退休端口,因此部分处理器只消除满足简单条件的 move。

常量零和某些立即数可以由特殊物理标签表示,而不占用普通寄存器文件。消费者看到该标签时直接获得常量值,减少分配和读端口压力。可表示的常量集合越大,标签和旁路逻辑越复杂,通常只对极高频常量采用。

5.5 Recovery and Checkpoints

分支误预测、异常和全局恢复会撤销错误路径上的映射更新。快速恢复通常依赖分支检查点,保存或增量记录当时的映射与空闲资源状态;另一种方法是沿 ROB 或历史缓冲逐项回滚。完整检查点恢复迅速但状态成本较高,逐项回滚节省存储却延长恢复时间,实际设计可以组合两者。

投机映射与退休映射的分离能够简化精确状态维护。正常执行更新投机映射, 指令退休时再确认对应的新映射; 发生精确异常时, 处理器可以恢复到最近的已退休状态。无论采用何种组织, 恢复后的映射、空闲列表和物理寄存器生命周期必须保持一致, 否则后端会读取错误版本或重复分配仍在使用的寄存器。

每个未解析分支都可能需要一个恢复点。直接复制完整 RAT 恢复快速, 但宽寄存器集合和大量在分支会增加存储; 增量检查点只记录分支之后发生的修改, 存储较少, 却需要在恢复时重建。部分实现保存 RAT 检查点, 同时从分配日志恢复 free list。

异常恢复与分支恢复的目标不同。分支误预测要回到分支之前的投机状态, 以继续执行正确路径; 精确异常则要回到异常指令之前的已退休状态。前者强调低延迟, 后者频率较低但必须覆盖全部体系结构状态。

Recovery bandwidth 也可能成为限制。一次恢复需要使错误路径目标寄存器失效、归还空闲表项, 并清除 scheduler 与 ROB 中的年轻操作。前端生成正确 PC 后, 如果 Rename 状态尚未恢复完成, 正确路径仍无法重新进入后端。

5.6 Register Reclamation

目的指令退休后, 被其覆盖的旧物理寄存器通常才可安全回收到 free list。该规则实现直接, 但会保留一些已无消费者、尚未达到退休条件的值。提前回收或引用计数方案可以提高利用率, 却需要证明恢复时仍能找回正确旧值, 因此会增加依赖追踪和异常路径复杂度。

退休受阻会间接耗尽物理寄存器。即使执行端口仍有空闲, 若 ROB 头部长期被缓存缺失阻塞, 旧映射无法回收, Rename 最终会因 free list 枯竭而停止。这说明物理寄存器容量既是命名资源, 也是乱序窗口吸收长延迟事件的组成部分。

值的最后一次读取通常早于覆盖该值的新定义退休, 因此传统回收保留了安全余量。基于最后消费者的提前回收需要准确知道所有动态消费者, 并处理错误路径消费者和 replay; 一旦判断过早, 后续读取可能访问已被重新分配的物理寄存器, 造成难以恢复的错误。

物理寄存器 pressure 还与代码生成有关。更多临时值、较长 live range 和较宽向量会提高占用; move elimination、融合和值直接转发可以减少部分需求。硬件容量与编译器寄存器分配并非完全独立, spill 减少了体系结构寄存器压力, 却会把流量转移到 LSU 和缓存。

5.7 Rename Scalability

Rename 的扩展难点来自其近似全局的映射视图。增加宽度需要更多读写端口、同周期依赖解析和空闲资源分配; 增加窗口深度则要求更多物理寄存器和检查点。分银行或分簇可以降低局部端口压力, 但跨簇依赖和映射一致性会引入新的延迟。

因此, 重命名器的实际设计目标不是最大化单一宽度, 而是在持续带宽、恢复速度、物理寄存器利用率、访问延迟和能耗之间平衡。它把顺序的体系结构名字转换成可乱序执行的物理数据流, 是后端调度、旁路与精确恢复共同依赖的基础。

SMT 进一步增加映射与物理寄存器管理难度。每个线程需要独立的体系结构映射和恢复状态, 物理寄存器池则可以静态分区或动态共享。动态共享提高利用率, 却可能让一个线程耗尽空闲寄存器并阻塞另一线程; 保留最低配额可以保证进展, 但减少峰值单线程容量。

Rename 的访问延迟通常位于前端到后端的高活动路径。提高频率可能要求把映射读取、批次旁路和资源分配拆成多个流水级; 流水化能够缩短每级逻辑, 却增加分支恢复距离, 并要求在中

间级保存更多批次状态。

Chapter 6

Dispatch

6.1 Dispatch and Window Allocation

分派(dispatch)把完成重命名的内部操作送入乱序后端,并分配其继续执行所需的在途资源。不同处理器对 Rename、Allocate 与 Dispatch 的流水级命名并不统一:ROB、scheduler、load queue 或 store queue 的表项可能在同一阶段分配,也可能分散在相邻阶段。因此,分析某一实现时应先确认阶段边界,而不能把所有资源分配都视为 Dispatch 的固定定义。[1]

从功能上看,分派完成两件事。第一,它为操作建立程序顺序与恢复身份,例如分配 ROB 项、记录异常和旧映射信息;第二,它把操作放入等待执行的结构,例如整数或向量 scheduler、load queue、store queue 以及专用队列。完成这些分配后,指令才真正成为乱序窗口中的在途工作。

分派还会携带由 Decode 与 Rename 产生的元数据,包括操作类型、源物理标签、目的标签、立即数、预测信息和异常标记。不同后端结构不一定保存完整副本:ROB 关注顺序和恢复, scheduler 关注操作数与端口, LSQ 关注访存年龄和地址。Dispatch 需要把同一操作的信息正确拆分到这些结构中。

融合操作会使分配粒度更复杂。一个融合实体可能在 scheduler 中占一项,在退休时对应两条架构指令;访存操作也可能拆分为地址和数据子操作,分别使用执行端口但共享 ROB 或 SQ 身份。比较分派宽度时必须确认计数单位。

6.2 Coupled Resource Allocation

一条普通整数操作可能只需要 ROB 与某个调度队列项,load 还需要 load queue 项,store 则可能需要 store queue、store data 与排序相关状态。若一个操作所需的任一结构没有空闲项,该操作及同批次中更年轻的工作通常无法继续分派。

分派带宽因此由最紧张的资源决定,而不是由各结构平均容量决定。整数代码可能首先受 scheduler 或物理寄存器限制,访存密集代码可能先耗尽 load/store queue,含大量未解析分支的代码则可能受检查点数量限制。ROB 很深并不意味着所有类型的操作都能使用完整窗口。

宽分派还需要在一个周期内更新多种尾指针、空闲计数和配额,并保证部分失败时不会留下不一致状态。若一个批次只能分配部分操作,硬件必须定义是否允许部分接受,还是整体停顿;该选择会影响控制复杂度与带宽利用率。

整体接受策略保证批次边界清晰,但批次末尾一条需要稀缺资源的操作可能阻止前面本可进入的操作。部分接受提高利用率,却需要保存剩余指令、更新多个结构的不同尾指针,并确保恢复时仍能对应同一程序顺序。

某些资源可以超额订阅或延迟分配。例如, 简单已就绪操作可能绕过长期驻留 scheduler, 直接进入短队列; 某些 store data 项也可在数据实际就绪时再占用专用缓冲。延迟分配减少平均占用, 却会增加后续阶段无法获得资源时的处理复杂度。

检查点和恢复标记同样是入口资源。即使 ROB 与 scheduler 都有空间, 未解析分支过多也可能耗尽 checkpoint; 包含序列化指令或微码序列的批次还可能要求停止更年轻操作进入窗口。

6.3 Dispatch and Issue

Dispatch 与 Issue 解决不同问题。Dispatch 判断操作是否具备进入后端的结构资源; Issue 则在操作数就绪后选择何时、向哪个执行端口发射。一个已经分派的操作可以在 scheduler 中等待多个周期, 而一个操作即使源操作数已经就绪, 也可能因缺少 ROB、LSQ 或其它入口资源而无法进入窗口。

这种区分解释了两类不同停顿: 入口停顿通常表现为 Rename/Decode 被后端反压, 发射停顿则表现为操作已经在窗口中, 却因依赖、端口或执行资源冲突无法前进。将两者都称为“调度受限”会掩盖瓶颈发生的位置。

Dispatch 也可能执行初步 steering, 把操作送入某个 scheduler cluster 或端口组。该决策发生时, 未来端口负载和操作数位置尚不完全确定; 错误 steering 不会破坏正确性, 却可能让某个队列拥塞或增加跨簇数据传输。

已就绪操作可以采用 dispatch-to-issue bypass, 在进入窗口的同周期或下一周期直接竞争端口。这能降低独立简单操作的最短延迟, 但需要与更老就绪项、公平性和恢复状态协调。若旁路失败, 操作仍需正确落入 scheduler, 不能因尝试快速路径而丢失。

6.4 Backpressure and Window Balance

后端资源不足会通过分派向前传播。若 ROB、scheduler 或 LSQ 接近满载, Dispatch 会降低接收宽度或完全停止, 随后 Rename、Decode 和取指队列依次积压。反压信号必须及时而稳定; 过晚会造成超额分配, 过早则会留下本可利用的空闲容量。

窗口结构之间需要保持合理配比。扩大 ROB 只有在 scheduler、物理寄存器和访存队列也能容纳足够工作时才有意义; 扩大某个专用队列则可能只改善特定指令组合。分区可以降低端口和时序压力, 却会产生容量碎片: 一个队列空闲时, 另一个队列仍可能阻止入口继续工作。

反压通常需要提前预测资源是否将在批次分配后耗尽。仅在表真正填满时才反馈可能来不及阻止已经在流水中的操作; 预留若干 entries 可以简化时序, 却降低名义容量。宽而深的前端会使“停止信号在途”问题更明显。

资源阈值还可以根据线程或操作类型动态调整。访存压力高时, 系统可以减少新的 load/store 进入, 给已有 miss 留出 LSQ 与 MSHR; 分支恢复期间则可暂时停止新分配, 避免清除逻辑与正常写入争用同一端口。

Window balance 应以瓶颈工作负载而非平均指令比例判断。向量代码可能迅速耗尽向量 scheduler, 指针密集代码受 LQ 与物理寄存器限制, 短依赖整数代码则更看重入口和 issue 吞吐。统一扩大所有结构通常面积代价过高。

6.5 Recovery and Thread Sharing

分支恢复或异常清除会使部分在途项失效, 并回收相应资源。恢复速度不仅取决于前端重定向, 也取决于 ROB、scheduler 与 LSQ 能否迅速识别并删除错误路径项。精细的年龄或分支掩码有利于选择性清除, 但会增加每个表项的状态和广播成本。

在 SMT 核心中, 分派还决定线程如何共享窗口。完全动态共享能提高平均利用率, 却可能让一个线程耗尽关键结构; 静态分区提供隔离, 但会留下不可借用的空闲项。保留份额与有限借用的混合策略常用于在吞吐和公平性之间折中。

恢复清除可以按 ROB 年龄、分支掩码或 checkpoint generation 识别年轻项。按年龄比较精确但需要广播边界, 分支掩码允许表项快速判断是否依赖某个未解析分支, 却会增加每项位数, 并受最大在飞分支数量限制。

SMT 的入口仲裁还要决定每周期各线程获得多少 slots。轮转公平实现简单, 但可能把带宽分配给暂时没有高价值工作的线程; 基于队列占用、停顿原因或优先级的策略能提高吞吐, 却需要避免长期饥饿。

Dispatch 的本质是控制乱序窗口的形成速度。其有效能力取决于入口宽度、各类资源配比、反压传播和恢复回收, 而不是单一的“每周期分派数”。分簇和重构式 issue queue 研究也说明, 窗口入口、依赖组织与调度能耗必须共同设计。[15]

Chapter 7

Issue

7.1 Issue and Dynamic Scheduling

发射(issue)从已经进入乱序窗口的操作中选择本周期可以执行的候选, 并把它们送往兼容的执行端口。操作能否发射至少取决于三个条件: 源操作数已经就绪, 目标端口与功能单元能够接收新工作, 且不会违反访存或其它体系结构约束。动态调度的价值在于绕过等待长延迟事件的旧操作, 利用年轻但独立的工作填充执行资源。[16]

7.2 Ready-State Tracking

调度器为每个等待操作保存源物理标签和就绪状态。生产者完成时, 其结果标签通过完成总线、旁路网络或分簇通知唤醒消费者。消费者的所有必要源就绪后, 才成为可选候选。多源操作、预测值、load replay 和跨簇传输都会使“就绪”不再只是一个简单比特, 而可能包含预计可用时间和验证状态。

就绪通知必须与执行延迟匹配。对固定延迟 ALU, 调度器可以提前知道结果将在何时产生并安排相关操作; 对 cache miss、可变延迟除法或 replay, 结果时间无法简单静态确定, 需要在真实完成后再次唤醒。错误的延迟假设会导致消费者过早发射并重放。

调度器可以保存每个源的物理标签, 并在完成标签广播时进行关联比较; 也可以在 Rename/Dispatch 阶段建立 producer-consumer 链, 只通知明确的依赖者。前者灵活但比较能耗高, 后者减少广播却需要维护动态链表, 并处理 producer 被取消或重放的情况。

Ready bit 与数值所在位置是不同信息。源操作数可能已经生成, 却仍位于旁路网络、远端 cluster 或尚未写回的 load buffer。调度器若只看到逻辑就绪而忽略传输时间, 消费者仍可能在寄存器读取阶段等待。部分实现因此使用 operand availability 或预计到达周期指导发射。

同一生产者可能唤醒许多消费者, 形成高扇出广播。物理标签比较、匹配线和就绪位更新会在一个周期内产生大量切换, 因此 wakeup 是调度器能耗的重要来源。分银行、分簇和依赖过滤主要试图减少每次完成事件需要触碰的表项数。

7.3 Wakeup and Select

wakeup 阶段识别哪些操作刚刚获得全部源, select 阶段则从就绪候选中挑选可占用端口的操作。集中式调度器能够在较大候选集合中灵活选择, 但标签比较、优先级仲裁和广播网络会随窗

口与宽度快速增长。分布式或分簇调度器降低局部复杂度, 却需要在分派时完成 steering, 并承担跨簇依赖延迟。

发射宽度只是 select 的结构上限。若候选操作集中在少数端口、操作数尚未到达, 或者某些端口被长启动间隔操作占用, 实际发射率会显著低于名义宽度。调度器容量同样不能直接等价为可利用窗口: 容量只有在条目能够及时获得操作数并映射到空闲端口时才有价值。

Wakeup 与 select 可以在同一周期串行完成, 也可以流水化。单周期 wakeup-select 能让一周期 ALU 的依赖链连续发射, 却形成较长组合路径; 分成多个周期有利于频率, 却增加依赖者启动间隔。设计者可以为最常见的简单整数路径提供快速旁路, 对其它操作采用较慢调度路径。

Select 需要同时解决端口冲突和年龄优先级。若两个操作都只能使用同一端口, 通常选择更老者; 若其中一个还能使用其它端口, 全局最优分配类似匹配问题。真实硬件多采用分层仲裁和局部优先级, 可能留下某些端口空闲, 即使理论上存在更好的组合。

年龄矩阵能够精确比较队列项相对年龄, 却随容量平方增长; 基于循环指针、分组 oldest-first 或分层编码的方案状态更少, 但选择精度下降。最老就绪优先的主要目的, 是推进较老依赖链并降低饥饿, 而不是保证每周期执行组合全局最优。

7.4 Port Binding and Steering

每种操作只能在支持其功能的端口上执行。完全灵活的端口映射有助于均衡负载, 却需要更复杂的选择与旁路网络; 较早绑定或专用队列能简化时序, 但错误 steering 可能让一个簇拥塞而其它资源空闲。

分簇设计还必须决定物理寄存器与旁路的组织方式。若生产者和消费者位于不同簇, 结果可能需要额外周期跨越网络; 若复制寄存器或广播到所有簇, 端口和能耗又会增加。因此, 端口表面数量相同的两个核心, 可能因 steering 和跨簇代价而表现出不同的有效吞吐。

端口绑定可以在 Dispatch 时静态完成, 也可以在 Issue 时从候选集合动态选择。早绑定减少每周期仲裁复杂度, 但无法适应后续负载变化; 晚绑定保持灵活性, 却要求一个操作同时参与多个端口选择器, 并防止被重复发射。

复制执行能力能减少热点。例如, 简单整数加法可以在多个端口执行, 而乘法或分支只存在于少数端口。编译器看到的是统一 ISA, 调度器则必须把不同操作组合映射到不对称资源。某些看似“整数端口很多”的核心, 在混合乘法、移位和分支代码中仍可能集中冲突。

Memory operations 的 steering 还要考虑 AGU、load/store 类型和 cache pipeline。一个端口可以生成 load 与 store 地址, 但 store data 可能走另一端口; 将“有几个 AGU”直接解释为同等数量的 load 完成率, 会忽略后续 DTLB、L1D 和 LSQ 限制。

7.5 Scheduling Policy

最老就绪优先是一种常见基线, 因为它有助于推进 ROB 头部并减少饥饿; 但实际选择还可能考虑端口兼容性、分支解析、关键依赖链和公平性。策略必须保持低延迟, 不能为了接近全局最优而把复杂决策放入每周期关键路径。

年龄、公平和端口利用率之间并非总是一致。持续偏向某类操作可能延后分支解析或阻塞另一条关键链; 过度平均又可能牺牲队首进展。因而, 实际调度策略通常是局部、近似且受硬件结构约束的, 而不是对整个依赖图求解最优计划。

关键路径优先在理论上可以缩短程序执行时间, 但动态识别关键链需要观察 ROB 位置、依赖

扇出、cache miss 和后续消费者。过于复杂的关键性估计不适合进入每周期 select, 实践中更常使用年龄、队首距离、分支类型或 load 依赖等低成本代理信号。

长期未发射的操作需要防止饥饿。若端口持续被更高优先级候选占用, 可以通过年龄提升、轮转或保留周期保证进展。公平机制不只服务 SMT, 也避免同一线程中的低优先级操作永久占据 scheduler 项。

7.6 Replay and Speculative Scheduling

load 命中预测、内存相关性预测和可变延迟执行会让操作在结果尚未完全确认时被投机调度。若预测成立, 消费者可以提前推进; 若预测失败, 相关操作必须重新进入等待状态。选择性 replay 可以保留不受影响的工作, 却需要追踪预测结果的依赖传播。

重放过于频繁会形成 replay storm: 同一依赖链反复占用调度器、寄存器端口和执行单元, 却没有增加退休进度。因此, 调度器需要与 LSU 和完成网络协同, 对重复失败进行节流, 并避免未经验证的结果扩散过远。

Load-hit speculation 是典型例子。调度器可以假定 load 将按 L1 命中延迟返回, 并提前安排消费者; 命中时依赖链无气泡, miss 时消费者必须取消或重放。若所有 load 都激进预测, 缓存压力高时会产生大量无效执行; 置信度或保守调度可以降低这种代价。

消费者被重放时, 其更年轻依赖者可能也已执行。设计可以只标记直接消费者重新等待, 并让错误值继续传播的操作在验证点被逐步捕获; 也可以维护依赖链并一次性失效全部后继。前者状态简单但可能多次 replay, 后者恢复精确但追踪成本高。

执行端口本身也可能拒绝已经选择的操作, 例如 bank conflict、未准备好的 store data 或跨簇 operand 延迟。调度器需要获得失败反馈, 并确保该操作重新进入候选, 而不是错误地标记完成。

7.7 Scheduler Organization and Scalability

传统完全乱序队列提供较高选择自由度, 但 wakeup-select 是后端显著的面积和能耗热点。研究设计常通过级联队列、就绪度分区、依赖链 steering 或部分有序结构降低比较和仲裁成本; 这些方法的共同目标, 是在减少每条操作参与全局选择的次数时, 保留足够的乱序机会。[15]

Issue 的实际能力由调度器容量、唤醒延迟、端口兼容性、操作数网络和 replay 行为共同决定。单独比较 issue width 或队列项数, 无法判断后端是否能够持续把有价值的工作送入执行单元。

Unified scheduler 允许不同操作共享容量, 适合动态指令比例变化; separate integer、vector 和 memory queues 减少端口和标签宽度, 却产生容量碎片。混合组织可以在大类之间分离, 在每类内部再共享多个端口。

Issue queue 的表项生命周期从 Dispatch 开始, 到操作成功发射或完成后结束。若操作发射后可能 replay, 硬件可以保留原表项, 或把它移入 replay queue。保留简化恢复但延长占用, 移出释放容量却需要重新注入路径。

调度器扩展最终受物理实现约束。更深队列增加可见独立工作, 但标签线、优先级网络和旁路距离也变长; 更多端口提高峰值吞吐, 却使每个候选参与更多选择逻辑。分簇并不是纯粹的性能优化, 而是让宽后端在频率、面积和能耗上可实现的基本手段。

Chapter 8

Execution Units (IXU/FPU)

8.1 Execution Back End

执行后端接收调度器发射的内部操作,完成整数、分支、浮点、向量和专用运算,并把结果写回物理寄存器或转发给依赖操作。分析执行资源时必须区分**发射端口**、**执行流水线**与**功能单元**:一个端口可以连接多种功能单元,多个端口也可能共享同一复杂流水线,功能单元数量不能直接相加为每周期可执行操作数。[1]

8.2 Latency, Throughput, and Initiation Interval

操作延迟(latency)表示结果何时可被依赖者使用;吞吐或启动间隔(initiation interval)表示同类流水线多久能接收一条新操作。一个乘法器可以具有多周期延迟,却每周期接受新乘法;除法器则可能既延迟较长,启动间隔也大于一周期。性能分析必须同时考虑这两个量。

关键依赖链主要受延迟限制,多个独立操作则更受启动间隔和流水线数量限制。扩大执行单元数量可以提高独立操作吞吐,却不能缩短单条串行链;降低单条操作延迟有助于关键链,但可能要求更复杂电路或更低频率。

例如,一条整数加法可能具有 1-cycle latency 和 1/cycle initiation interval;乘法可能延迟 3–4 个周期但仍可每周期启动;整数除法则可能具有数据相关延迟,并在多个周期内独占单元。三者都可被称为“一条执行管线支持”,实际调度行为却完全不同。

吞吐还需要区分单个端口与整个核心。若两条端口均支持加法,整体加法吞吐可以高于单端口启动率;若乘法只在其中一条端口执行,混合加法与乘法会竞争该端口。端口表是理解指令组合吞吐的基础,但仍需结合寄存器和旁路带宽。

某些操作的延迟取决于操作数。前导零、移位量、除数、浮点异常路径或向量掩码可能改变内部迭代次数;厂商延迟表常给出常见或最短路径,性能模型需要注意可变延迟和序列化情况。

8.3 Integer and Branch Execution

整数后端通常包含简单 ALU、移位、乘除、位操作和分支执行能力。简单加减与逻辑操作容易复制成多条低延迟流水线;乘法、除法和复杂位操作面积更大、延迟更长,通常只配置在部分端口。端口兼容性决定混合指令流能否均匀使用资源。

分支执行单元负责计算条件与目标并验证前端预测。较多分支端口可以减少分支与普通整数操作的竞争,但误预测代价还取决于条件依赖链、检测时刻、检查点恢复和正确目标重新进入前

端的时间。分支预测带宽与分支解析带宽必须配合,前者生成未来路径,后者及时确认这些路径。

简单 ALU 通常处理加减、逻辑与比较,并可能同时承担分支条件或地址辅助运算。若分支和普通 ALU 共用端口,控制流密集代码会与整数计算竞争;专用分支端口减少这种冲突,却仍需读取源寄存器并把解析结果送回前端。

移位和位操作可以集成在简单 ALU 中,也可以使用专用单元。复杂旋转、位域提取、计数前导零和加密辅助指令对组合逻辑要求较高,若所有端口都复制会增加面积;只在少数端口提供则形成特定指令热点。

乘法器可以流水化,面积和能耗显著高于简单 ALU。乘加或宽乘法可能产生两个目的结果,占用更多写回端口。除法器常采用迭代算法,长时间占用单元,但调度器仍可在其它端口执行独立工作。

分支解析结果包括方向是否正确、目标是否正确以及可能的类型修正。直接分支目标可较早形成,间接目标可能依赖寄存器 load;后者在错误时通常已经允许更多错误路径工作进入窗口。

8.4 Floating-Point and Vector Execution

浮点与向量后端通常采用更深流水线和更宽数据通路。其吞吐取决于向量数据通路宽度、支持的操作类型、FMA 数量、置换与类型转换能力,以及 load/store 路径能否提供足够数据。体系结构向量长度与内部执行宽度应当区分:一条较宽向量指令可以由较窄流水线分多个片段完成,也可以由多条并行管线一次处理。

置换、跨 lane 操作、除法和特殊函数常具有不同于普通加法或 FMA 的端口与启动间隔。仅引用“有几条向量管线”无法推出任意指令组合的峰值吞吐,还需了解端口映射、数据宽度和操作分解方式。

浮点加法、乘法与 fused multiply-add 可以使用独立或共享流水线。FMA 在一次舍入中完成乘加,既提高数值语义也提高计算密度;若加法和乘法都必须经过同一 FMA pipeline,独立简单操作仍会竞争相同端口。

向量寄存器文件需要提供远高于标量路径的数据带宽。一条三源向量指令可能在单周期读取数百比特数据并写回同等宽结果;复制读端口的成本很高,因此向量 scheduler、寄存器 bank 和执行 lane 往往物理对齐。跨 bank 或跨 lane 置换需要额外网络。

当体系结构向量长度大于内部数据通路时,一条指令会被分成多个 beat。它可能在 ROB 中仍占一个架构身份,却在执行端连续占用多个周期;依赖操作要等待全部结果还是可以按 lane 提前消费,取决于内部表示和旁路粒度。

掩码执行不会自动节省全部工作。即使只有少数 lane 有效,指令仍可能占用完整 issue slot、寄存器读带宽和流水线;更精细的稀疏执行需要额外压缩或 lane gating 逻辑。能耗收益和吞吐收益需要分别判断。

8.5 Operand Supply and Bypass

执行单元只有在操作数及时到达时才能维持吞吐。物理寄存器文件需要提供足够读写端口,旁路网络则把刚产生的结果直接送给下一条依赖操作,避免等待写回后重新读取。随着执行宽度增加,全连接旁路的连线数量、负载和能耗会迅速上升。

分簇寄存器文件、局部旁路和 operand buffer 可以降低物理复杂度,但会引入跨簇复制或额外传输周期。某些操作因此具有依赖其生产者位置的有效延迟,而不是固定的单一延迟。调度器

steering 与物理布局共同决定这种代价出现的频率。

寄存器读取可以发生在 issue 前、issue 后或执行流水线早期。提前读取允许操作携带数值等待端口, 减少寄存器端口峰值, 但 operand buffer 容量和错误发射会浪费能耗; 晚读取只为真正发射的操作访问文件, 却要求调度器准确安排端口和 bank。

旁路网络需要把多个生产者结果送往多个潜在消费者。完全交叉连接的复杂度近似随生产者与消费者数量乘积增长, 宽核心难以无限扩展。局部旁路、限制某些端口间直接转发或增加一周周期跨域延迟, 是常见物理折中。

写回端口也可能成为瓶颈。多个长延迟操作在同一周期完成时, 结果数量可能超过物理寄存器文件写端口; 执行单元需要错开完成、使用结果队列或让部分操作延迟写回。峰值 issue 并不保证峰值 completion。

8.6 Structural Hazards

即使操作就绪, 操作仍可能因端口冲突、寄存器 bank 冲突、写回带宽不足或共享功能单元占用而无法执行。复杂操作若长期占用非流水化单元, 还会阻塞后续同类操作。结构冲突通常与具体指令组合有关, 因此平均执行宽度不能描述最坏或常见的端口压力。

地址生成单元(Address Generation Unit, AGU)在功能上属于执行资源, 但其结果进入 LSU 的翻译、排序与缓存路径。AGU 数量只说明潜在地址生成率, 不等于 L1D 能够完成相同数量的访问; DTLB、cache banks、LSQ 和 store-data 路径可能具有不同带宽。

寄存器 bank conflict 是另一类结构冒险。若多个操作的源集中在同一 bank, 即使执行端口空闲, 也可能因读端口不足推迟; 通过物理寄存器分配均衡银行可以降低概率, 但数据依赖和 move elimination 会限制分配自由度。

执行流水线还可能共享前处理或后处理。例如, 多条向量端口共享一个 permute network, 多条整数端口共享一个除法器, 若只看前端 issue ports 会高估这些专用操作的并行度。

结果旁路与异常检测也会占用后级资源。浮点异常标志、整数溢出、分支恢复和特殊指令完成可能需要进入统一 completion network; 该网络的吞吐限制会使操作在功能计算结束后仍不能立即标记完成。

8.7 Physical and Energy Constraints

执行后端通常占据较大面积, 并具有高活动率。复制 ALU 相对容易, 复制多端口寄存器文件、全局旁路和宽向量数据通路则会显著增加布线与功耗。频率目标也会迫使设计者在流水级深度、旁路距离和跨簇延迟之间取舍。

因此, 执行后端的有效能力应从操作延迟、启动间隔、端口兼容性、操作数供给和结构冲突共同判断。资源数量描述能力集合, 持续吞吐则取决于调度器能否把真实指令组合映射到这些资源, 并及时提供数据。

宽数据通路的物理距离会直接影响频率。向量寄存器文件、FMA 和 load/store 单元若分布在核心不同区域, 跨域传输可能需要额外 pipeline stage。功能框图中的相邻模块不一定在版图上接近, 旁路延迟可能由布线而非逻辑决定。

Clock gating 可以在端口空闲时降低动态功耗, power gating 则可关闭更大功能块, 但唤醒延迟更高。很少使用的向量或加密单元若长期保持全速时钟会浪费能量, 按工作负载动态管理又可能使首次操作出现额外延迟。

执行后端的面积不能只按 ALU 数量估计。物理寄存器、多端口读写、scheduler、旁路和 completion network 常占据相当比例;复制简单 ALU 的成本可能远小于为其提供足够操作数与结果带宽的成本。

Chapter 9

Load/Store Unit (LSU)

9.1 Load/Store Pipeline

访存单元(Load/Store Unit, LSU)连接乱序执行窗口与存储层次,负责地址生成、地址翻译、内存相关性检查、缓存访问、store-to-load forwarding、缺失处理和内存顺序维护。与普通算术操作不同,load 与 store 既参与寄存器数据流,也可能影响其它核心和设备观察到的共享状态,因此 LSU 的投机空间受到精确异常与内存模型约束。

一条访存指令至少包含若干可能具有不同带宽的步骤:由 AGU 形成有效地址,查询 DTLB 与权限,分配或查询 load/store queue,判断与更老 store 的关系,访问 L1D 或执行转发,并在数据返回后唤醒消费者。“load 路径数量”可能只对应其中部分步骤,不能自动推出相同数量的完整 load-to-use 吞吐。

访存操作通常在 Dispatch 时获得程序顺序和 LQ/SQ 项,在 Issue 时等待地址源操作数就绪。地址生成后,load 可以进入翻译与缓存快路径;store 的地址和数据可能由不同操作分别产生,并在 SQ 中汇合。一个 store 地址较早就绪但数据较晚到达时,年轻 load 已经可以利用地址进行相关性判断,却未必能够获得转发数据。

Load 的完成和退休也需要区分。数据返回后,它可以唤醒依赖指令并在物理寄存器中保存结果;如果后来发现内存顺序违例、权限异常或一致性条件变化,结果仍需撤销。Store 的生命周期更长:地址与数据生成、供年轻 load 转发、指令退休、获得缓存行写权限和写入 L1D,可能发生在不同周期。

9.2 Address Generation and Translation

有效地址通常由基址、索引、比例和位移组成。简单地址模式可以在一条 AGU 中低延迟完成,复杂或跨页访问可能需要额外操作。地址越晚形成,DTLB、别名检查和缓存访问越晚开始,也越难隐藏后续延迟。

DTLB 查询与 L1D 访问可在 VIPT 缓存中部分并行,但最终命中仍需要物理标签和权限确认。跨缓存行、跨页、未对齐访问或不同内存类型可能被拆分为多个内部操作,并增加 replay 与异常处理复杂度。

常见地址模式为 $\text{base} + \text{displacement}$ 或 $\text{base} + \text{index} \times \text{scale} + \text{displacement}$ 。复杂缩放、更新基址或多个加数会增加 AGU 路径;部分处理器让简单地址使用低延迟 AGU,复杂地址先由整数 ALU 形成中间值。编译器选择的寻址模式会直接影响 AGU 压力。

跨页访问必须分别翻译两侧页面,并保证两部分权限和属性均允许。第一部分命中而第二部分缺页时,处理器不能只提交部分结果。实现常把访问拆成多个内部操作,并在异常与退休逻辑中重新维持一条架构指令的原子语义。

未对齐但位于同一缓存行内的访问可以通过移位与拼接完成;跨行访问通常需要两个 bank、两个端口或多个周期。若两部分命中不同缓存层级,完成时刻和 replay 管理会进一步复杂化。

翻译结果中的内存属性还决定访问能否进入普通 L1D。设备内存、不可缓存区域和强顺序区域通常走更保守路径,并限制投机、合并与重放。

9.3 Load Queue and Store Queue

Load queue (LQ) 记录尚未退休的 load,用于检测其是否越过了后来解析出同址的更老 store,并保存重放与顺序信息。Store queue (SQ) 保存尚未提交的 store 地址和数据,使年轻 load 能够查询并获得转发。部分实现把已退休、等待写入缓存的 store 放入独立 store buffer,也有实现让相关功能在连续结构中完成;分析时应区分投机排序状态与已提交写入队列。

LQ/SQ 容量限制在飞访存数量,搜索带宽与分段组织则影响每周期可比较的地址数。完全关联比较提供灵活性,但在大队列和宽访存后端中成本很高,因此实现可能采用 bank、分层比较、哈希过滤或预测降低搜索压力。

LQ 项通常保存地址有效位、执行状态、访问宽度、异常和 replay 信息。Load 执行后仍需保留项,直到所有可能相关的更老 store 地址已经确定,或者该 load 已经退休;否则硬件无法检测迟到的同址 store。

SQ 必须从所有重叠候选中选择程序顺序上最年轻的旧 store。若最近 store 只覆盖 load 的部分字节,硬件可能组合多个 store,或把未覆盖部分从 L1D 读取。完全支持任意部分覆盖会使比较、年龄选择和数据拼接网络迅速复杂化。

大队列常使用分层过滤。页内偏移或地址低位先排除明显不相关项,再对少量候选比较完整物理地址;地址尚未翻译时,虚拟别名可能要求保守等待。过滤器允许假阳性造成额外等待,却不能产生遗漏真实依赖的假阴性。

SMT 核心可以动态共享 LQ/SQ,也可以为每个线程保留配额。动态共享提高单线程容量,却可能让一个线程的长延迟 miss 长期占用大部分项;固定分区保证进展,但产生容量碎片。

9.4 Memory Dependence Prediction

年轻 load 常在部分更老 store 地址尚未生成时到达。保守等待能够避免顺序违例,却会丢失大量内存级并行性;预测为独立并提前执行能够降低延迟,但若更老 store 后来解析到重叠地址,该 load 及其依赖者必须重放。

内存相关性预测器利用 load/store PC、历史违例或依赖集合判断哪些 load 应等待哪些 store。其目标不是预测任意地址,而是在“过度等待”和“过度投机”之间找到平衡。预测准确率、store 地址产生时间与选择性 replay 粒度共同决定净收益。[17]

最保守策略让 load 等待所有更老 store 地址已知,正确但会让一个慢 store 阻塞大量独立访问。最激进策略立即发射 load,只在违例后恢复;当别名频繁时,重复 replay 会消耗比等待更多的执行资源。

Store-set 一类机制把曾发生依赖的 load 和 store 归入同一集合,使 load 只等待集合中最近的未完成 store。集合别名会让无关操作被错误关联,集合过细则可能遗漏新关系,因此仍需在训练

速度与过度串行化之间折中。

预测器通常在真实违例后训练,首次遇到的相关性仍需付出恢复代价。程序 phase 改变后,旧关系可能不再成立;老化、替换和上下文标记决定预测器能否及时适应。

依赖预测只回答 load 是否应等待,不必预测最终地址。更激进的 memory renaming 或 store-index prediction 会直接猜测生产者身份,可能缩短 store-load 链,但必须验证不存在更年轻的其它同址 store。

9.5 Store-to-Load Forwarding

当 load 读取尚未写入 L1D 的更老 store 数据时,LSU 可以直接从 SQ 或 store buffer 转发。成功转发需要确认地址范围、字节覆盖、数据就绪和年龄关系;部分重叠、跨行访问、不同宽度或数据尚未完整生成都会增加匹配和拼接难度。

转发路径可能比普通 L1D 命中更短,也可能因关联搜索和数据选择而更长。某些研究提出预测 store-load 生产者关系或更早提供转发值,以缩短依赖链,但这类机制需要严格验证,不能视为普通 LSU 的普遍“零周期 load”能力。[18]

错误转发或遗漏相关 store 会产生内存顺序违例。处理器必须撤销受影响的 load 结果,并重新唤醒其依赖链。若重放范围过大或同一 load 反复失败,执行资源会被大量无效工作占据。

完全覆盖是最简单情况:一个旧 store 已准备好 load 所需全部字节。部分覆盖可能要求从多个 store 合并,或从 SQ 与 L1D 各取一部分数据。若组合网络过于复杂,处理器可以选择让 load 等待 store 提交到缓存后重新执行。

访问宽度和对齐决定 byte mask。窄 store 向宽 load 转发、跨边界访问和向量 load 都需要按年龄选择正确字节;即使地址匹配,store data 尚未就绪时也不能完成转发。

转发路径包括关联搜索、年龄选择和数据多路选择,延迟可能随 SQ 深度增加。预测 store queue index 可以缩短常见路径,但错误索引仍要由完整比较验证。

Store-to-load forwarding 只处理同一核心尚未写入缓存的旧 store。其它核心提供的新值由缓存一致性处理,已经排入本核 L1D 的 store 则通过普通缓存命中被观察。

9.6 Cache Access and Miss Handling

L1D 命中延迟不仅由 SRAM 决定,还包括地址形成、翻译、bank 选择、标签比较、数据选择和旁路返回。多条 load 同时访问同一 bank、跨行访问或与 store 写入竞争时,实际吞吐可能低于端口数量。

L1 miss 需要分配 MSHR 或等价的未命中跟踪项。若多个请求访问同一缓存行,可以合并为一个下层事务;访问不同缓存行则需要足够的 MSHR、填充缓冲与互连带宽维持并行。MSHR 数量决定可追踪的缺失上限,但核心能否利用这些表项还取决于 load queue 容量和地址依赖是否允许产生足够多的独立请求。

数据回填后,LSU 需要唤醒等待该行的 load 及其消费者。选择性唤醒能减少无关重放,代价是更精细的依赖记录;较粗粒度 replay 实现简单,却可能让大量已经执行过的操作重新竞争调度与端口。

同一 cache line 上的多个 load miss 可以合并到一个 MSHR,并分别记录请求字节和消费者。Critical word 返回后,部分 load 可以提前完成,其余数据继续填充;MSHR 需要保存足够子请求信息,以便正确唤醒所有等待者。

Load miss 的实际返回层级可能是 L2、LLC 或内存, 延迟并不固定。调度器通常在数据真实返回时唤醒消费者; 若依据预测命中层级提前调度, 预测错误就会产生 replay。

等待期间可能发生一致性失效或权限竞争。Store miss 在获得写权限前会长期占用 SQ 与 MSHR, load 则需要确保读取结果在一致性顺序上仍然有效。

Prefetch 可以提前创建 miss 请求, demand 到达后与其合并并提高优先级。为了防止预取耗尽全部资源, 缓存通常为 demand 保留 MSHR 或提供抢占规则。

9.7 Ordering and Visibility

load 可以在退休前投机执行, 但其结果必须能够在分支错误、异常或内存违例时撤销。store 的地址和数据也可以提前生成, 却不能在仍可能被撤销时对共享缓存产生不可恢复影响。store 退休后通常进入已提交状态, 再由 store buffer 按缓存端口、一致性和排序条件逐步排出; “已退休”不等于“已经全局可见”。

内存模型决定哪些访问次序必须被其它观察者感知。栅栏、原子读改写、acquire/release、设备内存和不可缓存区域会限制 load 绕行与 store 排出。LSU 负责实现这些顺序约束, 缓存一致性负责同一地址副本的权限与值传播, 两者相关但并不等同。

Store buffer 允许已退休 store 与后续执行解耦。不同地址的 load 在模型允许时可以绕过等待中的 store, 同址 load 则必须转发。Fence 可能要求旧 store 达到指定可见点, 因此其延迟取决于 buffer 占用、缓存端口与一致性权限。

原子读改写需要在读取旧值和发布新值之间维持不可分割语义。实现可能先获得缓存行独占权限, 再由 L1D 或专用单元完成更新; 高竞争时, 性能主要受 cache line bouncing 和重试限制。

设备访问可能具有不可重复副作用, 不能像普通 load 一样随意 replay。处理器通常在发起前确认其已越过必要投机边界, 并使用更强的排序规则。

9.8 LSU Performance Limits

LSU 的有效能力由地址生成、翻译端口、LQ/SQ、相关性预测、转发、L1D banks、MSHR 和回填唤醒共同决定。较多 AGU 无法弥补 DTLB 或缓存端口不足, 较深的 LQ 也无法突破地址依赖链, 较多 MSHR 则只有在核心能够生成独立缺失时才有价值。

因此, load-to-use latency 应理解为整条命中路径的结果, 访存吞吐则应理解为不同访问类型在多个阶段上的可持续完成率。LSU 的设计目标, 是尽早执行可证明或高置信度独立的访问, 同时把错误投机、重放和共享状态可见性控制在可验证范围内。

典型 load 时间线可以概括为: Dispatch 分配 LQ, Issue 等待地址源, AGU 形成虚拟地址, DTLB 与 L1D 执行快路径, SQ 完成别名与转发判断, 数据返回并唤醒消费者, 随后持续接受顺序验证直至退休。任何环节增加周期, 都可能反映到实际 load-to-use。

典型 store 则经历 SQ 分配、地址与数据分别生成、向年轻 load 提供转发、退休后进入已提交状态、等待缓存端口和写权限、最终写入 L1D。Store throughput 可能受 AGU、store-data 端口、SQ、store buffer 或一致性中的任一环限制。

空载延迟与压力吞吐必须分开。单条 load 可以走最短路径, 多条并发访问会暴露 bank、TLB、SQ 搜索和写回冲突; 大量 miss 则把瓶颈推向 MSHR、互连和内存。一个核心可能拥有很低的单次 L1D 延迟, 却在并发访存下较早饱和。

Chapter 10

Retirement and Precise State

10.1 Retirement as the Architectural Boundary

乱序执行允许指令以不同于程序顺序的次序完成,但软件必须观察到符合 ISA 规则的体系结构历史。退休(retirement)把已经完成且不再需要撤销的指令按程序顺序提交到体系结构状态,是投机执行与软件可见状态之间的边界。部分文献使用 commit 表示相近概念;本书以“退休”描述指令越过该边界,并单独说明 store 的内存可见过程。[1], [19]

一条操作执行完成,只表示功能单元已经产生结果或异常状态;若它之前仍有未完成指令,它仍必须停留在在途结构中。年轻操作可以使用该结果继续投机执行,但这些结果在退休前都必须能够被撤销。

退休边界保证软件看到的是完整程序前缀。假设第 N 条指令发生异常,则前 $N - 1$ 条指令应当已经生效,第 N 条及之后不应留下体系结构结果。乱序执行可以让第 $N + 20$ 条指令先完成,却不能让它越过异常指令对软件可见。

这一边界也定义资源生命周期。新物理映射、ROB 项和投机 store 在进入后端时被分配,只有相关指令退休后,旧映射和顺序状态才可回收。退休停顿因此会通过资源耗尽逐步反压整个流水线。

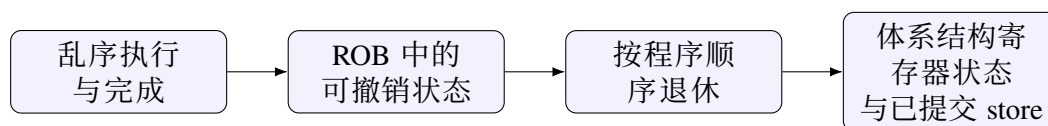


Figure 10.1: 执行结果先停留在可撤销状态中。寄存器结果越过退休边界后成为体系结构状态;已退休 store 仍可能等待缓存端口、一致性权限与规定的内存可见点。

10.2 Reorder Buffer

重排序缓冲区(Reorder Buffer, ROB)按程序顺序保存尚未退休的指令或内部操作。表项通常记录完成状态、异常、分支与恢复信息,以及退休时需要处理的资源元数据。采用独立物理寄存器文件时,结果值不一定保存在 ROB 中;ROB 的核心责任是追踪顺序与状态,而不是必然充当结果缓冲。

ROB 头部是最老未退休工作,尾部接收新分派的操作。退休逻辑只能越过从头部开始连续完成且无阻塞条件的若干项。ROB 容量决定可保留的程序顺序范围,但不等于 scheduler、物理寄存

器或 LSQ 的容量;这些结构中的任一项都可能先限制有效窗口。

融合与拆分会影响 ROB 计数口径。一个表项可能对应架构指令、宏操作或微操作,退休宽度也可能按不同粒度统计。因此,比较不同处理器的 ROB 与退休参数时,必须先确认内部操作和计数单位。

ROB 可以采用集中式循环队列,也可以把完成、异常和恢复信息分散到多个结构中,再由顺序队列维护退休次序。现代资料仍常用“ROB 大小”概括在途顺序容量,但具体表项保存哪些数据并不统一。

每个表项可能记录目的寄存器旧映射、分支检查点编号、load/store 身份、异常码和调试状态。状态越丰富,恢复与退休越直接,但 ROB 面积和访问端口越大。部分元数据可以放在 Rename history、LSQ 或专用队列中,换取更复杂的跨结构协调。

完成位由执行单元或 LSU 返回。一个架构指令拆成多个微操作时,只有全部必要子操作完成,退休端才能越过该指令;融合操作则需要在退休时正确增加架构指令计数,并保持异常只在合法边界报告。

10.3 Precise Exceptions and Interrupts

精确异常要求异常指令之前的所有指令已经生效,而异常指令及其之后的指令尚未改变软件可见状态。功能单元可以提前发现异常,但通常先把异常记录在对应 ROB 项中;只有该指令到达队首时,处理器才停止正常退休、清除更年轻工作并转入异常处理程序。[19]

这种延迟报告允许其它独立指令在内部继续执行,同时维持单一、可解释的异常边界。外部中断也需要在合适的退休位置被接受,使保存的 PC、寄存器和控制状态对应完整程序前缀,而不是乱序执行过程中的任意切片。

异常发现位置可能很早,例如 Decode 识别非法指令;也可能很晚,例如 page fault、浮点异常或 machine check。无论何时发现,异常信息都要与对应程序顺序项绑定,并阻止该指令退休。更年轻操作可以暂时存在,但最终必须被清除。

外部中断没有天然对应某条故障指令,处理器需要选择一个可中断边界。为了降低响应延迟,可以在当前可退休前缀结束后进入中断;若 ROB 头部被长延迟操作阻塞,具体实现可以使用不同的重启策略,但软件看到的状态仍需精确。

不可恢复硬件错误与普通异常的处理不同。缓存 ECC、寄存器 parity 或内部一致性错误可能被标记为 machine check,并根据是否能够定位到精确指令选择同步或异步报告。错误检测路径仍需与退休顺序协调。

10.4 Architectural Mapping and Resource Reclamation

重命名阶段投机地建立新物理映射,退休阶段确认该映射成为最新不可撤销状态。目的指令退休后,被覆盖的旧物理寄存器通常才可安全回收。ROB 头部长久阻塞时,旧寄存器、队列项和其它在途资源不能释放,最终会使 Rename 与 Dispatch 停止。

恢复机制可以维护独立退休映射,也可以依赖 ROB 中的旧映射历史。前者能够直接提供最近已提交状态,后者减少一部分复制但可能增加重建工作。无论组织如何,退休、free list 与检查点必须一致更新。

一个周期退休多条目的指令时,退休映射需要按程序顺序处理同批次多次写同一逻辑寄存器。最终映射指向最年轻退休定义,中间定义对应的旧物理寄存器也应正确回收。这与 Rename 的

批次旁路方向相反, 却具有相似的版本解析问题。

Scheduler 项通常在操作成功发射后即可释放, ROB 项要等退休, 物理寄存器旧版本也通常等覆盖者退休。不同资源的回收时刻不同, 使窗口在同一停顿下呈现不同占用。性能计数器若只报告 ROB full, 可能遗漏先由 scheduler、LSQ 或 free list 触发的入口阻塞。

SMT 中每个线程维持独立退休次序, 但可以共享物理资源。一个线程的 ROB 头部 miss 可能占据大量寄存器和 LSQ 项, 间接阻塞另一线程; 按线程配额可以限制影响, 却降低动态共享能力。

10.5 Stores and Memory Visibility

store 在执行阶段生成地址和数据, 并可以被年轻 load 通过 forwarding 观察, 但在仍可能撤销时不能写入对其它观察者不可恢复的共享状态。store 到达退休位置后, 处理器确认它不再被异常或分支恢复撤销, 并允许其进入已提交的排出路径。

已退休 store 通常仍可能停留在 store buffer 中, 等待 L1D 端口、缓存行权限、一致性响应和内存排序条件。因此需要区分三个时刻: 地址与数据已经生成, 指令已经退休, 以及写入已经达到体系结构内存模型规定的可见点。具体可见性还受到栅栏、原子操作和缓存一致性协议约束。

Store 若在退休时地址或数据尚未准备好, 通常会阻止退休继续前进, 因为处理器不能确认该操作未来不会产生异常或缺少必要状态。允许 store 退休后再生成地址会显著增加恢复与精确异常复杂度。

Store buffer 满会反压退休, 即使 ROB 头部的寄存器操作都已完成。持续 store 流的性能因此取决于退休带宽和 buffer 排出率的较小者; 缓存行权限争用、写回压力和 fence 会使排出率产生明显波动。

同一地址的多个已提交 store 可以在 buffer 中合并, 但必须维持其它核心可观察的写入顺序和原子语义。合并减少 L1D 写端口与一致性事务, 却不能跨越要求独立可见的操作或设备访问。

10.6 Recovery and Machine Clears

分支误预测通常在执行阶段被发现并触发前端重定向, 同时撤销分支之后的年轻操作。寄存器映射、空闲列表、scheduler、LSQ 和预测历史都需要恢复到对应检查点。快速检查点增加状态成本, 逐项回滚则延长恢复时间。

内存顺序违例、翻译或权限状态变化、自修改代码及复杂微架构事件可能要求比普通分支恢复更大范围的清除。选择性 replay 只重新执行受影响的数据流, machine clear 或全局 flush 则清除更广泛状态。前者性能代价较低但依赖追踪复杂, 后者实现统一但会重新经历取指、翻译、译码和窗口填充。

分支恢复通常只清除分支之后的年轻操作, 分支之前尚未退休的工作继续保留。异常恢复则以异常指令为边界, 并转入体系结构定义的处理程序。Machine clear 的触发点可能与具体指令无关, 而是处理器发现内部顺序假设失效, 需要重新建立统一状态。

恢复不仅清除 ROB, 还要修正 RAT、free list、scheduler、LSQ、预测历史和前端队列。某些缓存请求一旦发出便无法撤销, 可以允许其完成并把结果视为无主预取, 但不能让返回数据更新错误路径的体系结构状态。

恢复频率低并不意味着代价可以忽略。一次全局清除会破坏前端与后端稳态; 操作缓存、TLB 或数据缓存虽然仍可能命中, 队列和依赖窗口却必须重新填充。性能影响通常大于单纯计算丢弃了多少 ROB 项。

10.7 Retirement Width and Head-of-Line Blocking

退休宽度表示理想条件下每周期可从 ROB 头部提交的工作量。它只有在连续若干最老项均已完成、没有异常或序列化条件时才能达到。由于退休必须遵守程序顺序, 最老的一条 cache miss、页表遍历或长延迟操作可以阻止后续大量已完成工作越过边界, 这就是队首阻塞。

较大的 ROB 能让处理器在队首停顿期间继续寻找独立工作, 但只能延缓窗口耗尽。预取、内存级并行、分支预测和更短执行延迟之所以会改善退休吞吐, 是因为它们降低了最老操作长期未完成的概率, 而不仅是提高局部执行单元利用率。

退休阶段的实际能力由 ROB 组织、提交宽度、异常检查、资源回收、store 提交和恢复机制共同决定。它不是流水线末端的简单删除动作, 而是把内部投机结果整理成软件可见顺序的最终控制点。

标称退休宽度通常高于程序长期可达到的平均 IPC, 因为任何未完成项都会截断当周期可退休前缀。阻塞解除后可以短时间满宽退休, 随后又因下一个长延迟项停顿, 形成突发式提交。

更大的窗口提高找到独立工作的机会, 却不能越过队首。若关键 miss 位于 ROB 很早位置, 年轻工作即使执行完成也只会加速填满窗口。预取、关键 load 优先和及时分支解析的重要性, 最终都会反映为队首阻塞时间减少。

退休端还承担架构性能计数、调试单步和精确事件记录。按架构指令统计需要处理融合和微码序列, 硬件断点与 tracing 也通常在退休边界确认, 以避免记录错误路径事件。

Chapter 11

Cache

11.1 Cache Fundamentals

缓存利用程序访问的时间局部性与空间局部性, 在处理器和更远的存储层次之间保存近期使用的数据副本。它不能消除主存延迟, 而是通过提高近端命中比例、并行处理多个未命中以及提前回填数据, 降低长延迟访问对流水线的可见程度。[1]

现代 CPU 通常采用分离的一级指令缓存和一级数据缓存, 以便前端取指与 LSU 数据访问并行进行; 更低层级则常采用统一缓存。这里的“分离一级、统一下层”不等同于把整颗处理器简单归类为传统 Harvard 架构, 而是一种围绕端口、带宽和一致性需求形成的层次化组织。

缓存的性能不能只用容量或命中率描述。命中延迟决定关键路径, 端口与 bank 决定每周期可服务请求数, MSHR 与填充缓冲决定可保持多少未命中在途, 替换和预取则改变有限容量如何被使用。较高命中率若以更长命中延迟或严重带宽竞争为代价, 未必提高总体性能。

11.2 Capacity, Associativity, and Line Size

缓存按缓存行(cache line)传输数据。较大行可以利用空间局部性并降低标签开销, 却会增加无用字节传输、污染和一致性粒度; 较小行减少浪费, 但需要更多标签和事务。行大小还影响预取、写回、false sharing 与内存突发传输效率。

组相联缓存把地址映射到一个 set, 并在若干 way 中查找标签。提高相联度可以减少冲突缺失, 却增加标签比较、数据选择和替换状态。一级缓存更重视低延迟, 通常不适合无限提高相联度; 较低层级拥有更宽松的访问时间, 可以用更高相联度换取容量利用率。

容量缺失、冲突缺失和强制缺失是理解缓存行为的经典分类, 但真实处理器还会受到预取、替换、相干失效和地址映射影响。一条 line 被驱逐不一定只是容量不足, 也可能是共享写入、目录策略或预取流量造成的结果。

容量还需要区分物理容量与有效容量。标签、ECC、目录位和替换状态会占用面积; set 冲突、线程分区和 inclusion 约束会使部分数据无法自由使用所有 way。一个标称容量更大的缓存, 如果访问分布集中在少数 set, 实际可利用容量仍可能低于较小但映射更均衡的设计。

行大小会改变缺失的组成。顺序访问倾向于从较大行获益, 因为一次 fill 带来更多未来字节; 指针跳转或稀疏访问则可能只使用行中的少量数据。较大行还增加脏写回和一致性失效的最小粒度, 使两个线程修改同一行内不同变量时产生 false sharing。

相联度与替换策略需要配合。提高 way 数可减少固定映射冲突, 但如果替换器无法识别 scan 或预取污染, 额外 way 仍可能被低价值数据占据。反过来, 较低相联缓存可以通过 victim cache、地

址哈希或更合适的数据布局缓解部分冲突。

11.3 Banks, Ports, and Access Bandwidth

多端口 SRAM 的面积和能耗增长很快, 因此缓存常通过 bank 化提高并行访问率。不同地址落在不同 bank 时可以并行服务, 集中访问同一 bank 则产生冲突。所谓“每周期可完成若干 load”通常依赖良好的 bank 分布、对齐与地址类型, 并不对所有访问组合成立。

读端口、写端口、标签端口与数据阵列端口也可能具有不同带宽。store 命中、回填、写回、探测与 demand load 会竞争内部路径; 标签命中并不保证数据能在同一周期返回。评价缓存带宽时, 应区分请求接收率、标签查询率、数据返回率和下层传输率。

Bank 选择通常由地址低位决定, 因此固定步长访问可能长期落在同一 bank。即使总访问率低于峰值, 多条同时到达的请求仍会发生周期性冲突。设计可使用地址哈希、交错映射或复制部分标签端口改善分布, 但复杂映射会增加布线和验证成本。

多读单写、双读双写等端口描述也可能由时间复用或内部 pipeline 实现, 而非真正的多端口 SRAM。某些缓存可以每周期接受多个请求, 却需要多个周期返回全部结果; 另一些缓存则允许多个命中并行完成, 但 refill 或一致性探测会临时占用其中一部分带宽。

11.4 Instruction and Data Cache Differences

L1 指令缓存服务相对只读的取指流, 重点是低延迟、连续字节带宽和跨 taken branch 的供给。它需要与 ITLB、预译码、指令预取和操作缓存协作。代码修改较少, 但一旦发生自修改代码或权限变化, 处理器必须失效旧指令字节及相关预译码状态。

L1 数据缓存同时处理 load、store、回填、写回和一致性探测, 端口与 bank 压力更高。Load-to-use latency 位于依赖关键路径, store 则可以借助 store buffer 延后实际写入。数据缓存还必须完成 store-to-load forwarding、原子操作和写权限获取, 因此其控制状态通常比指令缓存复杂。

低层统一缓存同时接收指令与数据请求, 可以动态共享容量, 却会产生两类流量之间的干扰。代码 footprint 较大时, 指令 miss 可能被数据扫描驱逐; 数据压力较低时, 统一容量又能为指令提供更多空间。替换与 QoS 可以根据请求类型设置不同插入或保留优先级。

11.5 Addressing and Hit Path

缓存地址由 tag、index 与 block offset 构成。PIPT 缓存使用物理地址索引和标签, 语义简单但需等待地址翻译; VIPT 缓存使用页内位索引、物理标签比较, 可把 TLB 查询与数据阵列访问并行化。为了避免同一物理地址被索引到多个 set, VIPT 的每路容量通常受到最小页面大小与行大小约束。

命中路径可能采用标签与数据并行访问, 也可能先查标签再选择数据。前者延迟较低但读取更多 way, 后者节能却增加周期。Way prediction 试图先猜测可能命中的 way, 只访问部分数据阵列; 猜中可以降低能耗, 猜错则需要补充访问并增加延迟。

同一物理页通过多个虚拟地址访问会形成 synonym; 不同地址空间复用相同虚拟地址会形成 homonym。ASID、物理标签、页面着色和软件维护可以降低这些问题, 但具体责任取决于缓存索引方式和体系结构的一致性规则。

标签与数据访问的先后顺序影响能耗。并行读取所有 ways 能在标签比较完成时立即选择数据, 适合极低延迟 L1; 串行访问先确定命中 way, 再读取对应数据, 节省动态能耗但增加周期。部分实现采用预测 way 加回退路径, 在常见命中情况下接近串行能耗, 在误预测时支付额外访问。

虚拟索引还会受到页面大小变化影响。使用最小支持页面的页内位可以保证所有页面安全, 但限制 set 数; 若实现利用大页提供的更多页内位扩大索引, 需要在页面大小未知或小页访问时提供替代路径。TLB 与缓存组织因而必须在设计时联合确定。

11.6 Write Policies and Store Path

Write-through 缓存在命中时把写入继续传播到下层, 实现简单但增加带宽; write-back 缓存只修改近端副本, 并在驱逐时回写脏行。现代高性能数据缓存通常采用 write-back, 并用 dirty 状态追踪是否需要回写。

Write-allocate 在 store miss 时先把整行取入缓存, 再完成修改; no-write-allocate 则可以把写入直接送往下层。前者适合随后继续读写同一行, 后者适合流式写入或不希望污染缓存的场景。Store buffer 把退休与实际缓存写入解耦, 但其容量、合并能力和排出带宽会影响持续 store 吞吐与栅栏延迟。

Store merging 可以把同一缓存行上的多个小写入合并为较少的 cache access, 并减少下层事务。合并要求正确处理字节使能、年龄与异常状态; 若一个较老 store 尚未获得缓存行写权限, 较年轻同址 store 也可能在 buffer 中累积并共同等待。

写回路径需要保存 victim 地址、脏数据和一致性状态。若 writeback buffer 已满, 新的脏行替换无法继续, 即使 demand miss 的下层数据已经返回。缓存因此常预留或优先处理写回资源, 以避免读 miss 与脏驱逐形成循环阻塞。

原子读改写需要在读取旧值和写入新值之间维持不可分割语义。实现可以先获得缓存行独占权限, 再在 L1D 或专用原子单元中完成操作。原子吞吐不仅取决于 ALU, 还受权限获取、行冲突和跨核竞争影响。

11.7 Miss Handling and Non-Blocking Caches

阻塞式缓存一次 miss 会停止后续访问; 非阻塞缓存使用 MSHR 或等价结构记录未完成事务, 使其它独立访问继续命中或发起新的 miss。多个请求访问同一缺失行时可以合并, 避免重复下层请求; 访问不同缺失行则消耗独立 MSHR。

MSHR 数量只是并行未命中能力的一部分。填充缓冲、写回缓冲、互连信用、下层 cache banks 与 DRAM 并行度都可能更早饱和。Critical-word-first 或 early restart 可以让需求字先返回消费者, 其余缓存行随后完成填充, 从而缩短部分 miss 的可见延迟。

替换与回填也可能产生反压。若 victim 为脏行, 需要先获得写回资源; 若 set 中存在存在途填充或一致性事务, 新 miss 可能无法立即选择 victim。真实 miss service path 因而包括 victim 选择、下层请求、返回仲裁、标签更新和依赖唤醒, 而不只是“去 L2 或内存读取”。

Hit-under-miss 表示缓存可以在某个 miss 未完成时继续服务其它命中, miss-under-miss 则进一步允许发起多个独立 miss。实现这两种能力需要把每个请求与 MSHR、返回数据和依赖消费者正确关联, 并处理同一 line 的请求合并、升级和失效竞争。

Prefetch miss 与 demand miss 可以共享 MSHR, 也可以拥有独立配额。完全共享提高利用率, 却可能让激进预取耗尽所有表项; 严格分离保证 demand 进展, 却可能在 demand 稀少时留下空闲

资源。带保留份额的动态共享能够在两者之间折中。

缓存未命中还可能发生在标签命中但权限不足时。例如, store 命中 Shared 状态的 line, 仍需发起一致性升级并等待其它副本失效。这类 coherence miss 不需要从内存重新读取完整数据, 却可能具有显著控制延迟。

11.8 Multi-Level Hierarchies

一级缓存强调低延迟与高端口带宽, 私有 L2 常用于吸收一级缺失并扩大每核工作集, 末级缓存 (Last-Level Cache, LLC) 则在容量、共享数据、内存流量与多核干扰之间折中。不同层级可使用不同的行填入、替换和预取策略, 因为它们面对的访问流已经被上层缓存过滤。

Inclusive 层次要求下层缓存保存上层缓存内容的超集, 便于利用下层标签辅助定位上层副本, 但下层驱逐可能要求失效上层副本。Exclusive 层次倾向于让不同层级保存不同数据以提高总有效容量, 却增加行迁移与查找复杂度。Non-inclusive/non-exclusive 组织不规定严格包含关系, 通常仍需目录或 snoop-filter 信息追踪共享副本; 是否 inclusive 与是否拥有完整 sharer 目录并不是同一个问题。

层级之间可以采用串行或并行查找。L1 miss 确认后再访问 L2 能减少无效访问能耗, 却增加 L2 命中延迟; 预测 L1 可能 miss 并提前查询 L2 可以降低延迟, 但会提高低层活动率。类似取舍也存在于 LLC 目录查询与数据阵列访问之间。

Victim cache 或小型辅助缓冲可保存刚从主缓存驱逐的 line, 特别适合缓解少数地址在同一 set 间来回冲突。它不增加主缓存 hit path 的相联度, 却需要在 miss 路径额外查询, 并处理与主阵列之间的交换。

缓存层次还需要决定 fill placement。所有 demand miss 都填入 L1 实现直接, 但对 streaming 或低复用访问容易污染近端缓存; 某些请求可以只填入 L2/LLC, 或在 L1 采用低优先级插入。旁路决策必须结合访问类型、预取置信度和未来重用预测。

11.9 Shared LLC and NUCA

多核 LLC 可以物理集中, 也可以分成多个 slice 分布在芯片上。分布式组织提高总带宽并缩短部分本地路径, 但不同核心到不同 slice 的距离可能不同, 形成非一致缓存访问 (Non-Uniform Cache Access, NUCA)。地址到 slice 的哈希、数据迁移和复制策略会影响热点、延迟与互连流量。[\[20\]](#)

共享 LLC 会产生容量与带宽干扰。高 miss-rate 线程可能占用大量 ways、MSHR 和互连带宽, 使其它线程的命中率或延迟恶化。Way partition、容量配额、优先级和 QoS 机制可以提供隔离, 但也可能降低整体利用率。评价共享缓存时, 需要同时观察总命中率、每线程 slowdown、公平性和排队延迟。

Slice 映射通常使用物理地址哈希, 使连续地址分散到多个 slice 以提高带宽。哈希能够减少简单热点, 却也使软件难以直接判断数据物理位置。若工作集或大页映射与哈希产生偏置, 某些 slice 仍可能成为容量或带宽瓶颈。

NUCA 设计可以允许数据迁向访问频繁的核心, 或在多个位置复制只读 line。迁移降低局部延迟, 却产生移动和目录更新开销; 复制提高读并行度, 却扩大 sharer 状态, 并使后续写入需要撤销更多副本。静态哈希、动态迁移和复制分别适合不同访问稳定性。

LLC 还可能承担 snoop filter 或目录角色。即使数据不一定驻留于 LLC, 标签或目录仍可记录

哪些私有缓存可能持有副本,从而避免向所有核心广播探测。目录项被驱逐时,系统必须确保不会丢失 sharer 信息,可能需要反向失效私有副本或使用独立后备结构。

11.10 Cache Coherence

私有缓存允许多个核心同时持有同一物理地址的副本,因此系统需要缓存一致性 (cache coherence) 保证同一地址的写入以可解释的方式传播。基本目标可以概括为:同一时刻至多一个写者拥有修改权限,多个读者可以共享只读副本;新的写者获得权限前,旧的可写或可读副本必须按协议降级或失效;后续读者最终获得最新值。[21]

基于失效的协议在写入前撤销其它副本,适合写后多次本地访问;基于更新的协议主动向共享者传播新值,会增加写流量。实际通用 CPU 通常采用 invalidate-based 协议,并用类似 Modified、Exclusive、Shared、Invalid 的稳定状态表达权限。为处理请求交叉、失效确认和数据在途,真实控制器还需要若干瞬态状态。

Snooping 通过广播或有序互连让缓存观察事务,适合规模较小的共享介质;directory 协议记录 sharer 或 owner,使请求只发送给相关节点,更适合大规模多核。目录可能与 LLC 标签结合,也可能独立存在;目录容量、表示精度和驱逐处理都会影响一致性流量。

一致性解决的是**同一地址**的副本与写入顺序问题,并不定义不同地址访问之间的软件可见次序。后者属于内存一致性模型,需要由 LSU、栅栏、原子操作和体系结构规则共同实现。

稳定状态只描述事务完成后的权限。真实协议还必须处理 GetS、GetM、upgrade、writeback、forward 和 invalidation acknowledgement 等消息在互连中并发、延迟或乱序到达的情况。瞬态状态记录“已经发出请求但尚未收到全部响应”等中间阶段,避免同一 line 同时启动冲突事务。

同一 line 上的竞争写入必须被串行化。Home node、目录或有序 snoop 点通常负责为权限请求确定顺序;失去所有权的缓存可能需要直接把最新数据转发给新请求者,而不必先写回内存。数据来源和权限来源可以是不同节点,这也是协议状态复杂的重要原因。

False sharing 是一致性粒度带来的性能现象。两个核心修改同一缓存行内不同变量,语义上没有数据共享,却会反复争夺整行写权限。增大 line 有助于空间局部性,却也扩大 false sharing 范围;软件 padding 和数据布局可以缓解,但硬件仍以固定 line 粒度维护一致性。

一致性事务会与普通 miss 竞争 MSHR、互连和缓存端口。大量共享写入或原子竞争可能使缓存容量充足却仍然性能低下,因为瓶颈来自权限往返和失效确认,而不是数据是否位于某一级缓存。

11.11 Memory Types and External Agents

并非所有地址都按普通可缓存内存处理。设备寄存器、不可缓存区域和写合并内存可能要求不同的访问顺序与缓存行为。DMA 或其它 coherent agent 若不参与硬件一致性,软件必须在设备与 CPU 共享缓冲区时执行必要的清洗、失效和排序;若参与一致性,也仍需遵守对应的地址属性与协议范围。

这些机制与缓存有关,但其详细编程模型属于体系结构和系统软件范畴。微架构分析主要关注它们是否旁路缓存、是否占用一致性事务、是否要求更强排序,以及是否会阻塞普通 load/store 路径。

11.12 Cache Design Balance

缓存设计同时受延迟、容量、带宽、能耗和物理距离约束。扩大容量或相联度可能提高命中率, 却会增加访问时间; 增加端口可提高并行度, 却会显著扩大阵列与布线; 更激进的预取可以隐藏 miss, 却会消耗 MSHR、容量和下层带宽。

因此, 缓存参数应放在完整访问路径中理解: 一级缓存是否满足关键 load-to-use 延迟, 低层缓存是否提供足够未命中吞吐, 替换与预取是否控制污染, 共享 LLC 与互连是否在多核压力下维持可接受延迟。缓存的价值不只是保存数据, 而是以可控成本把远端访问转化为更多近端命中和并行等待。

可靠性状态也是容量和访问路径的一部分。一级标签常使用 parity 快速检测错误, 较低层数据阵列可能使用 ECC 纠正单比特错误。错误检测、scrubbing 和 poison 传播会增加元数据和维护流量, 但不能被视为与缓存组织无关的外围功能。

物理布局最终限制缓存规模。靠近核心的阵列延迟低但面积珍贵, 远端 slice 容量大却需要经过更长互连。设计者可以通过分银行、分层标签、流水化访问和非一致延迟扩大容量, 但这些方法都会使“某一级缓存延迟”从单一数字变成受地址、距离和并发影响的分布。

Chapter 12

Memory Ordering and Consistency

12.1 Memory Consistency as an Architectural Contract

内存一致性模型 (memory consistency model) 规定多线程程序允许观察到哪些 load/store 结果。它是 ISA 与软件之间的契约, 而不是某一种缓存协议或流水线实现。处理器可以在内部乱序执行、缓冲写入和推测读取, 只要最终可观察行为属于体系结构允许的集合。[21]

缓存一致性与内存一致性需要严格区分。Cache coherence 保证同一地址的副本与写入顺序保持一致; memory consistency 规定不同地址之间的访问顺序。一个系统可以对每个地址都保持一致, 却仍允许线程以不同于程序顺序的方式观察对多个地址的访问。

12.2 Orders in a Shared-Memory System

分析内存顺序时至少需要区分四种顺序。**程序顺序**是单线程指令在软件中的次序;**执行顺序**是微架构实际发起与完成访问的次序;**一致性顺序**是同一地址的写入与权限事务顺序;**全局可观察顺序**描述其它线程何时能够看到这些访问。不同模型的差异, 主要在于哪些程序顺序边必须反映到全局观察中。

寄存器数据依赖通常会约束单线程执行, 但内存地址与值还可能通过预测提前获得。即使处理器在内部越过某些访问, 也必须在检测到别名、异常或排序条件时重放, 确保最终结果符合模型。

还需要区分操作执行完成和操作对其它观察者可见。Load 在本核获得值后通常已经完成, 但该值来源仍受一致性顺序约束; store 在本核退休后可能继续停留在 buffer 中, 直到获得写权限并传播。内存模型主要约束可观察关系, 而不要求所有内部阶段保持同一顺序。

编译器也会重排内存操作。源程序首先受语言内存模型约束, 编译器把它转换为带有普通访问、原子和 fence 的机器指令, 处理器再按 ISA 模型执行。仅观察源代码顺序或硬件执行顺序, 都不足以单独判断高级语言并发程序是否正确。

12.3 Sequential Consistency

顺序一致性 (Sequential Consistency, SC) 要求所有线程的内存操作能够排列成一个单一全序, 并且每个线程在该全序中的操作顺序与其程序顺序一致。SC 提供直观语义, 但若硬件严格等待每次访问完成后再执行后续访问, 会严重限制 store buffer、乱序 load 和访存并行性。[22]

体系结构可以在内部采用投机和缓冲实现 SC, 只要任何可能违反全序的结果都不会对软件可见。代价在于需要更强验证与恢复, 因此多数高性能 ISA 选择允许部分重排, 并通过栅栏和同步操作恢复必要顺序。

SC 并不要求所有核心物理上在同一周期看到每个写入, 而要求最终观察能够解释为一个符合各线程程序顺序的全序。实现可以延迟、推测甚至重放访问, 只要不存在任何允许程序区分的违规结果。

经典 Store Buffering 测试中, 两个线程分别先写自己的变量, 再读取对方变量。在 SC 下, 两个读取不能同时得到旧值, 因为任何全序都必须让至少一个写先于对方读取; 允许 store-load 重排的模式则可能出现该结果。

12.4 TSO and Store Buffers

Total Store Order (TSO) 通常保持 load-load、load-store 和 store-store 的程序顺序, 同时允许较年轻 load 在较老 store 尚未对其它核心可见时继续执行, 只要不存在同址依赖。Store buffer 是实现这种行为的一种常见结构: store 可以从退休路径进入缓冲区, 后续 load 在满足转发与排序条件时绕过等待中的 store。

TSO 描述允许的观察结果, 而不是强制唯一微架构。不同处理器可以采用不同的 store queue、转发、合并和一致性接口, 只要外部行为满足模型。栅栏会限制 store buffer 绕行, 并可能要求等待部分或全部旧访问达到规定的可见点。

同址 load 不能简单越过较老 store。若 store 数据已经可用, load 应从 store buffer 转发; 若地址或数据尚未确定, 处理器可以等待, 或投机执行并在别名出现时 replay。TSO 的 store-load 放宽主要针对可证明或预测为不同地址的访问。

Store-store 顺序通常仍需维持, 即其它观察者不能把同一线程两个普通 store 以相反顺序观察。Buffer 可以合并同址写入, 但不能使程序要求的可见顺序发生不可解释变化。

12.5 Relaxed Memory Models

更宽松的模型允许更多 load/store 次序变化, 以扩大乱序执行和内存级并行空间。例如, 独立地址的 load 可以越过旧 load 或 store, store 也可能在不同观察者处以更灵活的时序传播。软件不能依赖普通访问之间隐含的强顺序, 而应使用 acquire、release、fence 和原子操作建立同步关系。

宽松并不意味着任意。数据依赖、地址依赖、控制依赖以及同步指令在不同 ISA 中具有明确规则; 实现必须保证编译器变换和硬件重排共同产生的结果仍位于允许集合内。描述某个处理器“更乱序”不能替代对其体系结构内存模型的说明。

弱内存模型的优势在于允许 load/store 更早发起, 并让 store buffer、写合并和分布式一致性网络拥有更大调度空间。代价是普通程序顺序不再自动构成线程间同步, 锁、消息传递和无锁数据结构必须明确使用相应原子与顺序语义。

依赖顺序的规则具有 ISA 差异。某些模型保证地址依赖 load 不被重排, 另一些软件接口仍建议使用 acquire 获得跨实现的稳定语义。软件应依赖体系结构和语言定义, 而不是某一代核心当前没有观察到的更强行为。

12.6 Microarchitectural Enforcement

LSU 使用 load queue、store queue、store buffer 和 memory dependence predictor 提前执行独立访问。若年轻 load 越过地址未知的旧 store, 随后发现两者重叠, 处理器必须撤销 load 及其依赖结果。Cache coherence 则保证同一行的权限转换和写入串行化, 为 LSU 的顺序验证提供外部事件。

投机 load 在退休前必须保持可撤销, store 则通常在退休后才获得向缓存排出的资格。即使 store 已退休, 它仍可能等待一致性权限或缓存端口, 因此退休顺序与其它核心看到写入的时间并不相同。模型实现的关键在于控制这些时间差, 而不是禁止内部重排。

Invalidate 到达时, 核心可能已经读取该 line 并执行年轻操作。若 load 在一致性顺序上发生于失效之前, 结果可以继续使用; 若处理器投机假定 line 有效、后来发现顺序冲突, 则需 replay。实现需要在缓存控制器、LQ 和一致性事务之间建立可验证的顺序点。

Store 获得独占权限也不等于立即对所有核心可见。其它 sharers 的失效确认、owner 数据转发和目录更新可能分阶段完成。内存模型定义后续同步操作何时可以认为写入已经达到所需传播范围。

同一核心内部的 load queue 可以检测本线程顺序违例, 却无法单独维护跨核心写顺序; cache coherence 提供每个地址的序列化点, 互连和目录再把该顺序传播到各观察者。Consistency 建立在 coherence 之上, 但还要约束多个地址关系。

12.7 Fences, Acquire/Release, and Atomics

Fence 强制特定类别的旧访问在特定类别的新访问之前达到规定顺序点。它不必使整颗核心完全停止, 但可能阻止 load 发射、store 排出或退休继续推进, 具体代价取决于前后访问类型和当前在途状态。

Release 操作约束其之前的相关内存访问不能在可观察意义上被推迟到 release 之后; acquire 操作约束其后的相关访问不能被提前到 acquire 之前。当 acquire 读取到对应 release 写入的值, 或满足体系结构、语言模型规定的其它 synchronizes-with 条件时, 两者才在线程之间建立 happens-before 关系。相比全栅栏, 这种同步保留了更多无关普通访问的重排空间。

原子读改写不仅要求单条操作不可被其它写入分割, 还需提供规定的排序语义。实现可能依赖缓存行独占权限、锁定事务或专用原子单元, 但软件可见保证由 ISA 定义。

Acquire/release 通常比全 fence 约束更少。Release store 主要保证此前访问先于发布动作, acquire load 主要保证后续访问不越过获取动作; 它们不必在所有方向上建立全序。正确使用可以实现消息传递, 同时保留无关访问的并行。

原子操作还具有失败与重试语义。Compare-and-swap 可能因值不匹配失败, load-linked/store-conditional 可能因一致性冲突导致 conditional store 失败。高竞争下反复重试会产生大量权限事务和 cache line bouncing。

Fence 的完成条件需要与内存类型对应。普通可缓存 store、不可缓存 I/O 和持久化写入可能要求不同的排空与确认范围; 单一“流水线停顿若干周期”无法描述所有 fence 成本。

12.8 Litmus Tests and Allowed Outcomes

Litmus test 使用极短的多线程程序枚举某个结果是否允许, 是描述内存模型的重要工具。Store Buffering、Message Passing、Load Buffering 和 IRIW 等测试分别揭示 store-load 重排、发布/订阅顺序、load-store 行为和写入传播顺序。

判断 litmus test 时, 应先依据体系结构规则确定允许结果, 再讨论可能的微架构路径。观察到某个结果并不能单独证明处理器内部采用了唯一实现; 未观察到某个允许结果, 也可能只是测试覆盖不足或实现比模型更强。

测试	基本形态	主要观察问题
Store Buffering	两线程各自 store 后读取对方地址	是否允许 store-load 绕行并同时读到旧值
Message Passing	写入数据后发布标志, 另一线程先读标志再读数据	release/acquire 是否保证数据先于标志被观察
Load Buffering	两线程先 load 后 store	是否允许 load-store 与读取来源形成循环
IRIW	两个写者更新不同地址, 两个读者以不同顺序读取	写入是否以相同顺序传播给多个观察者

Table 12.1: 常见 litmus test 所揭示的内存顺序问题。

Litmus test 的初始值、线程指令顺序和最终寄存器结果都属于定义的一部分。添加一次额外访问、依赖或 fence 就可能改变允许集合, 因此不能只记住测试名称而忽略具体程序。

硬件测试通常需要大量重复、随机延迟和线程绑定, 才能提高弱结果出现概率。模型允许的结果可能极少出现; 测试工具还必须阻止编译器删除或重排关键操作。

12.9 Multi-Copy Atomicity and Heterogeneous Observers

Multi-copy atomicity 描述一个写入是否在逻辑上以单一时刻对所有相关观察者生效。非 multi-copy-atomic 系统可能允许某些观察者先看到新值, 而另一些观察者仍看到旧值, 因此同步规则必须进一步约束传播顺序。

现代系统还包含 DMA、加速器和其它 coherent agent。它们是否处于同一一致性域、支持哪些原子与栅栏, 以及内存属性如何定义, 都会改变允许的观察结果。内存模型分析必须明确观察者范围, 不能把 CPU 核心之间的规则自动推广到所有设备。

内存一致性设计的目标, 是在软件可推理性与硬件并行性之间建立稳定边界。模型越强, 软件越直观但可能限制执行; 模型越松, 硬件与编译器拥有更多优化空间, 但同步原语和验证责任也更重。

非 multi-copy-atomic 行为常与写入先到达某个本地观察者、随后再传播到其它节点有关。即使每个地址最终一致, 不同核心可能在短时间内看到不同传播阶段。同步操作需要规定这种早期观察能否被继续转发, 以及何时建立全局顺序。

共享 LLC 或统一一致性 fabric 并不自动保证 multi-copy atomicity; 关键在于写入的序列化点和传播规则。反过来, 物理上分布式的系统也可以通过协议提供较强可见语义。不能仅从缓存拓扑推断内存模型。

CPU 与设备之间还可能使用不同一致性与排序域。一个加速器可以保持 cache coherent, 却不支持与 CPU 完全相同的原子或 fence 语义; DMA 写入也可能需要专用 completion 和 I/O barrier。跨设备共享数据结构时, 应依据平台定义建立顺序, 而不是只依赖 CPU 模型。

Chapter 13

Cache Replacement Policies

13.1 Replacement Fundamentals

当一个 set 已满而新缓存行需要填入时, 替换策略决定选择哪个 victim。理论上的 Belady 最优策略总是驱逐未来最晚再次使用的 line, 但真实硬件无法知道未来, 只能根据近期行为、访问上下文和在线反馈近似判断。[23]

替换不只是 miss 时的一次 victim 选择, 而是对缓存行生命周期的管理。**插入**决定新 line 的初始优先级, **提升**决定命中后如何保护, **老化**描述优先级如何随其它访问下降, **驱逐**则从当前最低价值候选中选出 victim。许多现代策略主要改进插入或提升, 而继续复用简单可靠的老化机制。

替换状态通常按 set 维护, 因为 victim 只能从该 set 的 ways 中选择。策略能够观察的信息包括命中顺序、填入来源、访问 PC、是否由预取产生、脏状态和线程身份。可用信息越丰富, 理论上越容易预测重用, 但状态、训练和访问延迟也越高。

替换器还必须处理 invalid way、被一致性事务锁定的 way、在途 fill 和不可立即驱逐的脏行。真实 victim 选择常先过滤不合法候选, 再在剩余 line 中应用优先级; 因此论文中的抽象优先级顺序未必直接等同于硬件最终驱逐顺序。

13.2 Recency-Based Policies

Least Recently Used (LRU) 假设较久未访问的 line 更可能继续不被使用。精确 LRU 在低相联缓存中容易理解, 但维护完整访问顺序的状态与更新逻辑会随相联度增加。Tree-PLRU、bit-PLRU 与 NRU 等近似策略用较少状态区分较新和较老候选, 更适合时序敏感的缓存。

Random 不利用重用历史, 却实现简单、更新成本低。在局部性较弱或相联度较高时, 它与复杂 recency 策略的差距可能有限。替换算法的价值因此必须与状态位、更新端口和访问关键路径一起衡量。

传统 LRU 通常把新 line 插到最受保护位置, 并在每次命中后提升到 MRU。该规则适合工作集反复重用, 却容易被一次性 scan 污染: 流式进入的新 line 会依次挤出真正有重用价值的数据。

精确 LRU 需要表示一个 set 内所有 ways 的相对顺序。若相联度为 N , 每次命中都可能改变多个 way 的年龄关系; 高相联 LLC 中, 这种更新既消耗状态写带宽, 也可能延长关键路径。Tree-PLRU 用二叉树方向位近似选择较少使用的子树, 状态量较小, 但其访问历史并不构成真实全序。

Recency 的局限在于它只描述“多久没访问”, 不描述“为何被带入缓存”或“再次 miss 有多昂贵”。一次 scan 中刚填入的 line 很新, 却可能永远不再使用; 某条较旧 line 则可能恰好位于下一阶

段的关键路径。后续策略正是通过插入控制和上下文预测弥补这一信息缺口。

13.3 Insertion and Promotion

LRU Insertion Policy (LIP) 把新 line 插入低优先级位置, 只有发生命中后才提高保护; Bimodal Insertion Policy (BIP) 大部分时间采用保守插入, 只以较小概率把新 line 放到高优先级。它们的共同思想, 是不给所有 miss line 自动赋予高重用假设。

Dynamic Insertion Policy (DIP) 使用少量采样 set 比较 LRU 风格与 BIP 风格插入, 并让大多数 follower set 选择当前表现更好的策略。Set dueling 把策略选择从离线固定配置转化为低成本运行时反馈, 适合程序阶段会变化的情况。[24]

命中后的 promotion 同样影响污染。每次命中都立即提升到最高优先级实现简单, 却可能过度保护偶然命中的 line; 较温和提升可以保留更多候选差异, 但需要更复杂的状态转移。

Set dueling 的关键是只用少量 leader sets 固定运行候选策略, 并用全局或分区选择计数器比较它们的 miss 行为。其余 follower sets 根据计数器选择当前赢家。Leader sets 本身会牺牲局部最优, 但占比很小; 挑战在于采样是否能代表整个缓存, 以及程序相位变化后选择器能否及时转向。

插入策略与 admission/bypass 决策相邻但不完全相同。低优先级插入仍会占据一个 way, 并可能在短时间内驱逐其它 line; bypass 则完全不安装新 line, 避免污染但失去任何潜在命中。只有在预测置信度较高或缓存压力明显时, bypass 才通常值得采用。

13.4 Re-Reference Interval Prediction

RRIP (Re-Reference Interval Prediction) 用离散的 RRPV 表示一条 line 预计多久后再次访问。较小值代表近重用, 较大值代表远重用; victim 优先从最大 RRPV 中选择。与精确 recency 排序相比, RRIP 更容易在较高相联度下实现, 并能把插入策略自然编码进初始 RRPV。

SRRIP 为新 line 赋予较保守但非最差的初值, 在命中后提升; BRRIP 大多数时候把新 line 插得更接近驱逐, 仅偶尔给予较好初值, 以抵抗 scan; DRRIP 再利用 set dueling 在 SRRIP 与 BRRIP 之间动态选择。该家族说明, 稳定的老化表示与自适应插入可以分别设计。[25]

当一个 set 中没有最大 RRPV 的 line 时, RRIP 会统一增加候选 line 的 RRPV, 直到出现 victim。该更新可以在选择 victim 时集中完成, 也可以用逻辑年龄偏移减少多项写入。位数越多, 重用距离分辨率越高, 但状态与老化开销也增加; 实践中较少比特已经能表达“近、较远、很远”等关键类别。

RRIP 对 scan 鲁棒的原因, 不是它准确预测了真实未来距离, 而是它避免让每个新 line 自动占据最受保护位置。该机制说明, 粗略但方向正确的重用分类, 往往比昂贵的精确 recency 更适合大容量缓存。

13.5 Reuse Prediction

更细粒度的策略试图区分由不同访问上下文带入缓存的 line。Dead-block prediction 判断一条 line 是否已完成最后一次命中; 若能够较早识别死块, 就可以优先驱逐, 甚至让某些低价值填入 bypass 缓存。

SHiP 使用 PC 或其它签名学习某类 miss 带入的 line 是否容易产生后续命中, 并把预测结果映射到 RRIP 插入优先级。采样 set 负责提供训练信号, 主缓存只保存较少的预测状态。其优势是针对每类访问做决策, 代价是签名别名、阶段变化和额外 predictor 状态。[26]

Hawkeye 一类方法利用采样访问近似 Belady 行为, 把访问上下文分类为 cache-friendly 或 cache-averse, 再用可在线实现的 predictor 指导插入与 victim 选择。它并未实现真正的未来最优, 而是用近似最优行为提供训练标签。[27]

签名通常来自触发 miss 的 load PC, 也可以加入地址、路径或线程信息。PC 能把重复执行的访问点聚类, 但同一 PC 在不同数据结构或程序阶段可能具有不同重用行为; 加入更多上下文可以减少混淆, 却会增大 predictor 和训练稀疏性。

Sampler 只跟踪少量 sets, 以低成本观察 line 从填入到最后一次命中的生命周期。采样结果不直接决定所有主缓存 line, 而是训练较小的签名表。这样把昂贵的历史记录放在少量样本中, 把廉价预测用于所有填入, 是细粒度替换器常见的实现模式。

预测错误仍需要稳定的回退机制。如果一条被判定为 cache-averse 的 line 实际重复命中, 命中 promotion 应能逐步提高其优先级; 如果 cache-friendly 预测失误, 普通老化应最终允许其被驱逐。因此, 细粒度 predictor 通常建立在 RRIP 等容错老化框架之上, 而不是完全取代它。

13.6 Beyond Miss Rate

最低 miss rate 不一定等于最高性能。一个孤立、位于关键依赖链上的 miss 可能比多个可并行隐藏的 miss 更昂贵; 脏行驱逐还会产生写回流量; 共享 LLC 中, 一个线程的命中收益可能以另一个线程的严重 slowdown 为代价。

Cost-aware replacement 可以把 miss 延迟、memory-level parallelism、写回成本或占用时间纳入 line 价值。EVA 等方法强调, line 的收益应与其长期占据容量的机会成本比较。此类策略更接近资源调度, 但需要获得稳定、低成本且可及时更新的代价信号。

关键性信号通常比重用信号更难稳定获得。一次 miss 是否阻塞执行取决于 ROB 位置、依赖链、同时在途的其它 miss 和线程状态; 这些条件在 line 填入时可能尚不明确。将过多后端信息反馈到 LLC 还会增加延迟和跨核状态。因此, cost-aware 策略通常使用近似分类, 而非实时计算完整性能损失。

脏 line 的驱逐会消耗写回带宽, 但无条件保护脏 line 也可能让低价值数据长期占据容量。写成本、存储介质耐久性和当前 writeback buffer 压力可以成为辅助信号, 最终仍需与未来读命中收益权衡。

13.7 Multi-Level and Shared Caches

不同缓存层级具有不同目标。L1 受命中延迟和更新能耗限制, 通常采用较简单策略; L2 与 LLC 拥有更大容量和更宽松时序, 可以使用采样、签名和自适应策略。上层过滤后的访问流也会改变下层看到的局部性, 因此同一替换器放在不同层级未必具有相同效果。

共享 LLC 还需要考虑线程间干扰。Utility-based partitioning 根据线程从额外容量中获得的收益分配 ways; 它本身属于容量管理, 但通常通过 victim 约束与替换状态落实。严格分区提供隔离, 动态共享提高利用率, 实际设计需要在总体吞吐与公平性之间折中。

Inclusive、exclusive 与 non-inclusive 层次会改变驱逐副作用。下层 victim 可能触发上层失效、数据迁移或目录更新, 因此替换决策不仅影响本级命中率, 也影响整个层次的流量与一致性事务。

私有缓存的目标通常更接近本线程延迟, 共享 LLC 则同时管理多个核心和请求类别。若只优化总 miss 数, 高强度线程可能获得大部分容量, 而缓存敏感线程遭受较大 slowdown。线程感知插入、way 配额或最大减速约束可以改善公平性, 但需要估计每个线程的独立行为。

在 distributed LLC 中, 替换还与数据放置和迁移相互作用。一条 line 的价值不仅取决于是否再次访问, 也取决于从哪个核心访问、位于哪个 slice 以及迁移后是否减少互连距离。将所有问题都塞进单一替换优先级会过于复杂, 实际系统常把 placement、partitioning 和 local victim selection 分层处理。

13.8 Interaction with Prefetching

预取 line 的重用概率与 demand line 不同。若替换器把所有预取填入都放到高优先级位置, 错误或过早预取会迅速污染缓存。Prefetch-aware replacement 可以采用较保守插入, 并在 demand 命中后再提升; 也可以根据预取器置信度或历史有用性选择是否 bypass。

替换策略的核心问题最终是: 在有限容量中, 哪些 line 值得继续占据位置。Recency、插入控制、重用预测和代价感知只是不同近似信号; 有效设计需要在预测收益、状态成本、更新延迟和工作负载适应性之间取得平衡。

Prefetch line 可以携带来源与置信度, 替换器据此选择初始 RRPV; 若该 line 被 demand 命中, 则提升为普通 demand line。这样既允许准确预取获得保护, 又限制未验证预取长期占据容量。被预取但在使用前驱逐的 line 还可反馈给预取器, 区分地址错误与过早到达。

替换与预取的耦合说明, 缓存管理不能只优化本地 hit rate。更保守的预取插入可能降低预取命中, 却保护 demand 工作集并减少写回和互连流量; 更激进的保留在带宽充裕、流式重用稳定时则可能更好。运行时反馈通常比固定策略更能适应这种变化。

Chapter 14

Prefetching

14.1 Prefetch Fundamentals

预取(prefetching)在需求访问真正发生前,把未来可能需要的指令、数据或翻译信息移动到更近的层级。它不改变近端缓存命中的固有延迟,而是试图让长延迟访问提前开始,从而减少需求 miss 暴露给流水线的等待时间。[28]

任何预取机制都必须回答三个问题:预取什么地址、提前多久发起、以及把结果放在哪里。地址错误会浪费流量;太晚无法隐藏延迟;太早则可能在使用前被驱逐;放得过近会污染宝贵容量,放得过远又可能在 demand 到来时仍然来不及。

预取器观察访问序列的位置也会改变问题。L1 附近能够看到较完整的访问流,适合捕捉短距离规律,却必须承担严格时序和污染成本;L2 或 LLC 附近只看到被上层过滤后的 miss 流,模式可能被扭曲,但更集中于代价较高的访问,并拥有更长时间发起远端请求。

Lookahead 可以用未来访问数量或时间距离表示。固定预取 distance 在稳定循环中容易工作,但真实内存延迟会随缓存命中层级、互连拥塞和 DRAM 排队变化;程序消费速度也会随分支和执行资源变化。因此,同一 distance 在一个阶段可能太晚,在另一个阶段又可能太早。

预取请求还需要在需求访问到达时被识别。若 demand 与在途 prefetch 指向同一 line,硬件应合并请求并提高优先级,而不是重复发送下层事务。若 prefetched line 已经回填,缓存可以标记其来源和是否被 demand 使用,为准确率、及时性和污染反馈提供依据。

硬件预取通常是提示型流量。无效地址、权限未知或资源紧张时,处理器可以丢弃或延迟预取;软件 prefetch 指令也通常不应产生普通 load 相同的精确异常。该语义允许系统把 demand 正确性置于预取收益之上。

14.2 Sequential and Stride Prefetching

Next-line 预取假设访问会继续到相邻缓存行,stream prefetcher 则识别持续递增或递减的地址流。Stride prefetcher 通常按 load PC 或流上下文记录最近地址差,当步长重复出现且置信度达到阈值时,按相同步长生成未来请求。

这类预取器实现成本低,对数组遍历、扫描和规则数值计算有效。工程难点主要来自多流交错、阶段切换、跨页访问与深度控制。预取 degree 表示一次生成多少请求,distance 表示领先需求流多远;二者共同决定能否覆盖底层延迟,以及会消耗多少缓存与带宽。

跨页预取通常需要更谨慎。模式在虚拟地址上连续并不保证下一页已经映射、权限相同或翻译可用;贸然跨页可能引入 TLB miss 与 page walk 流量。实现可以在已有翻译命中、权限已知或

置信度较高时才允许越过页面边界。

Stride predictor 通常为若干静态 load PC 保存最近地址、候选 stride 和置信度。连续两次差值相同并不足以证明长期规律, 因此状态机会在观察到重复后逐步提高置信度, 并在差值变化时降低或替换 stride。表项有限时, 不同 load PC 还会发生索引冲突。

多流交错要求预取器同时维护多个活动 stream。全局只看 miss 地址差会把两个独立数组混合成虚假 stride; 按 PC 或 stream context 分组能分离访问, 却增加表项和查找。对软件流水化循环, 同一 PC 还可能交替访问多个基址, 单一 stride 状态仍可能不够。

Degree 与 distance 的控制目标不同。增加 degree 提高每次触发产生的并行请求数, 适合需要快速追赶的流; 增加 distance 使请求更早发出, 却可能不增加同一时刻请求数量。两者都会增加 MSHR 和带宽压力, 但对及时性的作用并不相同。

14.3 Correlation and Footprint Prefetching

不规则访问未必具有固定数值步长, 却可能具有可重复的访问关系。Address-correlation 或 Markov 预取器学习“A 之后经常访问 B”, global history buffer 保存近期 miss 序列, temporal streaming 则重放过去出现过的较长访问片段。

Spatial footprint 方法学习一个触发访问附近常出现的缓存行集合。对象、代码区域或内存页每次出现于不同绝对地址时, 相对 footprint 仍可能保持稳定。这类机制用更丰富的上下文提高覆盖率, 却需要更多历史状态, 并面临签名别名与程序阶段变化。

相关性准确并不保证及时。若在观察到 A 后才开始预取即将立刻使用的 B, 请求可能仍然太晚。增加链式深度或重放更长序列可以扩大 lookahead, 却会放大错误路径和带宽消耗, 因此需要置信度与深度限制。

Markov 预取器可以为一个地址保存多个可能后继及其计数。多后继提高 coverage, 却会在分支化访问模式中迅速扩大请求数; 只保存最高概率后继节省带宽, 却可能遗漏路径变化。选择多少 successors、追踪多深以及何时停止, 是相关性预取器的核心节流问题。

Temporal streaming 保存较长的历史 miss 序列, 并在再次遇到已知触发点时回放后续片段。它能够获得比 pair-wise correlation 更长的 lookahead, 但历史日志容量较大, 而且同一触发点在不同程序阶段可能对应不同序列。加入路径、PC 或区域上下文可以减少歧义, 却提高元数据成本。

Spatial footprint 常以区域或页为单位记录哪些 line 在一次活跃期中被访问。后续再次进入该区域时, 预取器按 footprint 取回相关 line。它适合对象布局、代码页和规则稀疏访问, 但 footprint 过大或触发过早会一次性产生大量流量。

14.4 Indirect and Execution-Based Prefetching

链表、树和图遍历的下一个地址常由当前 load 的返回值决定。Indirect prefetcher 可以学习 producer load 与 consumer 地址生成之间的关系, 在 producer value 到达后提前生成后继地址。它仍受串行指针链限制: 如果每个节点地址必须等待前一节点返回, 单纯提前一步可能不足以覆盖内存延迟。[29]

Runahead execution、helper thread 与程序切片执行通过提前运行地址生成逻辑发现未来 miss。它们不等同于普通表驱动预取器: 表达能力更强, 可以穿过复杂计算和部分控制流, 但会消耗执行、寄存器、TLB 与缓存资源, 并需要限制异常和错误路径影响。

这些机制通常补充而非替代简单 stream/stride 预取。规则流由低成本引擎覆盖, 执行型机制只针对统计模式难以预测、但地址生成代码可以提前执行的剩余访问。

Pointer chasing 的根本限制是地址依赖。即使预取器准确知道“下一个地址位于当前节点的 next 字段”, 也必须先获得当前节点数据。Jump-pointer 或链式元数据尝试一次跨过多个节点, 用额外存储换取更深 lookahead; 当数据结构修改或路径分支时, 这些指针又可能失效。

Program-slice prefetch 只执行产生未来地址所需的指令子集, 而不复制完整程序。切片需要保留必要寄存器与控制依赖, 并避免具有外部副作用的操作。切片过小无法产生地址, 过大则接近重复执行主线程, 收益会被资源竞争抵消。

Runahead 通常在核心因长延迟 miss 无法正常退休时进入特殊执行模式, 把依赖于缺失值的结果标为无效, 同时继续执行能够独立生成地址的操作。退出 runahead 后, 投机体系结构结果被丢弃, 但期间产生的 cache/TLB 请求可以作为预取保留。它利用主核心执行能力, 因而表达力高, 也会消耗明显能量。

14.5 Instruction and Translation Prefetching

指令预取预测未来控制流上的代码块, 主要缓解 L1I 与 ITLB 缺失。顺序取指容易预测, taken branch、间接跳转、调用与返回形成的控制流不连续则需要 BTB、分支历史或先前取指轨迹提供目标。较深 lookahead 往往要求沿预测路径跨越多个控制流边界。

数据预取更多依赖 load PC、地址差分、空间 footprint 与数据结构关系。两者共享准确率、及时性和污染问题, 但预测信号不同, 通常由不同结构负责。

翻译预取的对象是 TLB 项或页表元数据。它可以使数据或指令预取在跨页时不被 page walk 阻塞, 但也会竞争 TLB、walker 和缓存资源。数据预取与翻译预取需要协同: 只提前准备 cache line 而没有翻译, 或者只准备翻译而没有远端数据, 都可能留下另一段关键延迟。

指令预取的一个特殊困难是错误路径。数据预取通常由已执行 load 训练, 指令预取却必须沿尚未确认的控制流向前推进。分支预测准确率、BTB 覆盖和代码布局共同决定它能跨越多少 discontinuities; 每多跨越一个未解析分支, 未来路径错误的概率都会累积。

Fetch-directed instruction prefetch 可以使用 FTQ 中已预测的未来块, 在实际取指前向 L2 或 L1I 发出请求。其 lookahead 受 FTQ 深度和预测目标生成率限制; 若前端已经因为 BTB miss 无法生成未来 PC, 单独扩大指令预取带宽也没有作用。

翻译预取可以利用数据预取器即将跨越的页号, 提前触发页表查询; 也可以利用一次 page walk 同一 cache line 上的相邻 PTE。前者与数据流紧密耦合, 后者增量成本较低但不保证相邻页面很快被访问。

14.6 Placement and Request Priority

预取结果可以填入 L1、L2、LLC 或专用 stream buffer。靠近核心能缩短最终 demand 路径, 却更容易驱逐有用数据; 较低层级污染较小, 但 demand 到达后仍需额外传输。专用缓冲允许预取数据在不直接占用主缓存 way 的情况下等待匹配访问, 但增加标签与命中旁路。

预取请求通常应低于 demand request 的优先级。系统拥塞时, 继续发出大量预取可能占用 MSHR、NoC、内存 bank 与 row-buffer 机会, 反而延长真实访问。另一方面, 若预取总被无限推迟, 它又无法形成足够 lookahead。因此, 请求优先级与节流策略必须结合当前资源压力。

Fill policy 可以根据预取置信度决定放入哪个层级、插入哪个替换优先级以及是否允许

bypass。低置信度 line 可先停留在 L2 或专用 buffer, demand 命中后再提升;高置信度且即将使用的 line 可以直接填入 L1。动态 placement 比固定“全部进 L1”更能控制污染。

预取与写回也会竞争带宽。激进预取驱逐脏 line 时,不仅产生读取请求,还会增加 writeback 流量;内存控制器的 row-buffer 局部性也可能被预取地址打乱。因而,污染成本不应只统计被驱逐 line 数,还应包括由此引发的下层事务。

14.7 Accuracy, Coverage, and Timeliness

Accuracy 表示发出的预取中最终被有用 demand 使用的比例;coverage 表示原本有代价的 demand miss 中有多少被预取消除;timeliness 表示有用数据是否在需求之前到达。Pollution 与 bandwidth overhead 则描述获得这些收益所付出的缓存和链路成本。

高 accuracy 不一定带来高性能:预取可能总是晚到,或者只覆盖本可由 memory-level parallelism 隐藏的 miss。较低 accuracy 也不必然无效,如果它覆盖的是关键依赖链上的昂贵 miss,且系统仍有富余带宽。最终目标不是最大化某个比例,而是减少阻塞执行的等待周期。

Useful、late、unused 和 harmful prefetch 应当区分。Late prefetch 表明地址可能正确但 lookahead 不足;unused prefetch 反映覆盖预测错误或过早填入;harmful prefetch 则进一步导致有用 line 被驱逐或 demand 请求排队时间增加。

Accuracy 的分母也需要明确。若在途预取被后来的 demand 合并并提升,它应计为有用但可能晚到;若预取 line 被一致性失效而未使用,原因不一定是地址预测错误;若 line 在低层被 demand 命中,却未进入 L1,它仍可能减少了部分延迟。不同层级的统计口径会得到不同结论。

Coverage 应以原本会造成何种 miss 为基线。把 L1 miss 转化为 L2 hit 与完全消除 DRAM miss 的价值不同;覆盖一条被其它 miss 并行隐藏的访问,与覆盖 ROB 头部关键 load 的性能意义也不同。IPC、阻塞周期和带宽必须与预取指标一起观察。

Timeliness 既可能通过增加 lookahead 改善,也可能通过减少队列延迟改善。更早发出请求并非唯一方法;提高 demand 合并优先级、选择更合适的 cache level 或避免预取请求在内存控制器长期排队,同样可以让数据更及时到达。

14.8 Feedback and Throttling

反馈机制可以根据预取 line 是否被使用、是否晚到、是否造成驱逐,以及 MSHR、互连和内存带宽压力,动态调整 degree、distance、置信度阈值与放置层级。带宽充裕时可以适当提高覆盖率,系统拥塞或污染上升时则降低激进度。[30]

多个预取器同时工作时还需要协调。L1 stream、L2 stride、相关性预取、指令预取和翻译预取可能竞争同一组下层资源;简单叠加各自局部最优的策略,可能使总体请求率失控。系统级控制应根据 demand 延迟和资源占用评价净收益,而不是只依据每个预取器自己的命中统计。

反馈窗口过短会对随机波动过度反应,频繁切换激进度;窗口过长则无法及时跟随程序 phase。控制器可以分别维护短期资源压力和较长期准确率,用前者快速抑制拥塞,用后者决定预取器是否值得保持开启。

多核系统还需要考虑跨核心外部性。一个核心的预取可能提高自身 coverage,却占用共享 LLC 与内存带宽,使其它核心 slowdown。全局控制可以按核心限额、优先级或公平性目标调节请求,而不能只让每个核心依据本地命中率独立加速。

预取器之间还可能重复请求同一 line。去重过滤可以减少流量,但维护全局候选集本身需

要状态;简单优先级规则可让更近端或更高置信度引擎拥有请求,其它引擎只负责训练,不再重复发出事务。

预取是一种受预测正确性、时间提前量和资源成本共同约束的延迟隐藏机制。最有效的设计不必始终最激进,而应在工作负载阶段和存储压力变化时,持续调整它对未来访问的投入程度。

Chapter 15

Security

15.1 Microarchitectural Security

现代处理器依靠分支预测、乱序执行、缓存、TLB、预取和多种共享队列提高性能。体系结构正确性只要求错误路径上的指令不改变最终软件可见状态,但这些指令仍可能改变缓存、预测器、翻译结构或资源占用。若另一执行上下文能够测量这些变化,微架构状态就会形成信息通道。

因此,需要区分**体系结构状态**与**微架构状态**。前者包括退休后的寄存器、内存和异常结果;后者包括缓存标签与替换状态、分支历史、BTB、RAS、TLB、预取器、填充缓冲和队列占用。体系结构回滚能够撤销前者,却不必然清除后者。现代微架构安全的核心问题,正是未退休执行是否让敏感信息影响了跨安全域可观察的内部状态。

微架构状态成为安全问题,通常需要同时具备三个条件。第一,状态与受保护的数据、地址或控制流存在相关性;第二,该状态跨线程、进程、特权级或虚拟机被共享或复用;第三,攻击者能够以足够分辨率观察命中、冲突或资源竞争。只具备其中一项通常不足以形成实用通道。

安全边界也必须明确。相同的缓存时序原语,在同一进程内可能只是局部信息流,在跨进程、跨特权级或跨虚拟机时则破坏不同级别的隔离承诺。微架构分析应先确定状态在哪些上下文之间共享,再判断攻击是否真正越过目标边界。

性能状态的生命周期往往比体系结构状态模糊。缓存行可以在任务切换后继续驻留,分支历史可能被后续上下文继承,TLB项可由ASID隔离但仍共享容量,执行队列则只在同核并发时可观察。不同结构需要不同的隔离、标记或清洗策略,不能用统一“清空处理器状态”概括。

15.2 Transient Execution Model

瞬时执行攻击通常包含四个阶段。第一,攻击者使处理器沿错误预测或尚未验证的路径推进;第二,瞬时指令接触本不应在该路径中使用的数据;第三,秘密值改变缓存、预测器或共享资源;第四,体系结构恢复后,攻击者通过时间或冲突测量恢复这些变化。

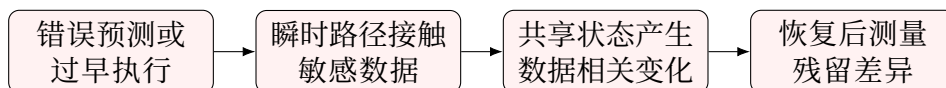


Figure 15.1: 瞬时执行泄露的一般传播链。体系结构恢复不会自动除此前形成的微架构痕迹。

攻击是否成立取决于入口、瞬时数据流、传输载体和安全边界同时存在。错误预测本身只造成性能损失;共享缓存本身也不必然泄露秘密。只有当瞬时路径能够读取敏感数据,并把它编码

到攻击者可观察的状态中时,才形成完整泄露链。

瞬时窗口的长度由错误条件何时解析决定。分支条件依赖 cache miss 时,错误方向可能维持较长时间;目标计算或权限检查较早完成时,窗口则较短。更大的 ROB 和更宽的执行后端可以在窗口中推进更多操作,提高性能的同时也扩大潜在传输能力。

秘密并不一定直接作为 cache index。它可以先影响分支方向、目标地址、循环次数、端口使用或预取器训练,再由攻击者观察相应资源。只要秘密相关操作在恢复前留下可区分状态,传输链就可能成立。因此,限制某一种经典数组编码并不能覆盖所有载体。

训练与触发也不必发生在同一上下文。攻击者可能利用地址别名、共享历史或上下文切换残留,在自己的安全域训练预测状态,再等待受害者消费。跨域训练能力取决于索引是否包含完整地址空间标识、预测器是否分区,以及切换时是否执行必要清洗。

15.3 Spectre V1: Conditional Misprediction

Spectre Variant 1 利用条件分支方向误预测绕过软件边界检查。攻击者先使预测器相信检查通常成立,再提供使真实条件失败的输入。分支结果尚未解析时,处理器可能投机执行检查后的 load,并用越界读取值选择另一个缓存地址。错误路径最终被清除,但与秘密相关的缓存行可能仍然处于不同命中状态。[31]

该变体说明,源码中的控制依赖并不自动构成执行依赖。若敏感 load 的地址已经就绪,分支尚未解析并不会阻止它进入 LSU。缓解可以使用投机屏障阻止后续操作越过检查,也可以对索引或指针做数据依赖式掩码,使错误路径只能访问安全区域。编译器必须保证这些约束不会在优化中被重新消除。

Variant 1 的微架构条件包括分支误预测窗口、load 发射时机、权限与异常路径、缓存传输载体以及恢复前可执行的依赖链长度。单独降低方向误预测率只能减少触发概率,不能替代对高风险数据流的限制。

边界检查后的数据依赖掩码是一种重要思路。若不可信索引在条件失败时通过按位操作强制变为安全值,后续 load 的地址将依赖已处理索引,而不是只依赖分支方向。即使处理器仍然误预测分支,错误路径也难以生成越界地址。该方法是否有效取决于编译器是否保留真实数据依赖,以及处理器是否存在会绕过该依赖的新预测机制。

投机屏障提供更直接的顺序约束,但代价通常高于局部掩码。屏障可能阻止后续 load、分支或其它操作进入执行,具体语义由 ISA 定义。将屏障无差别插入所有条件分支会显著损害性能,因此编译器和软件需要识别真正跨越安全边界的检查。

Variant 1 还可以出现在类型检查、对象长度、权限标志和指针验证之后,并不限于简单数组。共同条件是:检查结果尚未解析时,后续地址生成已经具备足够输入;且瞬时访问能够接触比软件语义允许范围更广的数据。

15.4 Spectre V2: Target Injection

Spectre Variant 2 利用间接分支目标预测,使受害者瞬时执行到攻击者选择或影响的 gadget。目标预测通常依赖 BTB、间接历史和其它控制流上下文;若这些状态在安全域之间共享或发生别名,攻击者可以训练某一模式,再让受害者的间接分支消费被污染的预测结果。[31]

这一攻击面不同于普通方向误预测。即使受害者的条件判断完全正确,错误目标仍可改变瞬时控制流。缓解可通过限制间接预测、清除或隔离预测状态、使用体系结构控制位,以及把高风险

间接跳转改写为不依赖相同预测路径的序列来切断训练与消费关系。

预测器并非单一表。方向历史、目标缓存、间接目标选择和全局路径信息可能共同决定最终目标;隔离其中一部分状态并不保证其余历史不能影响预测。因此,安全边界需要覆盖实际参与目标生成的完整状态链。

Target injection 的可行性还取决于地址索引和标签。若 BTB 只使用虚拟地址低位或压缩标签,不同地址空间中的分支可能发生别名;更完整的标签和 ASID 信息可降低跨域冲突,但增加访问成本。多级 BTB 中,不同层级还可能采用不同索引与共享范围。

间接分支 gadget 必须在瞬时窗口内形成可观测数据流。目标位置可以是受害者程序中原本存在的指令序列,也可能由代码布局使若干指令在错误入口下产生不同语义。控制目标预测只是入口,真正泄露仍依赖后续数据访问和载体。

硬件预测控制通常具有作用域和成本差异。有的机制限制低权限训练影响高权限消费,有的在上下文切换时刷新状态,有的要求分支之后重新建立可信预测环境。评价这些机制时,需要确认它们覆盖哪些预测器、何时生效,以及是否同样处理 sibling thread。

15.5 Return Prediction and History Injection

返回指令通常由 RAS 预测。错误路径调用、栈深度溢出、异常切换或攻击者构造的调用序列可能使 RAS 与真实调用栈失配,从而让返回指令瞬时进入错误目标。返回预测攻击说明,任何能够决定下一取指地址的结构都需要考虑跨上下文状态管理,而不只是一般 BTB。[32]

Branch History Injection (BHI) 进一步利用分支历史影响后续间接目标选择。即使目标表本身受到某些隔离,攻击者仍可能构造全局历史,使受害者在自己的目标集合中选择危险路径。该问题强调,历史状态的共享范围和更新规则与目标条目隔离同样重要。[33]

RSB underflow 时的行为尤其重要。部分处理器会退化到一般间接分支预测器,另一些实现可能使用循环栈或特殊填充。若退化路径重新引入共享 BTB,原本针对返回预测的隔离就可能失去作用。操作系统常在特权切换时填充 RSB,以减少使用旧上下文返回历史的机会。

全局历史通常需要投机更新,否则前端无法连续预测多个分支。错误路径历史在恢复时应被撤销,但攻击者仍可能利用预测发生前的共享历史。按线程维护、加入安全域标识或在边界处重置历史都能减少影响,却会损失跨切换的预测热状态。

15.6 Fault-Driven Transient Execution

Meltdown 类攻击不是以错误分支目标为主要入口,而是利用数据访问、权限检查和异常报告之间的时序。如果实现让非法访问的数据在权限失败生效前进入瞬时执行通路,后续操作就可能把该值编码到缓存中;异常最终仍然精确报告,却无法撤销已经形成的缓存痕迹。[34]

这类问题要求区分“访问最终会产生异常”和“数据在何时允许被消费者使用”。为了缩短 load 延迟,翻译、权限检查、缓存访问和数据旁路常被重叠执行;安全实现必须保证未经授权的数据不会在可传播的形式下到达依赖操作。

Fault-driven leakage 与 Spectre 都利用退休前执行,但根因不同。Spectre 主要错误预测控制流或依赖, Meltdown 类问题则源于保护检查与数据可用时序不当。缓解也因此不同:前者重在控制预测状态和错误路径推进,后者重在阻止未经授权的数据进入执行数据流。

异常并不一定立即停止后端。为了维持精确异常,处理器通常等故障指令到达 ROB 头部后才向软件报告,而较年轻指令在此前可能已经执行。安全实现需要允许这种内部并行,同时保证

故障 load 不产生可供年轻操作使用的秘密值。将结果标记为无效、返回固定值或阻止依赖唤醒, 都是可能的实现方向。

权限检查与缓存层次的关系也影响攻击面。若标签或数据阵列在权限确认前已经返回内容, 后续旁路必须严格门控; 若访问在较低层才被拒绝, 窗口可能更长。大页、TLB 命中与缓存命中会改变检查路径, 但不能改变最终保护要求。

对故障的抑制机制可能让攻击者在用户态反复执行触发序列, 而不被操作系统终止。例如事务执行或信号恢复可隐藏异常的体系结构效果。硬件安全不能假定异常处理开销本身足以阻止高频利用。

15.7 Speculative Store Bypass

Speculative Store Bypass (SSB, 常称 Spectre Variant 4) 利用内存相关性预测。若一条较老 store 的地址尚未计算完成, 处理器可能预测它与年轻 load 不重叠, 并允许 load 提前读取缓存或内存中的旧值。地址最终匹配时, 核心会重放 load, 因而体系结构结果仍然正确; 但旧值在重放前可能已经沿依赖链影响缓存或其它共享状态。[35]

SSB 与普通 store-to-load forwarding 错误需要区分。前者的关键是年轻 load 绕过尚未解析的旧 store, 读取到本应被覆盖的旧内存值; 后者则是从错误的 store buffer 项取得数据。两者都发生在 LSU 的依赖预测与验证路径, 但攻击入口、可见值来源和适用缓解并不相同。

这类攻击尤其需要关注在同一地址空间内执行不可信代码的语言沙箱和即时编译运行时。攻击者若能影响地址计算和代码形态, 可能构造由可信代码瞬时消费旧值的 disclosure gadget。进程隔离能够使秘密不再映射到攻击者可利用的同一地址空间; 局部屏障或真实数据依赖可以阻止危险 load 过早推进; 体系结构提供的 SSBD/SSBS 类控制则通过限制相关 load 绕过旧 store 来换取更直接的覆盖。[35]

全面关闭 store bypass 会降低 load 与 store 地址生成的重叠程度, 尤其影响具有大量未知地址 store 的代码。因此缓解应依据安全边界选择作用范围: 跨进程隔离已经切断地址空间内 confused-deputy 路径时, 不一定需要对所有线程持续施加同等限制; 运行不可信脚本、字节码或插件的进程则需要更严格地审查同址旧值是否能够进入瞬时数据流。

15.8 Microarchitectural Data Sampling

并非所有瞬时执行泄露都从可指定地址的 cache line 取得数据。Microarchitectural Data Sampling (MDS) 利用 store buffer、fill buffer、load port 等临时结构中残留的数据: 某些故障、assist 或特殊 load 路径可能在退休前向年轻操作提供陈旧值, 攻击者再通过 disclosure gadget 对这些值进行采样。与经典 Meltdown 或 Flush+Reload 相比, 这类方法通常更难精确选择目标地址, 却可能越过进程、特权级或同核线程边界观察此前流经共享结构的数据。[36]

采样攻击说明, 微架构结构的安全属性不仅取决于访问控制, 还取决于**重新分配前的数据生命周期**。队列项被标记为空闲, 并不意味着其中的位已经清零; 若后续异常路径能够错误读取旧内容, 逻辑上的资源释放就会变成跨域数据残留。硬件可以在故障或 assist 时阻止依赖操作执行、返回固定值, 或保证旧位不会进入转发网络; 软件则可在安全域切换时触发结构清洗。

Gather Data Sampling (GDS) 展示了同一模式如何出现在更专门的数据通路中。受影响实现的 gather 指令可能把先前向量寄存器或内部向量状态的陈旧数据瞬时送给依赖操作, 攻击者仍需通过缓存等载体恢复采样值。其触发条件和数据来源不同于 MDS, 但共同点都是无效结果不会退

休, 泄露来自结果被撤销前的微架构传播。[37]

数据采样缓解通常需要硬件、微码与系统软件共同工作。硬件修订可以从源头阻止陈旧值转发; 微码可以扩展既有指令, 使其清除受影响缓冲; 操作系统和 hypervisor 则在内核到用户、宿主到来宾或其它信任域切换处执行清洗, 并决定是否允许不同安全域共享 SMT 核心。清洗时机不足会留下残留窗口, 过度清洗则增加系统调用、虚拟机切换与调度成本。

15.9 Microarchitectural Side Channels

缓存是最常见的传输载体。Flush+Reload 利用共享物理页的缓存命中差异, Prime+Probe 通过 set 冲突观察受害者是否占用了相同缓存集合。攻击者测量的不是秘密值本身, 而是秘密相关访问对缓存状态产生的时间差。

分支预测器也可以独立形成侧信道。预测命中、别名与训练状态反映先前控制流, 可能泄露代码布局或执行路径; 相关研究表明, BTB 行为可被用于削弱地址空间随机化。[38]

局部优化结构同样可能泄露信息。Cache way predictor 使用地址相关状态提前选择可能命中的 way; 若标签或别名行为跨上下文可测量, 就可能成为不依赖传统缓存共享方式的侧信道。该例说明, 任何保存地址或数据相关历史、并影响可测延迟的预测结构都应纳入安全审查。[39]

非缓存载体还包括 TLB、执行端口、队列、内存 bank 和互连争用。它们能否形成实用攻击取决于共享范围、信号稳定性、时间分辨率和噪声。不能因为某结构不保存架构数据, 就假定它与信息泄露无关。

侧信道可以按共享方式区分。空间共享指攻击者与受害者同时竞争同一结构, 例如 SMT 下的端口或 L1 cache; 时间复用指受害者离开后, 攻击者读取残留状态, 例如上下文切换后的预测器和缓存。前者通常带宽高, 后者更依赖状态保持时间和切换行为。

攻击还可分为基于占用和基于冲突。占用通道测量结构整体使用量或延迟, 通常不需要精确地址映射; 冲突通道构造与受害者映射到同一 set、bank 或 predictor entry 的访问, 分辨率更高, 但需要了解或推断索引函数。

计时器精度会影响攻击难度, 却不是唯一观测方式。攻击者可以通过重复执行放大微小差异, 使用共享线程计数, 或把资源状态转换为更容易测量的事件。单纯降低软件时钟精度通常只能增加样本数量, 不能消除根本共享通道。

15.10 SMT and Shared State

同时多线程(SMT)让不同线程并发共享前端、执行端口、缓存、TLB 和后端队列, 通常提高资源利用率, 却也提供高带宽的同核观察条件。攻击者可以更连续地测量受害者造成的端口、缓存或队列变化, 而不必等待上下文切换后残留状态仍然存在。

完全禁用 SMT 可以消除一类同核并发通道, 但会损失吞吐, 也不能解决跨时间复用或共享 LLC 的所有问题。更细粒度的设计包括按线程标记预测状态、资源分区、敏感切换时清洗, 以及调度器避免不可信安全域同核共驻。其效果取决于实际共享的结构, 而不是 SMT 这一名称本身。

静态资源分区能够限制一个线程观察另一个线程的占用, 却会产生碎片; 动态共享保持利用率, 但资源使用本身就是信号。随机仲裁和噪声可以降低可测性, 却难以提供严格隔离。安全敏感设计通常需要明确哪些资源必须硬分区, 哪些资源只需在调度层避免不可信共驻。

即使 L1 cache 与 TLB 按线程区分, 执行端口、前端带宽、ROB、scheduler 和互连仍可能共享。攻击者可以通过自身吞吐变化推断受害者使用了哪类指令或发生了何种 cache miss。完整 SMT

安全分析不能只检查传统 cache partitions。

15.11 Mitigation Principles

微架构缓解可以按切断泄露链的位置分类。**限制错误推进**包括投机屏障、降低特定预测或序列化高风险操作；**阻止敏感数据传播**要求权限确认前不向依赖操作提供数据，或把未验证值标记为不可用于地址和控制流；**隔离共享状态**包括按地址空间或安全域标记、分区、清洗和限制跨域训练；**降低可观测性**则尝试削弱高分辨率计时或资源共享，但通常只能增加攻击成本。

缓解应同时评价安全覆盖和性能代价。广泛序列化可以缩短瞬时窗口，却会牺牲乱序与内存级并行；频繁清洗预测器或缓存会增加切换成本；严格分区提高隔离，却降低资源利用率。更合理的设计是在明确安全边界后，只对真正跨域的状态和高风险数据流施加约束。

软件、编译器和硬件需要共同维持缓解语义。硬件提供屏障、预测控制和状态管理接口；编译器识别可能被优化破坏的依赖与边界；操作系统在上下文切换和调度中管理共享范围。任何一层对接口含义理解不一致，都可能留下新的瞬时路径。

预测器标记可以把 ASID、VMID、特权级或线程身份加入索引或标签，减少跨域训练。完整标记会增加存储和访问延迟，压缩标记仍可能别名；在同一安全域内保留共享有利于准确率，在域切换时更新标识则需要防止旧项被错误复用。

状态清洗适合生命周期明确、容量有限的结构。对大型 cache 或复杂 predictor，完全清空代价高且可能缺少精确接口；选择性失效需要结构能够按安全域定位状态。无法有效清洗时，调度隔离或持续分区可能更可靠。

安全检查前移能够从源头阻止秘密进入瞬时数据流，但可能延长 load 或 target generation 关键路径。另一种方法是允许操作继续执行，却为未验证值附加 taint 或限制其用于地址、分支和共享状态更新。后者保留更多并行性，但数据流追踪复杂度更高。

缓解还需考虑组合性。针对 BTB 的隔离可能不覆盖 BHB，缓存分区不覆盖执行端口，投机屏障也不清除此前留下的预测状态。安全声明应明确覆盖的入口、载体和边界，避免用单一“已缓解 Spectre”概括多种机制。

15.12 Mitigation Lifecycle

微架构漏洞的修复通常不是单个补丁，而是一条跨层生命周期。首先由处理器型号、能力位和漏洞枚举确定系统是否受影响；随后由微码或固件提供新的硬件行为与控制接口；操作系统和 hypervisor 在上下文切换、特权切换及虚拟机切换处使用这些接口；编译器、运行时与应用再处理需要局部屏障、代码改写或地址空间隔离的路径。任一层缺失，都可能使硬件已经提供的缓解没有真正覆盖软件边界。

枚举能力与启用状态必须分开。处理器支持某个清洗指令或控制位，不等于固件已经加载所需微码，也不等于操作系统在正确的边界调用它；虚拟机看到的能力还可能由 hypervisor 合成。系统审计应同时确认处理器受影响状态、微码版本、内核策略、虚拟化暴露方式和应用是否依赖特定编译器转换。

缓解的作用域也可能是每线程、每核心或全系统。按线程控制适合只限制运行不可信代码的进程；共享于物理核心的缓冲和预测状态则可能要求 sibling thread 同步清洗、可信共调度或暂时闲置；全局微码控制会统一改变所有工作负载。若忽略作用域，局部启用可能留下跨线程路径，而全局启用又可能承担不必要的性能成本。

部署验证需要同时观察安全状态与性能状态。功能测试应确认控制位、缓冲清洗和屏障在真实切换路径中生效;压力测试则覆盖系统调用、虚拟机切换、SMT 共驻、向量 gather、分支密集与高访存并行负载。缓解造成的性能变化应追溯到被限制的具体机制,而不是只给出一个脱离工作负载和威胁模型的平均百分比。

15.13 Security Evaluation

评价一种微架构机制时,应依次确定:攻击者能够影响哪一种预测或推测;敏感数据在何种条件下进入瞬时数据流;哪些结构保存了数据相关痕迹;这些结构是否跨越目标安全域共享;攻击者能否以足够信噪比恢复差异;缓解又切断了传播链中的哪一环。

实验还必须区分攻击原语与完整边界突破。能够在同线程中测出缓存差异,不等于能够稳定跨进程或跨特权级泄露;理论上共享的结构,也可能因噪声和调度条件难以利用。反过来,低带宽通道在长期密钥或控制流泄露中仍可能具有安全意义。

评估应报告训练条件、受害者与攻击者的调度关系、地址共享要求、计时来源、成功率和泄露带宽。只展示最佳条件下的单次时序差异,无法说明真实攻击稳定性;只报告平均性能损失,也无法说明缓解是否覆盖了全部上下文。

性能评估需要覆盖常见与最坏工作负载。预测状态清洗可能主要伤害分支密集程序,屏障可能限制内存级并行,禁用 SMT 则影响高线程吞吐。缓解成本应与它保护的安全边界和攻击前提对应,而不能脱离部署模型给出单一百分比。

形式化模型可以帮助验证允许的信息流,但必须包含足够的微架构状态和时间行为。过度抽象的模型可能证明体系结构结果正确,却遗漏 cache replacement、预测历史和资源竞争。实验攻击与形式化分析因此是互补关系。

现代处理器安全的基本张力在于:性能设计希望更早预测、更远执行并更广泛共享状态,安全设计则要求边界在敏感信息传播前生效。将微架构状态视为需要明确生命周期、共享范围和恢复语义的安全资源,是处理这一矛盾的前提。

Part II

Microarchitectures Application

Chapter 16

AMD Microarchitectures

16.1 Zen 4

16.1.1 Overview

Zen 4 延续了 Zen 3 的宽发射乱序框架,但并不是一次单纯的工艺迁移。它同时扩大分支预测、Op Cache、退休队列、物理寄存器和访存队列,并把私有 L2 容量提高到 1 MB;执行侧则首次在 AMD 通用 CPU 核心中加入 AVX-512。AMD 给出的代际评估称, Zen 4 相较 Zen 3 的平均 IPC 提升约 14%,并在相同电压下获得约 16% 的频率提升。这些数字来自特定服务器工作负载与实现条件,不能直接解释为所有程序都具有固定幅度的加速。[40], [41]

从核心组织看, Zen 4 的重点可以概括为三条路径。第一,前端用更大的 BTB、条件分支预测器和 Op Cache 减少错误路径与译码停顿;第二,中段用 6-wide dispatch、320 项在途窗口和 9-wide retirement 扩大可并行工作量;第三,后端用四条整数 ALU、三条 AGU、六条浮点/向量路径以及更大的 load/store queue 消费这些工作。AVX-512 并未通过增加完整的 512-bit 执行数据通路实现,而是让多数 512-bit 操作在现有 256-bit 数据通路上连续执行两个周期。[40], [42], [43]

结构	主要参数
控制流	L1 BTB 容量较 Zen 3 增加约 50%, L2 BTB 增加约 8%, 最高每周预测两个 taken branch
指令供给	常规译码路径最高每周期 4 条指令;Op Cache 最大容量约 6.75K operations, 峰值可提供 9 个 macro-ops/cycle
在途状态	6-wide dispatch、最多 320 个在途 macro-ops、最多 9 个 macro-ops/cycle retirement
整数执行	4 个独立整数 scheduler, 4 条 ALU、3 条 AGU、2 条分支执行路径
浮点与向量	2 个浮点 scheduler、6 条浮点/向量路径、64 项 non-scheduling queue; AVX-512 使用 256-bit 数据通路
访存	每周期最多 3 个 256-bit memory operations, 最多全部为 load、其中最多两个为 store; 72 项 load queue、64 项 store queue
私有缓存	32 KB、8 路 L1I; 32 KB、8 路 L1D; 每核 1 MB、8 路统一 L2

Table 16.1: Zen 4 的主要核心级参数。不同宽度位于取指、调度、执行和退休的不同阶段,不能直接相加为统一的 IPC 上限。

16.1.2 Design Positioning: Deepening a Six-Wide Core

Zen 4 的中段仍以每周期分派 6 个 macro-op 为中心。前端 Op Cache 可以在命中时提供更多操作, 退休端也能每周期提交 9 个操作, 但进入执行引擎的长期供给仍受到 dispatch 宽度限制。这种前宽、后宽、中间略窄的组织并非浪费: Op Queue 可以吸收前端短期波动, 较宽退休端可以快速释放已经完成的窗口项, 而 dispatch 则维持寄存器重命名、资源分配和调度器写入的周期时间。

扩大窗口的目的也不是让任意 320 个操作同时参与全局选择。Zen 4 把整数和浮点操作送入多个独立 scheduler, load/store 操作还要占用各自的队列和访存资源。窗口容量决定处理器可以跨越多长的依赖链和缓存停顿寻找工作, scheduler 则决定某一类操作中有多少候选可以等待发射。两者相互关联, 却不能等同。

16.1.3 Front End

16.1.3.1 Branch Prediction and Control-Flow Bandwidth

Zen 4 增大了两级 BTB 和条件分支预测器。L1 BTB 可记录的分支数较 Zen 3 增加约 50%, L2 BTB 增加约 8%; 分支预测器在适合的控制流中可以每周期生成两个 taken target。这个能力的主要价值, 是在短基本块和高分支密度代码中跨越更多控制转移, 使 Fetch Queue、L1I 与 Op Cache 不必逐个基本块等待目标生成。[40]

两个 taken branch/cycle 描述的是目标预测带宽, 并不保证后续阶段可以从两个任意地址持续取得、译码并分派完整指令块。第二个目标可能受到 BTB 命中、分支类型、缓存行边界、页边界和前一个目标内分支位置的限制; 即使目标已经生成, L1I、Op Cache 与 Op Queue 仍可能成为独立瓶颈。因此, 评价 Zen 4 前端时应区分预测器向前推进的距离与实际交付到 dispatch 的有效操作数。

分支预测错误的恢复代价随路径和命中层级变化。优化手册给出的范围约为 11–18 个周期, 常见值约为 13 个周期。这个数字包含错误检测、状态恢复、前端重定向以及正确路径重新进入流水线过程, 不应被解释为所有分支均具有固定的 13 周期代价。更大的 BTB 和方向预测器减少错误发生频率, 而较宽退休与更大的窗口则决定一次错误会丢弃多少在途工作。[42], [43]

16.1.3.2 Instruction Cache, Decode, and Op Cache

Zen 4 的 L1 指令缓存为 32 KB、8 路组相联, 缓存行为 64 B。常规译码路径最高每周期处理 4 条 x86 指令; 对于热点代码, Op Cache 保存已经转换为内部 macro-op 的结果, 使前端绕过变长指令边界识别与重复译码。Op Cache 的最大容量约为 6.75K operations, 命中时峰值供给达到每周期 9 个 macro-op。[42], [43], [44]

与 Zen 3 相比, Zen 4 的 Op Cache 项数增加约 50%, 单个取指组可携带的最大操作数由 8 提高到 9。容量增加提高大型热点循环和常用函数的覆盖率, 操作密度提高则使同样数量的标签与控制状态能够承载更多有效工作。不过, Op Cache 的 9-wide 峰值并不会直接转化为 9-wide 执行: 操作仍要进入 Op Queue, 并受到 6-wide dispatch、后端资源可用性和每个 scheduler 接收限制的约束。[40]

16.1.4 Rename, Dispatch, Retirement, and SMT

Zen 4 每周期最多向执行引擎分派 6 个 macro-op, 并可在非 SMT 模式下跟踪最多 320 个在途 macro-op。退休单元每周期最多提交 9 个操作, 因此在大量短延迟操作同时完成时, 可以比

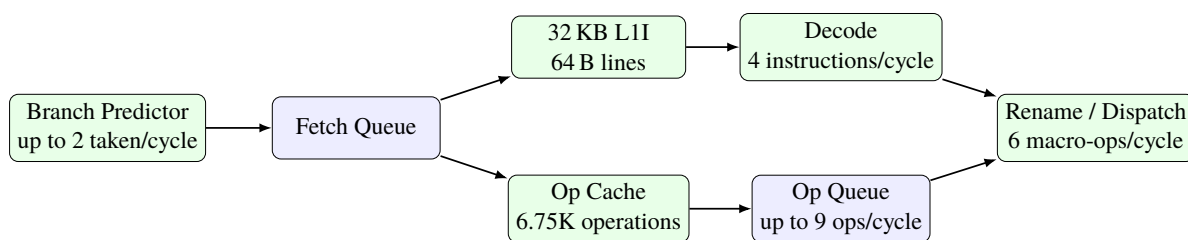


Figure 16.1: Zen 4 前端的两条指令供给路径。Op Cache 提供更高峰值带宽，常规 L1I 与译码路径承担未命中的代码；两条路径最终均受 6-wide dispatch 约束。

dispatch 更快地回收 ROB/RCU 项、物理寄存器和队列资源。更宽退休并不会提高未完成操作的执行速度，却可以避免完成端成为持续瓶颈。[42], [43]

整数物理寄存器文件包含 224 项，重命名阶段每周期最多消除 6 个满足条件的 move。Move elimination 通过让目标体系结构寄存器直接引用已有物理值，避免相同数据再次占用 ALU、scheduler 和写回端口。对于 x86 编译器产生的大量寄存器搬移与调用约定代码，这种优化能够提高窗口中真正计算操作的比例。[42], [43]

AMD 同时扩大了浮点物理寄存器、load/store queue 与退休队列。厂商估计中，Zen 4 的 SMT 吞吐增益由 Zen 3 的约 25% 提高到约 34%。这说明更深的共享资源池让第二个线程更容易填补一个线程留下的执行空槽，但该结果依赖工作负载与测试方法；它并不意味着任意两个线程都能获得相同增益。两个线程仍会竞争前端、dispatch、scheduler、缓存和带宽，资源扩张减少了竞争概率，却没有消除线程间干扰。[40]

16.1.5 Integer Scheduling and Execution

整数后端由四个相互独立的 scheduler 组成。它们合计每周期最多接收 6 个 macro-op，但单个 scheduler 每周期最多接收两个，并向各自关联的执行路径发射操作。每个 scheduler 都连接一条通用 ALU；前三个还各连接一条 AGU，首尾两个 scheduler 则各具有一条分支执行能力。分布式组织缩短局部选择和旁路距离，但也引入端口不对称：一个操作能否及时执行，取决于它是否进入具有对应功能的 scheduler。[42], [43]

四条 ALU 并非完全同构。ALU0 还承担整数除法和一部分分支操作；整数乘法主要由 ALU1 完成，CRC、PDEP/PEXT 等复杂位操作也集中在有限路径；普通移位、旋转和位提取则可使用更多路径。Zen 4 改善了若干位操作指令的延迟，但如果程序连续生成只能由同一功能单元执行的操作，名义上的四条 ALU 仍可能退化为单端口吞吐。[42], [43]

三条 AGU 为 load 与 store 生成地址，并与后文的三条访存流水线对应。地址生成和数据访问仍是两个阶段：AGU 可以开始三个地址计算，不表示所有请求都能同时命中不同 L1D bank，也不表示三个结果都能在同一周期完成转发。复杂寻址、跨缓存行访问、DTLB miss 和 store-forwarding 冲突都会降低有效吞吐。

16.1.6 Floating-Point Unit and AVX-512

Zen 4 的浮点后端使用两个相互独立的 scheduler，合计连接六条浮点/向量路径。四条主计算路径承担加法、乘法、FMA、比较、转换、逻辑和置换等操作，另外两条路径主要处理 store 与浮点到整数域的数据移动。浮点 macro-op 还可以先进入 64 项 non-scheduling queue，即使某个浮点 scheduler 暂时已满，前端仍可在有限范围内继续分派并等待后端腾出空间。[42], [43]

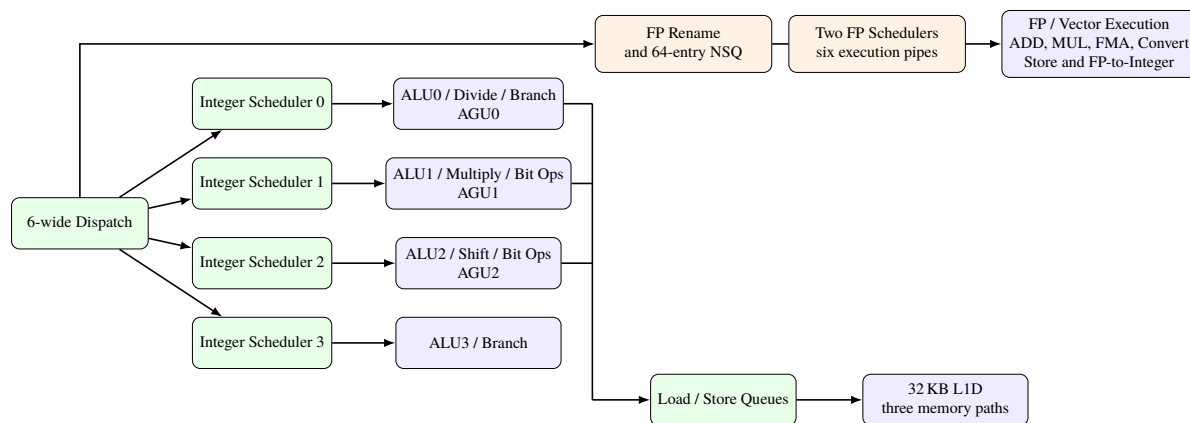


Figure 16.2: Zen 4 分布式整数调度、浮点调度与访存路径。执行资源按 scheduler 分组, 功能并不完全对称; 该图表示公开的逻辑关联, 而非完整物理旁路网络。

AVX-512 将 64 位模式下可见的体系结构向量寄存器总数扩展到 32 个, 相比 AVX2 增加 16 个, 并引入 opmask 掩码语义以及 VNNI、GFNI、BF16 和更完整的 scatter/gather 支持。更宽的体系结构向量可以在单条指令中表达更多数据并行工作, 减少指令取指、译码和窗口跟踪数量; 掩码执行还能降低处理尾部元素和条件向量化所需的控制流。[40]

Zen 4 的关键取舍是保留 256-bit 数据通路。多数 512-bit 操作被拆成上下两个 256-bit 部分, 在连续两个周期中通过相同的执行资源; 寄存器文件和重命名状态则维持 512-bit 体系结构语义。这样可以避免为少量 AVX-512 代码复制完整的 512-bit ALU、旁路与寄存器端口, 并控制面积、峰值功耗和时序压力。代价是单条 512-bit 指令通常不会比两条等价的 256-bit 指令提供两倍执行吞吐, 其主要收益来自更高代码密度、更少前端操作和更强的掩码/专用指令语义。[40], [42]

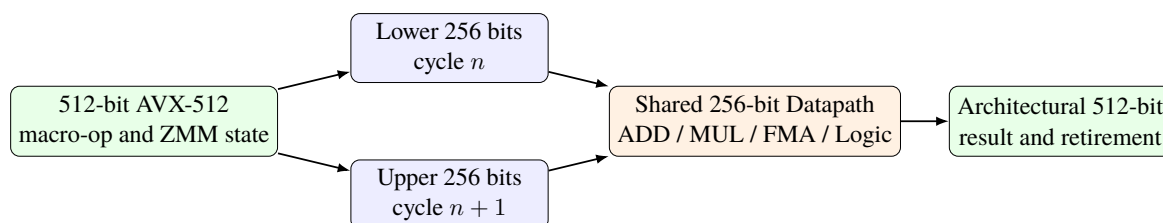


Figure 16.3: Zen 4 AVX-512 的概念性执行方式。512-bit 体系结构操作通常分为两个 256-bit 部分连续通过同一数据通路, 因此节省前端和窗口状态, 但不自动翻倍执行带宽。

AVX-512 是否提高实际性能取决于指令组合。若代码受取指、译码、寄存器压力或掩码控制限制, 减少指令数可以直接改善吞吐和能效; 若代码已经受 256-bit FMA、load/store 或内存带宽限制, 改写为 512-bit 指令的收益可能较小。因而, Zen 4 的 AVX-512 更适合被理解为扩大 ISA 表达能力并降低前端开销, 而不是把核心改造成完整的 512-bit 数据面。

16.1.7 Load/Store Unit and Memory Ordering

Zen 4 的 LSU 包含三条大体独立的 256-bit 访存流水线。三条均可执行 load, 其中最多两条可以同时承担 store; 核心可跟踪 72 个乱序 load 和 64 个 store。三条 AGU 与三条访存路径共同提高地址生成和 L1D 请求带宽, 使更大的乱序窗口能够同时维持更多独立访问。[42], [43]

L1D 为 32 KB、8 路组相联, 并支持 byte-writable 更新。较细粒度的写入能力可以减少小尺寸 store 因读改写或端口合并产生的冲突, 特别适合对象字段、栈变量和网络协议头等频繁更新少数

字节的代码。不过, byte-writable 并不意味着任意三个 store 都能无冲突完成; 同一 bank、跨缓存行、未对齐访问和 store-data 路径仍会降低吞吐。[40], [44]

简单地址生成且 L1D 命中时, 整数 load-to-use 延迟最低为 4 个周期, 部分地址模式需要 5 个周期; 浮点/向量 load-to-use 延迟约为 7 个周期。这个延迟只覆盖近端命中路径, 不能代表发生 DTLB miss、store forwarding、bank conflict 或缓存未命中时的等待。三条 load 路径提高的是并行读取能力, 而单条指针依赖链仍受每次 load 返回之后才能生成下一个地址的串行限制。[42], [43]

较年轻的 load 可以在预测为无冲突时绕过地址尚未解析的更老 store; 随后若检测到别名关系, 就必须重放 load 及其依赖操作。Zen 4 扩大 load/store queue 可以容纳更多等待地址、数据或缓存结果的操作, 但也增加地址比较、转发和违反顺序检测的压力。评价其访存能力时, 需要同时考虑 load/store queue 容量、store-to-load forwarding 条件、L1D bank 分布与未命中处理资源, 而不能只引用三条 AGU。

16.1.8 Translation and Data Reach

Zen 4 同时扩大一级和二级 DTLB, 以匹配更大的 load queue 和更高访存并行度。数据 TLB 容量增加可以减少服务器程序、数据库和虚拟机在 4 KB 页工作集扩大时触发 page walk 的频率; 二级 TLB 则在一级未命中时提供比完整页表遍历更低的后备延迟。[40]

地址翻译与缓存访问并不是完全独立的两条路径。L1D 访问需要及时获得物理标签, 预取器也需要避免在跨页时生成无效请求; 当多个 load 同时造成 DTLB miss 时, page-walk 请求还会与普通数据访问竞争缓存和内存带宽。因此, 扩大 TLB 的主要意义不仅是增加 translation reach, 也是在三条访存流水线持续工作时降低翻译侧反压。

16.1.9 Private and Shared Cache Hierarchy

Zen 4 每核配置 32 KB L1I、32 KB L1D 与 1 MB 私有统一 L2。L2 容量是 Zen 3 的两倍, 同时增加可并行处理的未命中请求, 以提高 memory-level parallelism。更大的 L2 可以直接容纳更多代码、堆对象和页表数据, 也使预取器有更大的提前填充空间; 但若程序工作集远超 1 MB, 性能仍取决于共享 L3、片上互连和内存系统。[40], [44]

标准 Zen 4 CCD 由 8 个核心共享 32 MB L3, 每个核心拥有独立的 1 MB L2。3D V-Cache 产品在同一 core complex 上堆叠额外 SRAM, 使总 L3 容量最高达到 96 MB。堆叠缓存改变的是共享容量和远端未命中率, 并不改变 Zen 4 核心内部的 dispatch、scheduler 或执行端口; 它应被理解为核心之外的缓存层次扩展。[40], [44]

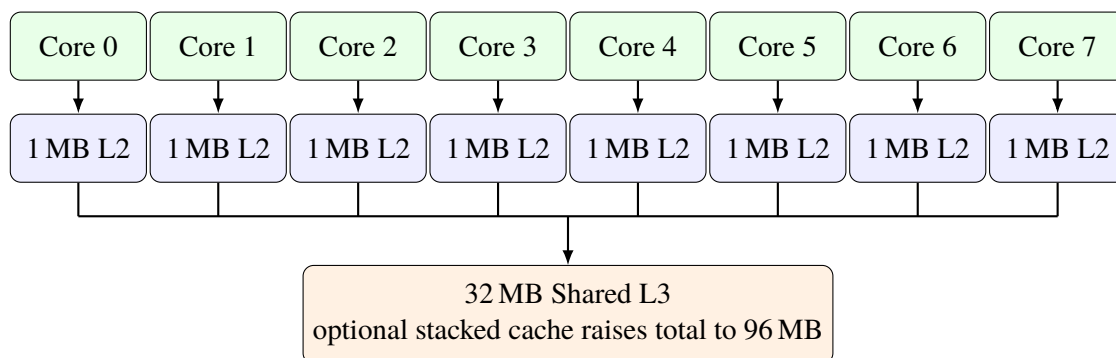


Figure 16.4: 标准 Zen 4 CCD 的缓存层次。每核拥有 1 MB 私有 L2, 8 个核心共享 32 MB L3; 3D V-Cache 扩大共享容量, 而非改变核心流水线。

增加 L2 容量与未命中并行度会把更多压力推向共享 L3 和片外内存。若多个核心同时产生大量预取或 demand miss, 单核可见的有效延迟会受到 L3 仲裁、CCD 到 I/O die 链路与内存控制器排队影响。因此, Zen 4 的缓存改进不能仅用“L2 翻倍”概括; 容量、访问延迟、在途请求数量和共享层拥塞需要同时考虑。

16.1.10 Zen 4c: The Same ISA Under a Different Physical Target

Zen 4c 保持与 Zen 4 相同的 IPC 目标和体系结构功能, 但采用更紧凑的物理实现与较低最高频率。核心加 L2 的面积约减少 35%, 使单个计算晶粒可以从 8 个 Zen 4 核心扩展到 16 个 Zen 4c 核心。两者每核均配置 1 MB L2; 为了提高密度, Zen 4c 每核对应的共享 L3 容量由约 4 MB 降至约 2 MB。[40]

项目	Zen 4	Zen 4c
体系结构功能与 IPC 目标	相同 Zen 4 功能集合	相同 Zen 4 功能集合
主要优化目标	较高单线程频率与性能	核心密度与功耗效率
核心加 L2 面积	基准	约减少 35%
每计算晶粒核心数	8	16
每核私有 L2	1 MB	1 MB
名义平均共享 L3 容量	约 4 MB/core	约 2 MB/core

Table 16.2: Zen 4 与 Zen 4c 的核心级差异。Zen 4c 的主要变化来自物理实现、频率目标和共享缓存密度, 而非采用不同 ISA。

“相同 IPC”并不意味着在任意产品和功耗下具有完全相同的运行时间。更低频率、更高核心密度、每核较少的共享 L3 以及更多线程对内存带宽的竞争, 都会改变系统级表现。Zen 4c 的意义在于用相同软件目标覆盖吞吐导向产品, 而不是用更小核心取代所有高频 Zen 4 场景。

16.1.11 Summary

Zen 4 的微架构演进并不依赖某一个特别夸张的宽度。前端通过更大的 BTB、条件预测器和 6.75K-operation Op Cache 提高正确路径供给; 中段通过 6-wide dispatch、320 项窗口、较大的寄存器文件和 9-wide retirement 延长并行观察范围; 整数后端以四个独立 scheduler 驱动四条 ALU、三条 AGU 和两条分支路径; 访存侧则以三条 256-bit 流水线、72 项 load queue、64 项 store queue 和 1 MB L2 提高数据并行度。

AVX-512 最能体现 Zen 4 的工程取舍。AMD 提供完整的 512-bit ISA 语义, 却复用 256-bit 数据通路分两周期执行多数宽向量操作, 以控制面积和功耗。它不会自动把浮点峰值翻倍, 却能减少指令数量、前端带宽和窗口占用, 并提供掩码、VNNI、BF16、GFNI 与更强 scatter/gather 等能力。

因此, Zen 4 更适合被理解为一颗在多个约束点同步扩张的高频 x86 核心: 预测器让未来路径更早出现, Op Cache 降低译码成本, 更大的窗口和队列保留更多独立工作, 执行端与缓存层次则尽量避免这些工作因端口或数据供给不足而停顿。其真实性能仍取决于分支可预测性、Op Cache 覆盖率、端口组合、地址独立性和共享缓存压力, 而不是由单一的 IPC、频率或 AVX-512 标签决定。

16.2 Zen 5

16.2.1 Overview

2024 年,AMD 发布 Zen 5 微架构,并将其定位为未来多代产品的基础。与 Zen 4 相比,AMD 宣称 Zen 5 的平均 IPC 提升约 16%;这一数字来自厂商选定的工作负载集合,实际增益仍随频率、编译器、缓存配置与程序特征变化。客户端产品的最高加速频率可达 5.7 GHz。[45], [46], [47] Zen 5 扩展了 AVX-512 支持,并在高性能核心中配置原生 512-bit 浮点与向量数据通路;分派(dispatch)宽度提高到每周期 8 个 macro-op, 整数侧提供 6 条 ALU 路径。取指与解码(fetch/decode)阶段采用双流水线前端。不同产品中的 Zen 5 与 Zen 5c 计算晶粒采用 4 nm 或 3 nm 级工艺,每个核心支持两个硬件线程。值得注意的是,尽管 Zen 5 通过引入更多逻辑以提升性能,工艺与版图优化在一定程度上抵消了面积增长,使其核心面积与 Zen 4 大体相当。[47], [48]

每个 Zen 5 计算核心复合晶粒(CCD)集成 8 个 Zen 5 核心。在服务器产品(如 EPYC 9005)中,单个 socket 最多可配置 16 个此类 CCD,从而实现最高 128 个 Zen 5 核心;每个 CCD 共享 32 MB 的 L3 缓存。相较上一代,该配置通过更多核心、更宽内存接口和更强的核内执行能力提高整体吞吐。

对于 Zen 5c,单颗处理器最多可集成 12 个 die,且每个 die 包含 16 个 Zen 5c 核心。各 die 同样共享 32 MB 的 L3 缓存,因此可根据具体配置实现约 96–192 核的规模。[48], [49]

16.2.2 Instruction Fetch Unit (IFU)

相较前代,Zen 5 的取指单元(IFU)发生了显著变化。其中较为关键的设计取舍包括采用双流水线前端,并对分支预测器进行重构。总体而言,Zen 5 在预测相关结构上投入了更多资源;前端的系统性改造很可能是其 IPC 提升的重要来源之一。

16.2.2.1 Branch Predictor

随着后端执行带宽的提升,前端需要在每个周期向后端输送更多“有效”指令,因此分支预测成为前端的关键组成部分。围绕 Zen 5,AMD 显著扩大了目标预测、返回预测与取指跟踪资源。结合后文的 MMU、调度与执行结构可以看出,这并非孤立扩大某一张预测表,而是通过增加前端前瞻距离,为更宽的分派与执行后端持续准备正确路径。[47], [50]

Zen 5 的分支误预测恢复开销约为 12–18 个周期,具体值取决于分支类型以及操作是否由 Op Cache 供给。[50] 与部分旗舰级 ARM 与 RISC-V 核心相比,这一数值可能处于劣势:许多现代设计已将恢复开销压缩至约 11 个周期,部分甚至低于 10 个周期。鉴于预测精度的提升往往呈现边际收益递减,尤其在 RISC 设计中,越来越多的架构师倾向于通过缩短误预测惩罚来提升整体性能。需要指出的是,x86 需要在复杂 ISA 约束下实现极高的目标频率(5 GHz+),其流水线深度与控制流特性难以与部分 RISC 核心直接对齐,因此不同架构之间的取舍并不完全可比。

目标地址预测的基本粒度为 block。每个 block 为 64 B;若按平均 4 B/指令估算,则对应的理论预测带宽约为每周期 16 条指令。结合 x86 的常见统计特性(例如平均约每 ~12 条指令出现一次 taken branch、每 ~6 条指令出现一次分支),该粒度可视为较为保守但合理的设计点。若未来能够更有效地缓解实际取指带宽的限制(通常受 cache line 对齐与分支布局约束),block 粒度可能进一步增大。即便峰值取指带宽不变,适度增大 block 仍可能带来一定收益。

AMD 在优化手册中明确指出,Zen 5 的 L1 BTB 规模为 16K entries,较 Zen 4 的约 ~1.5K entries 提升显著(约 10×)。对于 direct call、目标稳定的间接分支(same-target indirect branch)、条件直接分

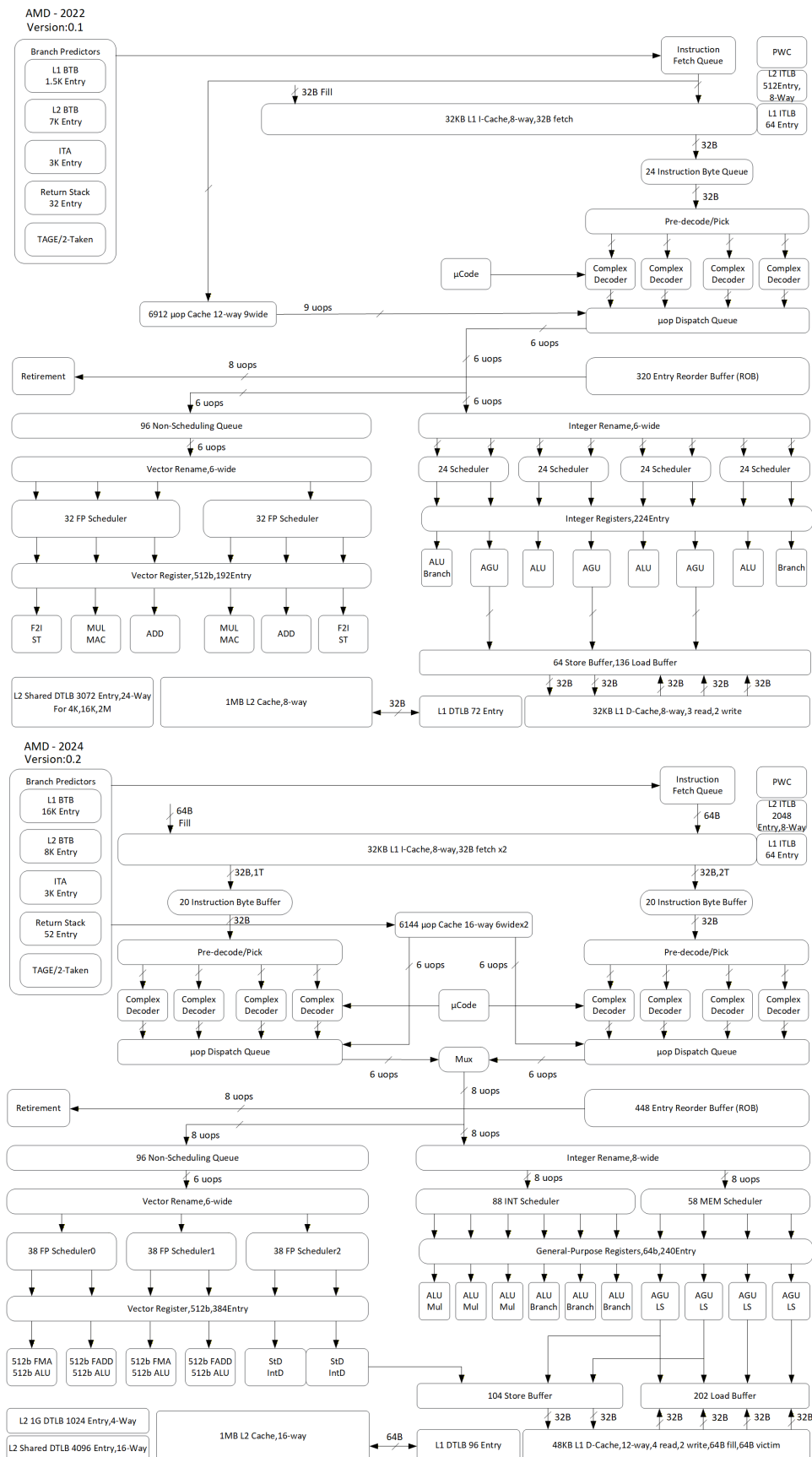


Figure 16.5: Zen 4 与 Zen 5 的微架构概览。图中的容量和宽度属于不同流水阶段, 不能直接相加为统一吞吐上限。

支以及无条件直接分支, L1 BTB 可实现 0 周期预测气泡; 而对于返回 (return) 与多目标间接分支, 则对应 2 个周期的预测气泡。考虑到一级 BTB 同时受到容量、访问延迟、目标编码和功耗限制, AMD 在 Zen 5 上的资源配置可被视为较为激进。[50]

L2 BTB 规模为 8K entries。L1 BTB miss 后需要进入更慢的后级目标路径, 因此目标虽然可能仍被记录, 前端却会暴露额外预测气泡。分级 BTB 的本质是在低延迟命中与较大代码覆盖范围之间分工: 一级承担热点控制流, 后级扩大覆盖, 但不能提供与一级相同的目标到达时间。[50]

间接目标预测器常被描述为约 3072 entries。由于间接分支往往具有多个潜在目标, 预测器通常基于分支历史从候选目标中选择最可能的目标。对于长期收敛到单一目标的间接分支, 常规 BTB 往往即可提供较低延迟的预测; 当此类模式较为普遍时, 微架构通常会在硬件层面进行专门优化。这也对应一条软件层面的经验: 应尽量避免具有过多潜在目标的间接分支, 因为其对预测硬件不友好, 且往往会带来可观的性能损失。关于其具体算法的公开信息仍然有限, 不能仅凭容量和行为将其确定为某一种 TAGE 变体。另有公开材料指出, 当某条间接分支的目标在足够长时间内保持稳定时 (阈值实现相关), 其目标地址可能被缓存至延迟更低的 L1 BTB。[2], [50]

从资源效率的角度看, BTB 通常不会对所有分支目标一视同仁地采用完全独立的存储方式; 相反, 工程实现往往会利用目标地址之间的对齐关系与高位共享特征, 以提高单位面积下的信息密度。直观地说, 位于一定地址范围内、且目标地址具有较强相关性的多个分支, 可能被压缩组织到同一 BTB 表项之中。相关文献与工程经验中, 常将这类编码方式概括为“长分支/短分支”等不同表示形式: 其中一类表项保存更完整的目标信息, 另一类表项则利用多个目标之间的共享部分, 仅记录必要的差分或补充字段。其核心目的并不复杂, 即在相同面积预算下提升可容纳的有效目标数量, 或者在相同性能目标下压缩预测器的实现成本。[3], [4]

就 Zen 5 而言, AMD 在优化手册中提到: 若分支目标跨越 16 MB 对齐边界, 则该分支将无法与其他分支共享 BTB 表项; 当此类情况频繁出现时, BTB 的有效容量与带宽都可能下降。更一般地看, BTB 的面积-性能优化往往建立在若干经验事实上: 其一, 大量的分支有相同的分支目标; 其二, 大量的分支目标共享页面; 其三, 大量的分支目标跳转不会越过一个页面。因此, 在具体实现中, 设计者往往会选择一个合适的目标地址共享粒度, 以在容量、延迟、功耗与编码复杂度之间取得平衡; 从设计经验看, 数 MB 到数十 MB 的量级都属于可以理解的工程取舍, 而这一阈值通常也会受到 MMU 组织方式、前端时序约束以及核内存结构特性的共同影响。[4], [50]

方向预测通常被认为采用了规模更大的 TAGE 类设计。文献中较常见的配置约为 8 张表, 部分研究指出当表数量超过约 12 之后, 预测精度提升的边际收益将明显下降。因此, 性能改进并非仅靠无限制增加表数量即可实现, 实践中往往是在 TAGE 框架之上对参数进行细致调优。此外, 一些现代“难预测”分支的行为并不完全符合经典 TAGE 的建模假设, 从而促使研究者引入额外技术以捕获 TAGE 难以覆盖的模式。

一个值得注意的细节是, 有资料称 Zen 5 的 BTB 采用了双端口 SRAM 结构, 这在预测器实现中并不常见。该结构有助于实现每周期预测两条 taken 分支, 并且相较某些替代方案 (例如限制连续 block 或禁止跨 cache line) 具有更少的约束条件。这一设计取向与经典工作 “Multiple-Block Ahead Branch Predictors” 所提出的思路具有一定相似性。[5]

架构师对多分支预测的说明

George Cozma: Can the branch predictor do two taken and two ahead branches in a cycle?

Or is it two branches in total?

Mike Clark: It's complicated; there are rules around when we can do two ahead.

But yes, in general, I would say yes.

George Cozma: So yes to two branches per cycle, or yes to two ahead and two taken branches?

Mike Clark: In certain situations, we can do both.

[51]

从原理上看, 该机制具有较强的前沿性。若 BTB 的确采用双端口实现, 则其与部分既有分析中“指令缓存 (I-cache) 可能亦为双端口”的推断相一致, 也与诸如“由双端口取指为解码器供给指令”(They get fed from the dual ported instruction fetch to feed the decoder) 之类的表述相呼应。不过, 后续讨论也指出, 双解码流水线未必与“双端口取指”存在一一对应关系。单线程模式下, 一个线程可以使用两组解码资源以及双端口 Op Cache; 进入双线程模式后, 两组 decoder 才分别分配给两个线程。因而, 双解码结构既提高单线程峰值供给, 也在 SMT 模式下减少两个线程对同一 decoder 的直接争用。[51]

传统的预测流程可概括如下: 令 i 表示一个 block 内有效指令的起始位置。由于以 SRAM 实现的大规模 BTB 通常在下一周期返回结果, 因此前端往往每周期最多获得一个 block 的预测信息。例如, block A 中的 taken 分支 (记为 A_a) 跳转至 block B; 预测器返回 block B 的有效起始位置 (B_i) 以及该 block 内的分支位置 (B_b), 随后再依据 B_b 的目标地址推导下一 block 的位置, 并重复上述过程。理论上, 若采用由触发器 (DFF) 构成的全相联预测结构, 可在单周期内完成更多工作; 但其时序收敛往往较为困难。

相关文献提出的一类改进思路是: 用 A_a 的分支目标直接预测 C_i , 而非沿用“以 B_b 预测 C_i ”的传统链式方式; 同时仍使用 B_b 预测 D_i 。该方案通常需要双端口 SRAM 以支撑并行访问。

取指窗口跟踪结构采用 FIFO 组织, 用于保存分支预测相关的元数据, 其深度为 128 entries。在无分支的情况下, 每个 entry 至多可覆盖一个 cache line 的信息; 而当遇到分支时, 通常需要额外的 entry 来记录控制转移元数据, 因此单个 entry 所承载的“有效信息量”是可变的。总体而言, 128 entries 可视为一个相对合理的设计点。该 FIFO 仅在指令退休 (retirement) 阶段释放; 若其被填满, 则取指流水线将发生停顿 (stall)。

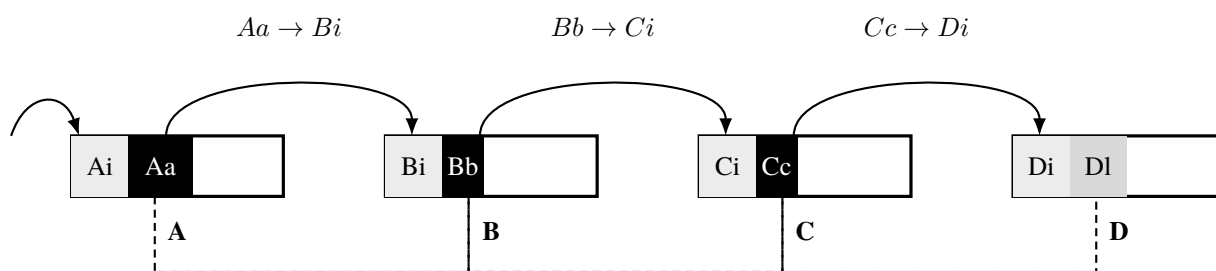


Figure 16.6: Multi-block ahead branch prediction flow (sketch).

16.2.2.2 ICache

指令缓存 (I-cache) 是前端最关键的组成部分之一, 相关研究在文献中已较为充分 (例如预取机制、替换策略与组织结构等)。就 Zen 5 而言, 其 L1 I-cache 为 32 KB、8 路组相联, cache line 大小为 64 B。核心理论上可在每周期取回 64 B 指令, 但需要两个对齐的 32 B 块, 这在特定场景下可能

构成瓶颈。单线程模式可以使用两条取指与解码路径;双线程模式则把两组 decoder 分配给不同线程。峰值 64 B/cycle 仍要求两个对齐的 32 B 块均可取得,并受到分支目标、cache-line 边界和下游队列空间限制。[50], [51]

L1 I-cache 采用 LRU 替换策略,并通过奇偶校验 (parity) 进行差错检测;这两项选择在业界均较为常见。更复杂的替换策略通常会显著增加面积与实现复杂度,因此对 L1 I-cache 往往“性价比”不高;相比之下,部分处理器在 L1 D-cache 上会采用略复杂的策略以追求命中率收益。与 Zen 4 相比, Zen 5 的理论取指带宽翻倍。如前所述, I-cache 是否为双端口实现仍未得到充分证实。此外需要强调,峰值取指带宽仅反映理论上限:已有测量表明,在某些工作负载下 Zen 5 的实际取指吞吐可能弱于 Zen 4。结合前端的大幅改动与较高峰值宽度, Zen 5 在特定场景中仍可能因目标生成、对齐、Op Cache miss 或代码 footprint 出现前端瓶颈;这与单线程是否能够使用两组 decoder 是不同问题。

AMD 与 ARM 在 I-cache 上的设计理念差异,也反映了二者在性能、功耗与面积 (PPA) 之间的不同取舍。自 Cortex-A76 起,许多 ARM 核心采用 64 KB、4 路组相联的 I-cache (部分实现支持容量配置);而 AMD 自 Zen 2 至 Zen 5 基本维持 32 KB、8 路组相联的 I-cache 组织。考虑到 x86 指令的信息密度通常更高,采用相对更小的 I-cache 在一定程度上也具有合理性 (尽管 CISC 与 RISC 的界限在持续弱化)。不同的组织形式会显著影响功耗、访问延迟与命中率。

一些较为通用的模型研究表明:在容量固定 (例如 32 KB 或 64 KB) 的条件下,将相联度由 4 路提升至 8 路通常会显著增加访问延迟。相反,若保持相联度不变而将容量从 32 KB 扩展至 64 KB,无论在 4 路还是 8 路设计中,延迟往往仅呈现小幅增长。在读取能耗方面,模型亦显示:当容量固定时,8 路设计的读能耗通常可超过 4 路的两倍;而在相联度固定时,将容量由 32 KB 提升至 64 KB 对读能耗的增幅相对有限。从该类模型出发,采用更低相联度并增加容量在延迟与能耗之间可能呈现更好的平衡。然而,一旦将面积、命中率以及工艺/定制 SRAM 等因素纳入考量,便难以给出普适的优劣结论。总体而言,上述对比提示 ARM 更偏向极致能效优化,而 x86 设计通常更强调峰值性能。

关于回填 (fill) 路径,处理器每周期可回填一个缓存行 (cache line),即 64 B。为缓解 L1 与 L2 之间潜在的访问延迟,业界通常采用两类主流方法:其一是引入更复杂的预取算法;其二是提高前端预测带宽,使其显著高于实际取指带宽,从而在较早阶段将指令提前取入 L1 指令缓存 (L1 I-cache)。然而,若预测精度不足,此类更激进的带宽配置可能导致额外的无效取指与资源浪费,进而使整体性能下降。

16.2.2.3 Op Cache

操作缓存 (Op Cache, OC) 用于存储先前已经完成译码和部分融合的操作序列。OC 命中时仍需要正确的预测地址与标签查找,但可以绕过常规 x86 指令边界识别和主译码路径。AMD 的优化手册明确给出: Zen 5 的 OC 采用 64-set、16 路组织,最大容量约为 6K 条指令或融合后的指令,峰值供给带宽可达每周期 12 条操作。相比之下,常规取指-解码路径在每个线程上最多提供每周期 4 条指令。两者差异较大,说明 OC 命中率对前端实际吞吐具有一阶影响。相较 Zen 4,名义容量的缩小更适合被理解为以更高有效存储密度换取前端效率的设计调整,而不宜简单视作资源退化。[47], [50]

从结构上看, OC 采用 64-set、16 路组织。每个 set-way (即一个 entry) 最多可存储 6 条指令或融合后的指令。优化手册还明确指出, OC 使用物理标签 (physical tags), 从而支持两个硬件线程之间的共享。指令融合作为一种提升信息密度的手段,可提高 OC 的存储效率;类似的“密度增强”技术也常见于其他 SRAM 结构 (例如 BTB 与 TLB)。[50]

对于流水线较深且目标频率较高的处理器而言,引入类似 op cache 的结构是一项较为成熟的工程优化,并非近年的新概念。在部分 RISC 设计中亦曾引入相似机制;是否保留操作缓存,取决于译码复杂度、热点代码覆盖率、访问能耗和前端流水线深度之间的权衡。[13]

Instruction fusion and density 关于指令融合与信息密度,AMD 的优化手册明确指出,Zen 5 支持将一条寄存器到寄存器(reg-reg)的 MOV 与其后续的 ALU 指令进行融合。[50] 其约束条件可概括如下:

- MOV 必须为寄存器到寄存器形式,且操作码为 0x89 或 0x8B。
- ALU 指令必须紧随 MOV 之后,且两者的目的寄存器一致。
- ALU 指令的源操作数必须为寄存器或立即数(不允许内存源操作数)。
- ALU 指令仅可使用 MOV 的源或目的寄存器作为其输入。
- 若 ALU 指令包含两个寄存器输入,则两者必须为不同寄存器。

16.2.3 Memory Management Unit (MMU)

在 L1 TLB 方面,ITLB 采用 64-entry 的全相联结构,覆盖 4 KB、2 MB 与 1 GB 页面;DTLB 采用 96-entry 的全相联结构,覆盖 4 KB、16 KB、2 MB 与 1 GB 页面。此外,针对 4 KB 页面,Zen 5 还支持对连续页面进行聚合(aggregation),即将 4 个连续页面合并为一个更大粒度的表示。总体而言,L1 TLB 以低延迟命中为首要目标,而连续页聚合用于提高有限表项的有效覆盖范围。[50]

L2 ITLB 采用 8 路组相联结构,共 2048 entries,用于缓存 4 KB 与 2 MB 页面。值得注意的是,1 GB 页面未被缓存至 L2 ITLB,而仅保留在 L1 ITLB。在指令地址转换场景下,操作系统通常较少为代码段分配 1 GB 大页;将其纳入 L2 ITLB 可能增加潜在探测(probe)范围,从而导致更大的访问延迟。因此,从访问延迟与能耗权衡角度看,将 1 GB 页仅保留于 L1 ITLB 可被视为合理设计点(尽管并非唯一选择)。[50]

L2 DTLB 采用 16 路组相联结构,共 4096 entries,用于缓存 4 KB 与 2 MB 页面;此外,另设一组独立的 4 路组相联 L2 DTLB,用于缓存 1 GB 页面,共 1024 entries。这种拆分体现了典型的功耗-性能-面积(PPA)权衡:将不同页大小分别缓存,通常有助于降低查找延迟,但也可能由于需要访问更多结构而增加瞬时功耗;反之,将不同页大小混合缓存则可能提升结构利用率并降低单周期功耗,但在某些情况下会引入更多查找与更复杂的管理。需要强调的是,上述权衡具有明显的工作负载相关性;实际设计取舍通常比该简化描述更为细致。[50]

值得注意的是,Zen 5 的 L2 TLB 配置规模较大。作为横向参照,公开资料描述 2024 年 ARM 旗舰服务器核心 V3 采用共享的 2048-entry L2 TLB;Intel 的 Lion Cove 则被描述为 1024+1024 entries 的配置。相较之下,Zen 5 的指令侧为 2048 entries,而数据侧为 4096+1024 entries。其中,4096-entry 的 DTLB 可被解释为在预取能力与地址覆盖范围(reach)方面采取了更为激进的资源配置。另一方面,针对 1 GB 页面单独设置 1024-entry 结构的动机相对不直观:当前软件生态是否需要如此大规模的 1 GB 地址翻译仍不明确,但该设计也可能出于平台长期演进与生命周期的考量。

一个常见疑问是:尽管 x86 在体系结构层面主要定义 4 KB/2 MB/1 GB 页面,但 TLB 结构中为何还会出现诸如 16 KB、4 MB 等“非标准”粒度。类似现象在部分 ARM 微架构中亦可观察到。这些粒度通常来源于硬件对连续页面的聚合(aggregation)、所谓的“smash”机制,以及体系结构支持的连续页提示(contiguous-page hints)等。其中,聚合的基本思想是将连续的虚拟地址(VA)与物

理地址 (PA) 范围用单个 TLB entry 表示以提升存储密度; 常见设计点为 $4\times$ 聚合, 即将四个 4 KB 页面合并为等效的 16 KB 区域。“Smash” 机制则指将大页拆分为小页后再进行缓存; 若该机制为纯微架构实现, 则在一致性维护与失效管理上通常需要更谨慎的设计。连续页提示通常由软件配置并由体系结构提供支持。例如, 部分 RISC-V 扩展支持将连续 16 个 4 KB 页面视为 64 KB 区域; ARM 也提供类似机制(往往同样以 16 页为粒度), 且在实践中通常更为完备。

在硬件页表遍历 (page walk) 方面, Zen 5 提供 6 个并行的页表遍历引擎。考虑到核心私有 TLB 的存在, 持续的 page-walk 流量通常不会长期处于极端水平, 因此业界常将 6–8 个页表遍历引擎视为偏“冗余”的设计点。此外, Zen 5 还采用页表遍历缓存 (page-walk cache, PWC) 和 64-entry Page Directory Cache 保存中间页表信息, 以减少重复读取页表层级。[50]

总体而言, CPU 的 MMU 微架构已相对成熟。与 GPU 相比, 主流 CPU 厂商在 MMU 设计上的差异通常不大, 更多体现为围绕少数关键维度进行的参数与 PPA 调优, 以及随体系结构与软件生态演进而引入的支持特性。近年来公开研究的重心也更偏向安全性与虚拟化支持、以及软硬件协同优化, 而非提出全新的 MMU 微架构范式。

16.2.4 Instruction Decode Unit (IDU)

解码单元 (IDU) 的一个显著特征是采用双流水线组织: 两条解码流水线以 $32\text{B}\times 2$ 的粒度工作, 从而匹配 L1 I-cache 每周期最多 64 B 的取指带宽。每条解码流水线配备一个深度为 20 的队列 (IBQ), 用于在预测/取指与解码之间实现解耦。每周期, 每条流水线可扫描 2 个 IBQ entry, 并最多输出 4 条指令。单线程模式下, 两组 decoder 均可服务同一线程; 开启 SMT 后, 硬件把一组 decoder 分配给每个线程。该策略使单线程保留完整前端能力, 同时在双线程模式下提供明确的译码带宽隔离。两条路径仍共享后续的 8-wide 分派入口, 因此峰值 8 条常规译码指令只在两个基本块、字节边界和合流条件均合适时实现。[50], [51]

两条解码流水线可分别处理不同的基本块 (basic blocks), 并相对独立地运行。值得关注的是, 进入解码阶段的指令流可能呈现一定程度的乱序性, 这与“rename 是主要乱序边界”的常见直觉存在差异。鉴于 x86 指令为变长编码, 业界常通过“解码簇”类技术, 将复杂指令译码为更接近 RISC 风格的微操作 (micro-operations), 以缓解 x86 译码在带宽与时序方面的固有挑战。两条路径在进入统一的分派与重命名逻辑之前仍须恢复单线程程序顺序, 因此这种并行基本块处理不应解释为体系结构意义上的乱序译码。AMD 的具体合流、仲裁和线程分配规则并未完全公开。[47], [50]

16.2.5 Rename, Dispatch, Retirement, and Recovery

Zen 5 每周期最多接收并分派 8 个 macro-op。这里的分派宽度描述操作进入乱序后端时的资源分配能力, 而不是任意周期都能完成 8 条体系结构指令: 每个 macro-op 仍需获得退休状态、物理寄存器、调度器、load/store queue 等相应资源, 其中任一结构接近满载都会向前端形成反压。来自 Op Cache 的瞬时供给可以高于分派宽度, 前端队列负责吸收这种短期速率差异。[50]

16.2.5.1 Register Renaming

寄存器重命名通过把体系结构寄存器映射到更大的物理寄存器集合, 消除写后读 (WAR) 与写后写 (WAW) 伪相关, 使调度器只需等待真正的读后写 (RAW) 数据依赖。Zen 5 的整数物理寄存器文件包含 240 个通用寄存器; 每个线程最多有 40 个物理寄存器用于体系结构状态或微架构临

时状态, 其余条目用于保存尚未退休的投机结果。条件码不存放在通用整数 PRF 中, 而由独立的标志物理寄存器文件维护, 并提供 192 个可用于乱序重命名的条目。[14], [50]

整数域与浮点/向量域采用彼此独立的重命名资源。浮点域可以接收 8 个已分派 macro-op, 但每周期最多重命名 6 个浮点 macro-op; 当浮点重命名或调度器不能立即接收全部工作时, 96-entry non-scheduling queue (NSQ) 暂存溢出的操作。这样的两级入口使地址生成等相关工作仍可继续推进, 同时避免把 8-wide 全局分派宽度直接转化为一个同样宽、同样昂贵的统一浮点重命名器。[47], [50]

重命名阶段还承担部分依赖消除。零值生成 idiom、部分寄存器移动以及前文所述的 MOV+ALU 融合, 可以减少进入调度器和执行端口的辅助操作。它们不会改变软件可见语义, 却能缩短依赖链并减少物理寄存器写回; 因此, 评价 Zen 5 的“8-wide”入口时, 还需区分体系结构指令、macro-op、融合操作和最终执行 micro-op。

16.2.5.2 Retire Control and Precise State

Retire Control Unit (RCU) 记录整数、浮点与访存操作的完成状态, 并在程序顺序上提交最老的已完成工作。单线程模式下, Zen 5 最多跟踪 448 个在途 macro-op; 开启 SMT 后, 每个线程对应 224 项。单个退休队列条目在部分情况下可以表示相邻指令产生的两个 macro-op, 因此“448 项”描述的是顺序跟踪容量, 不能简单等同为 448 个统一调度项。[47], [50]

退休端每周期最多提交 8 个退休队列条目, 而条目压缩可能使对应的 macro-op 数略高于 8。操作只有在其全部 micro-op 完成、异常状态已知且所有更老操作均可提交时, 才有资格更新体系结构状态。若队首指令等待长延迟 load、除法、缓存维护或异常处理, 即使更年轻的操作已经执行完成, 退休仍会因精确状态要求而阻塞。[19], [50]

精确异常要求处理器在任意可见异常点呈现“所有更老指令已完成、所有更年轻指令尚未生效”的状态。物理寄存器映射、store queue 与退休队列因而不能只服务正常执行, 也必须支持错误路径丢弃、异常重启和资源回收。已完成但尚未退休的 load 仍可能需要保留顺序与一致性信息; 已退休 store 则可能继续停留在写入路径中, 等待获得缓存行权限或写入 L1D。

16.2.5.3 Recovery and Backpressure

分支误预测发生后, 前端需要恢复正确取指地址, 中段需要撤销错误路径上的映射和资源分配, 后端则丢弃尚未形成体系结构效果的年轻操作。Zen 5 的误预测代价不仅由分支执行延迟决定, 还包括预测错误检测、重命名状态恢复、前端重定向, 以及正确路径重新到达译码或 Op Cache 供给端的时间。因此, 12–18 周期应被理解为随路径变化的恢复范围, 而不是固定流水级数量。[50]

448 项窗口扩大了跨越缓存等待寻找独立工作的机会, 也放大了错误路径可能占用的状态。若退休端长期停顿, 物理寄存器、调度器、load/store queue 或 128-entry fetch-window tracking FIFO 都可能先于名义窗口耗尽。Zen 5 的有效乱序深度因而由多种资源的最小余量共同决定, 而不是只由一个“ROB 容量”数字决定。

16.2.6 Integer Execution Unit (IEU)

整数执行单元 (IEU) 可划分为三个逻辑部分: 调度器 (scheduler)、执行簇 (execution cluster) 与退休单元 (retirement unit)。整数重命名 (rename) 阶段的宽度为 8。ALU 调度器与 AGU 调度器均可在每周期接收最多 8 条宏操作 (macro-operations); 其中 ALU 调度器每周期最多可发射 (issue) 6 条微操作 (micro-operations), 而 AGU 调度器每周期最多可发射 4 条微操作。公开框图分别给出约

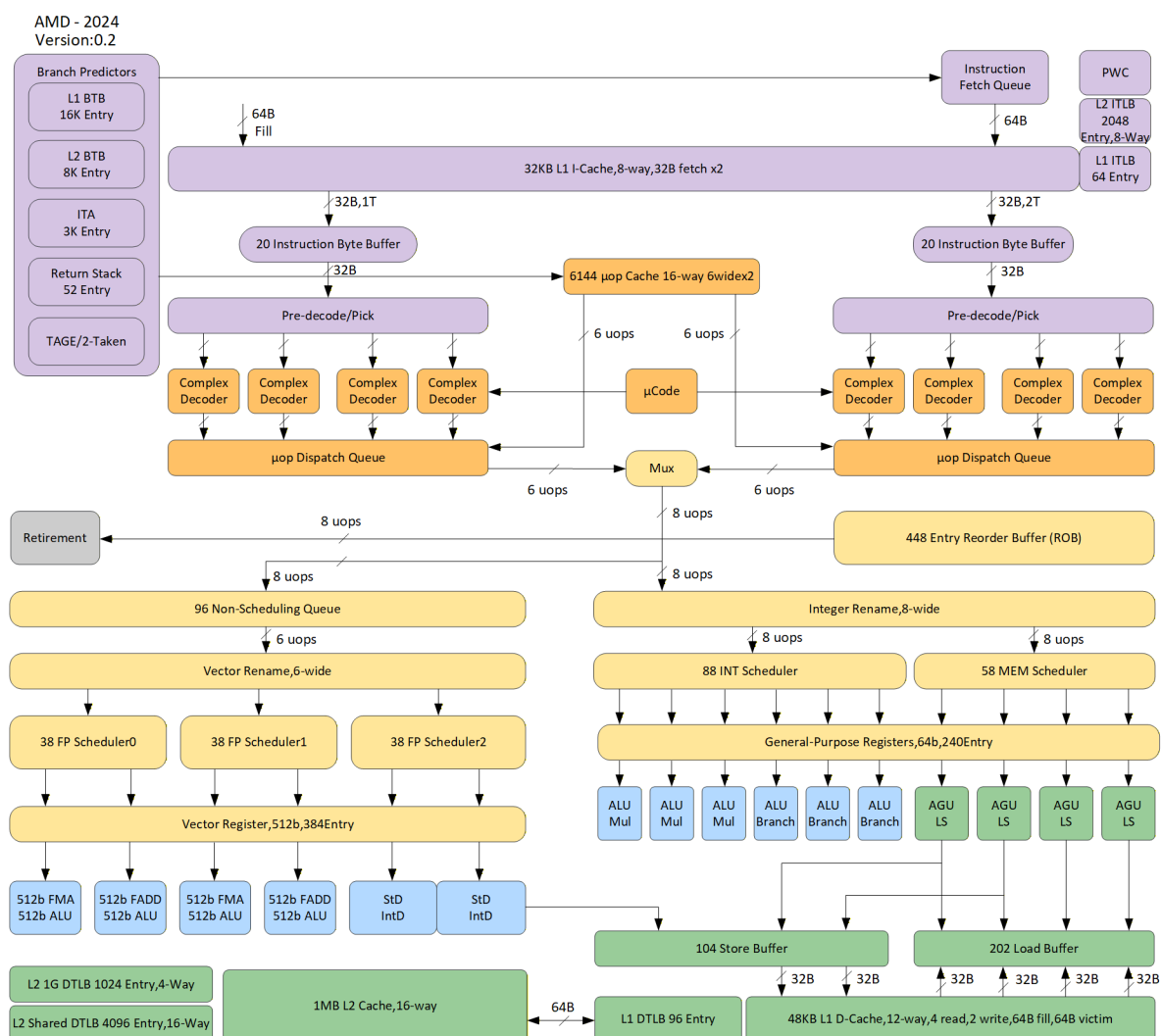


Figure 16.7: Zen 5 的分析性核心框图。图中展开了双取指与译码、Op Cache、整数与向量重命名、分布式调度、执行路径、访存队列和翻译层次；448 项表示整体顺序跟踪窗口，不意味着所有操作进入同一个统一调度阵列。

88 项整数调度状态和 58 项访存调度状态; 这些队列服务不同操作域, 不能相加为一个 146 项统一 scheduler。[47], [50]

16.2.6.1 Scheduling and Operand Readiness

整数 scheduler 维护源操作数是否就绪、操作年龄以及可用执行路径等信息。每个周期的选择逻辑需要从大量候选项中挑出最多 6 个可发射整数操作, 同时避免把只支持乘法、除法、移位或分支的操作送往不兼容端口。访存 scheduler 则独立选择最多 4 个地址操作, 使整数算术与地址生成不必争用同一个全局选择器。

这种分域调度降低了宽选择逻辑的时序和能耗压力, 却引入容量碎片: 整数队列空闲并不能帮助已经塞满的访存队列, 反之亦然。程序中的地址计算、分支与普通算术比例因此会影响实际可见窗口; 即使总体退休队列仍有空间, 某个局部 scheduler 耗尽也会阻止相应类型的新操作继续分派。

Zen 5 提供 6 条通用整数执行流水线。其中, ALU0/ALU1/ALU2 额外支持整数乘法与 CRC 操作; ALU3/ALU4/ALU5 支持 PDEP/PEXT、移位以及分支执行等能力, 且 ALU3 进一步集成整数除法单元。考虑到 Zen 5 可实现每周预测两条分支, 为其中三条流水线配置分支执行能力可视为较为合理的设计点。乘法单元支持最高 64 bits×64 bits 的乘积计算, 并采用全流水实现, 延迟为 3 个周期。除法单元的延迟与有效商位数相关; 较短操作数可以走较低延迟路径, 而更长结果需要额外迭代, 因此不能把最短延迟解释为所有整数除法的固定值。[50]

六条 ALU 路径并非同构, 这意味着峰值“6 operations/cycle”只适用于能够分散到不同兼容端口的指令组合。连续的乘法、PDEP/PEXT、除法或特定移位会集中争用较少路径; 普通加减、逻辑、比较和条件移动则更容易利用较宽后端。端口兼容性、启动间隔和操作延迟应共同用于评价吞吐, 而不能只比较执行单元数量。

所有整数 ALU 路径都能够承担 store-data movement, 但每周进入 store queue 的整数 store data 带宽最多为两条。因而, 地址生成与 store data 是两个不同阶段: 四条 AGU 可以生成更多地址, 但同周期可接受的 store 数仍受数据路径和 store queue 端口限制。[50]

结果旁路决定消费者能否在生产者写回物理寄存器之前获得数据。简单整数操作通常可以通过短旁路链快速唤醒后继, 而乘除、跨整数/向量域移动和访存返回具有不同路径。随着执行宽度增加, 旁路网络的连接数量与线长迅速上升, 因此端口不对称和分域调度也是控制周期时间与功耗的必要手段。

16.2.7 Floating-Point Unit (FPU)

浮点单元原生支持 32 位单精度、64 位双精度与 80 位扩展精度等主要浮点数据类型, 并支持 128/256/512 位打包向量类型, 覆盖整数、单精度、双精度以及 bfloat16 等数据格式。其浮点加载与存储数据通路宽度为 512 位, 并提供对 AVX-512 的支持。[48], [50]

Zen 5 的浮点单元包含 96-entry 的非调度队列(non-scheduling queue, NSQ)。浮点重命名阶段每周最多可接收 6 条宏操作。调度器被划分为 3 个 38-entry 结构, 并连接六条功能不同的执行路径: 其中四条承担主要算术与向量计算, 另外两条侧重 store data 以及浮点/整数域之间的数据移动。每条物理执行管线可以在条件允许时每周接收一个 micro-op, 但单一调度域、特定操作类型或寄存器端口仍可能先于六条总路径达到饱和。退休队列与整数单元共享。[47], [50]

16.2.7.1 NSQ, Schedulers, and Register State

NSQ 不执行常规 wakeup-select, 它的主要作用是缓冲超过浮点重命名或 scheduler 瞬时接收能力的操作。这样, 全局分派仍可在包含大量浮点指令时接近 8-wide, 同时较早的 load/store 地址计算不必完全等待浮点 scheduler 腾出空间。代价是 NSQ 中的操作尚未参与动态选择, 其停留时间仍受到下游重命名、物理寄存器和 scheduler 容量限制。

Zen 5 的浮点/向量物理寄存器集合较 Zen 4 显著扩大, 公开框图给出 384 个 512-bit 条目。扩大 PRF 可以容纳更深的向量数据流, 但也增加读取、写回和旁路成本。三组 scheduler 与局部数据路径把选择和操作数供给分散开, 避免所有向量操作集中访问一个全局阵列; 不同 scheduler 之间不能完全替代, 因而代码的指令组合仍会影响端口平衡。[47]

16.2.7.2 Native 512-bit Execution

高性能 Zen 5 核心的四条主要计算路径采用 512-bit 数据宽度, 可直接执行主要 AVX-512 算术操作。相较 Zen 4 普遍使用两个 256-bit 执行节拍处理 512-bit 状态, 这提高了宽向量指令的峰值吞吐, 并减少部分指令占用执行管线的周期数。512-bit load 仍需成对使用两条 256-bit load 数据路径, 说明执行单元宽度、缓存数据端口与地址生成宽度仍是不同指标。[47], [50]

Zen 5c 保持相同的 AVX-512 体系结构语义, 但可使用两个 256-bit 节拍完成 512-bit 操作, 以换取更高核心密度和能效。软件功能兼容并不意味着不同核心形态具有相同的物理吞吐; 在受向量计算限制的负载中, Zen 5 与 Zen 5c 的执行宽度差异需要与频率、缓存和内存带宽共同考虑。[48], [52]

合并掩码、跨 lane 置换、归约以及整数/浮点域移动可能需要额外读取旧目的值或使用较少的专用路径, 因此不能从“原生 512 bit”推出所有 AVX-512 指令都具有相同吞吐。只有计算操作、load/store 数据路径和寄存器供给同时匹配时, 更宽向量单元才能转化为端到端性能。

16.2.8 Load/Store Unit (LSU)

访存单元(LSU)负责数据访问相关操作。Zen 5 的 L1 D-cache 采用 48 KB、12 路组相联设计, 并划分为 8 个 bank。相较 Zen 4 的 32 KB、8 路 L1 D-cache, 该设计在容量与相联度上均有所提升, 理论上有助于提高命中率, 同时仍维持具有竞争力的访问延迟。[47], [50]

在每个周期内, LSU 最多可支持 4 次 cache 访问操作, 且这些操作可全部为 load。其中每周期最多有 2 次 128/256/512-bit 宽 load; 一条 512-bit load 会配对占用两条 256-bit 数据路径, 因此两条 512-bit load 已经使用全部四条 load pipe。此外, LSU 亦可支持每周期最多 2 次 store; 但对于 512-bit store, 则每周期最多支持 1 次。总体而言, 根据 AMD 公布的数据, 持续带宽相较上一代实现了约 2× 的提升。[47], [50]

16.2.8.1 Address Generation, Banks, and Alignment

四条访存流水线首先承担有效地址生成和 DTLB 查询, 随后才访问 L1D 标签与数据阵列。四条 AGU 并不意味着任意四次访问都能在同一周期完成: 多个地址若映射到冲突 bank、跨越 64 B cache-line 边界、使用复杂地址模式或发生 DTLB miss, 实际吞吐都会下降。L1D 使用地址位选择 bank, 连续且自然对齐的访问更容易分散到不同 bank。[50]

512-bit load 在一个周期内配对使用两条 256-bit load 管线; 512-bit store 则由一个 store-address micro-op 和相应 store-data 路径共同完成。因此, “四次缓存访问”“两次宽 load”和“一次 512-bit

store”描述的是不同约束。未对齐访问若跨越缓存行, 需要两次数据阵列访问, 其吞吐通常降为对齐访问的一半, 并至少增加一个周期。[50]

LSU 可跟踪 64 个未完成的 load, 并且不对已完成 load 的数量施加显式限制。其支持动态依赖关系跟踪: 允许更年轻的 load 绕过更老的 load, 亦允许其绕过不冲突的更老 store, 同时确保体系结构规定的 load/store 顺序不被破坏。此外, LSU 还可跟踪 124 个未完成的 L1 miss。[50]

16.2.8.2 Load and Store Queue State

公开框图给出 202 项 load-buffer 状态, 而优化手册另称最多跟踪 64 个“尚未完成”的 load; 两者描述的不是同一个集合。load 在数据返回后仍可能保留退休、顺序或一致性状态, 因此总 load queue 容量可以明显大于当前未完成访问数。类似地, 124 个在途 cache miss 描述未命中处理能力, 也不能与 load queue 总项数直接互换。[47], [50]

Store Queue 具有 104 项, 用于保存从分派直到数据能够写入缓存的 store。Store 通常需要分别等待地址、数据、退休和缓存行权限; 即使它已经退休, 也可能继续占用写入路径。较大的 store queue 可以吸收短期写入拥塞, 但屏障、原子操作、不可缓存访问或长期 ownership 等待仍会让队列耗尽并向前传播反压。[50]

16.2.8.3 Memory Dependence and Forwarding

年轻 load 可以在预测不冲突时越过更老 load 和地址尚未完全确定的更老 store, 以提高 memory-level parallelism。若更老 store 随后解析到相同地址, 处理器必须撤销错误读取并重放 load 及其依赖者。过度保守会损失并行性, 过度激进则增加 replay; memory dependence prediction 的目标是在二者之间选择。[17], [50]

当更老 store 已经产生完整数据, 且其覆盖 load 所需的全部字节时, LSU 可以执行 store-to-load forwarding, 避免等待数据写入并重新读取 L1D。部分重叠、跨边界、地址不匹配或数据尚未就绪都会使转发路径复杂化, 并可能退化为等待或重放。因而, store queue 不只是写缓冲, 也是低延迟内存通信结构。[18], [50]

关于 load-to-use 延迟, 在简单地址生成 (simple address generation) 条件下: L1 命中时, 整数 load-to-use 延迟为 4 个周期, 浮点为 7 个周期; L2 命中时, 整数延迟为 14 个周期, 浮点为 17 个周期; L3 命中时, 整数延迟为 46 个周期, 浮点为 53 个周期。对于复杂地址生成模式, 相较简单地址生成情形, 延迟将额外增加 1 个周期。[50]

16.2.8.4 Hardware Data Prefetching

Zen 5 同时在 L1D 和 L2 层训练多个数据预取器。L1 Stream 面向连续访问, L1 Stride 按单条指令观察固定地址差, L1 Region 学习一个空间区域内反复出现的访问位置; L2 Stream 与 L2 Up/Down 则在较远层级提前请求顺序或相邻缓存行。不同预取器覆盖的模式并不相同, 组合使用可以减少单一算法的盲区。[50]

预取是否有效取决于准确率、及时性和系统带宽。请求过晚无法隐藏延迟, 请求过早可能在使用前被替换, 错误请求则占用 cache capacity、miss handling state 和互连带宽。四条 load 路径与更大的乱序窗口提高了数据消费速度, 也要求预取器更早、更准确地准备缓存行; 否则增加的执行资源会转化为更快到达同一个内存瓶颈。

Zen 5 还扩展了软件预取控制, 使数据或指令可以被提示到特定缓存层级。PREFETCHNTA 填入 L2 的缓存行会获得更低的保留优先级, 并在逐出时避免继续插入 L3; 该语义使软件能够减少

流式、一次性数据对共享末级缓存的污染。[50]

16.2.9 Simultaneous Multithreading and Resource Sharing

每个 Zen 5 核心支持两个硬件线程。SMT 的目标是在一个线程因依赖、缓存未命中或端口不匹配而留下空闲资源时, 由另一线程填补这些执行槽; 它提高的是资源利用率和总吞吐, 而不是保证单线程延迟下降。[50], [53]

Zen 5 将双线程资源分为三种管理方式: 竞争共享 (competitively shared)、带水位限制的动态共享 (watermarked) 和静态分区 (statically partitioned)。L1I、ITLB、Op Cache、dispatch interface、L1D、DTLB、L2/L3 与执行流水线主要采用竞争共享; 整数 scheduler、整数 PRF、load queue、浮点 PRF、浮点 scheduler 和 memory request buffer 使用水位限制; operation queue、store queue、write-combining buffer 与 retire queue 则静态分区。[50]

这种混合策略在利用率和隔离之间取舍。竞争共享允许单个活跃线程使用全部资源, 却可能受到另一线程的 cache 或端口干扰; 水位限制保留一定公平性, 同时仍允许部分动态借用; 静态分区防止一个线程耗尽关键顺序状态, 但在另一线程停顿时可能留下不可使用的空闲份额。开启 SMT 后, fetch-window tracking FIFO 从单线程的 128 项变为每线程 64 项, 退休窗口则为每线程 224 项, 这使单线程可见的前瞻深度发生明确变化。[50]

共享方式	代表性资源	主要取舍
竞争共享	L1/L2/L3、TLB、Op Cache、执行管线	提高空闲资源利用率, 但线程间干扰更明显
水位限制	Scheduler、PRF、load queue、memory request buffers	保留最低份额, 同时允许有限动态共享
静态分区	Operation queue、store queue、write-combining buffer、retire queue	提高隔离和前进保证, 但可能产生资源碎片

Table 16.3: Zen 5 双线程模式下的资源管理方式。实际性能取决于两个线程的控制流、执行端口和访存行为是否互补。

16.2.10 L2 Cache (L2C)

Zen 5 为每个核心配置 1 MB 的私有 L2 缓存, 并采用 16 路组相联组织。相较 Zen 4, 其容量保持不变, 但相联度翻倍, 理论上有助于提高 L2 命中率。从历史设计取向看, AMD 的 L2 容量相较部分 ARM 与 Intel 方案往往更为克制, 体现出其长期以来对 L2 访问延迟的重视。从作者观点出发, L2 容量仍然重要: 延迟往往可通过更强的预取与预测在一定程度上被掩蔽, 而 L2 miss 在许多场景下仍具有较高代价, 尤其当服务器工作集持续增长时更为明显。不过, AMD 的缓存层次结构反映了更广泛的微架构权衡; 同时, 3D V-Cache 产品在市场上的成功也表明, 部分“容量焦虑”可在平台层面得到缓解。[47], [50]

L1 与 L2 之间的数据路径宽度为 512 bit, 即 64 B/cycle; 同时, AMD 在公开资料中明确给出了经改进的预取机制, 并提及更强的二维步长识别能力。该带宽描述 L2 与近端缓存层次之间的数据传输边界, 不意味着单条 load 的 load-to-use 延迟可以由带宽直接推导。[47], [50]

16.2.10.1 Inclusivity, Latency, and Miss Handling

Zen 5 的 L2 对核心的 L1 缓存采用包含关系。L1 中存在的缓存行同时占用 L2 的标签或数据覆盖, L2 替换相关缓存行时需要维护上层副本的一致性。包含关系有助于简化探测和核心私有层级的目录判断, 但 1 MB 标称容量并非完全独立于 L1 工作集。[50]

AMD 给出的 L2 load-to-use 延迟不低于 14 个周期, 具体产品与访问条件可能有所变化。较大的乱序窗口可以重叠多个彼此独立的 L2 命中, 却无法缩短单条指针依赖链中的串行 load。因而, L2 的容量、相联度、命中带宽、未命中并行度和访问延迟需要同时评价。

L2 同时接收 L1I、L1D、页表遍历、预取和一致性请求。64 B/cycle 数据路径只有在请求能够及时仲裁、目标 bank 可用且未命中状态未耗尽时才能持续发挥作用。当前资料没有完整公开 L2 miss queue、writeback queue 与内部 bank 数量, 因此不应由接口带宽反推任意访问组合下的持续吞吐。

据报道, Zen 5 将部分 SRAM 宏单元由 8T 迁移至 6T, 从而显著提升了存储密度。这使得核心在增加多项资源配置以提升性能的同时, 仍能在面积上保持相对可控。基于芯片照片的比较也显示, 在缓存容量相当条件下, Zen 5 的 L2/L3 结构占用面积更小; 这一现象与工艺缩放及 SRAM 密度提升均相一致。总体而言, 更高的 SRAM 密度为核心引入更多资源、进而提升 IPC 提供了空间, 同时避免处理器总面积出现过快增长。[47]

16.2.11 L3 Cache (L3C)

Zen 5 的末级缓存延续了 AMD 长期采用的“每个 CPU complex (CCX/CCD) 内共享缓存”设计, 而其优化手册也给出了一些对微架构分析尤其重要的实现细节。AMD 明确指出, 该 L3 为容量最高可达 96 MB、16 路组相联的共享式 write-back 缓存, 由同一 complex 内的多个核心共同访问。对于本文重点关注的服务器产品而言, 每个 CCD 集成 8 个核心并共享 32 MB L3; 因此, 文档中“最高 96 MB”的表述更应理解为产品家族层面的上限, 而非单个 CCD 的固定容量点。相较私有的中间层缓存, L3 的主要作用一方面是扩展容量, 另一方面是承担片上多核之间的数据共享与转运枢纽, 其设计目标因此不仅受平均命中延迟约束, 也受到一致性维护与 complex 内数据搬运效率的共同塑形。[48], [50]

一个值得注意的细节是, Zen 5 的 L3 由 L2 victim line 填充, 这表明它并非“始终包含上层私有缓存副本”的简单 inclusive 结构。换言之, L3 的驻留内容在很大程度上受私有缓存替换行为以及多核共享模式动态变化的影响。AMD 进一步说明: 当发生 L3 hit 时, 若该访问为 store, 或该缓存行此前仅被单个核心读取, 则该行可能会从 L3 中失效; 而对于代码取指命中或被多个核心读取过的缓存行, 则可能继续保留在 L3 中。这一策略说明, Zen 5 的 L3 并非单纯以最大化原始 hit rate 为目标, 而是在有效容量、一致性流量以及“保留潜在共享数据”的价值之间进行折衷。对于面向服务器场景的设计而言, 这种取舍具有相当合理性: 对多核共享数据维持片上可见性, 往往比反复重建其共享状态更具系统层面的收益。[50]

另一个关键机制是, Zen 5 在 complex 内部为各私有 L2 配置了 shadow tags。当某核心在本地 L2 与共享 L3 中均 miss 时, 系统会进一步查询这些 shadow tags, 以判断目标数据是否仍驻留在其他核心的 L2 中; 若答案为是, 则可在 complex 内直接发起 cache-to-cache transfer。这一机制在体系结构上具有重要意义, 因为它避免了将所有本地 miss 一概视为“必须继续向更深层级访问”的统一事件。更准确地说, 该层次结构实际上区分了至少三类情况: 本地私有命中、共享缓存命中以及对等核心私有缓存命中。对于局限于单个 CCD 内部的 producer-consumer 型共享模式而言, 这种区分可显著降低有效访问延迟与片上互连压力, 即便 L3 本身的名义命中延迟并不算极端激进。[50]

在延迟方面, AMD 给出的 L3 hit 平均 load-to-use 延迟约为 46 个周期。与某些更强调超低共享缓存延迟、或配置更大私有中层缓存的设计相比, 这一数值并不算特别激进。然而, 不能孤立地解读该延迟指标。若将 L3 与 Zen 5 更宽的前端资源、更深的队列、更强的预取能力以及改进后的 load/store 带宽一并考察, 可以得到更一致的图景: AMD 似乎愿意接受一个并不算低的共享缓存延迟, 只要整个层次结构在总吞吐、共享效率与 miss containment 方面仍能保持较强竞争力。这与 AMD 多代以来的设计哲学大体一致, 即更强调端到端吞吐与容量利用效率, 而非在每一级缓存上都一味追求最低名义延迟。[50]

PREFETCHNTA 的处理方式也值得关注。根据 AMD 的描述, 带有该 hint 填入 L2 的缓存行会被标记为更快逐出; 并且当其自 L2 被逐出时, 不会继续插入 L3。这说明 Zen 5 并未将 L3 仅仅当作被动的“溢出空间”, 而是将其视为需要进行选择性保留的共享容量资源。对于流式访问或一次性使用的数据, 将其排除在共享 L3 之外, 有助于降低污染, 并为更具时间局部性或跨核心复用价值的数据与代码保留空间。从工程实现角度看, 这意味着 Zen 5 上的软件可见预取 hint 仍可能对缓存层次结构产生实质性影响, 尤其是在服务器与吞吐导向的工作负载中, LLC 污染的机会成本往往较高。[50]

16.2.12 SoC and Platform Features

前文主要在 core level 上讨论 Zen 5; 但对于服务器级实现而言, 这一视角并不充分。EPYC 9005 的实际行为并不只由核心微架构决定, 还取决于 compute die、I/O logic、内存接口、封装外互连以及安全机制如何被整合为单一 SoC。因此, Zen 5 的体系结构意义有相当一部分体现于平台组织方式, 而不仅仅体现于流水线宽度本身。[48]

从这一角度看, Turin 可被理解为 AMD 基于 chiplet 的服务器路线的进一步发展。封装内部的职责划分更为明确: 计算侧 chiplet 主要针对执行密度与核心性能进行优化, 而 I/O die 则承担内存挂接、外部通信以及系统级协调。CPU die 采用更先进工艺、I/O die 采用相对保守工艺的配置, 与这种分工是相一致的。这种安排体现了一条看似常规但十分关键的工程原则: 高密度逻辑与以 PHY 为主的电路遵循不同的缩放约束, 因此未必需要服从同一制程目标。[48]

内存子系统同样应被视为体系结构组成部分, 而非仅作为背景规格存在。12 通道 DDR5-6400、约 614 GB/s 的理论单插槽带宽以及 TB 级容量支持, 会实质性地改变核心的运行边界。当 Zen 5 同时扩展前端供给、执行宽度与核心内部数据搬运能力之后, 性能上限将越来越取决于私有缓存层次之外的有效数据能否持续供给。在这一意义上, 主存系统并不只是配套基础设施, 而是将更宽核心资源转化为持续服务器吞吐的必要条件之一。[48]

类似的观察也适用于 I/O complex。对 EPYC 9005 而言, 平台能力并不只取决于原始计算吞吐, 还取决于高速链路在 PCIe、socket 间通信、存储与 CXL 相关功能之间的可分配程度。这种灵活性具有明确的体系结构意义, 因为服务器处理器的效率约束并不只来自核内结构, 也来自主板拓扑、加速器挂接方式以及多 socket 通信模式。因此, 采用统一 SERDES 资源支撑多类接口, 本质上是一种面向封装边界可配置性的系统级优化。[48]

在多 socket 部署中, 这一点的重要性会被进一步放大。尽管远端访问在构造上仍然保持非一致性, socket 间带宽仍会显著影响这种非一致性在实践中的强弱程度。公开披露的三链路与四链路配置说明, socket-to-socket 传输被作为一等设计资源进行配置, 而非仅被视为一致性协议之外的剩余通路。因此, 对 Zen 5 服务器中 NUMA 行为的理解, 应当同时结合延迟、放置策略与可用 fabric 带宽, 而不能仅以局部访存代价加以概括。[48]

另一个平台层面的含义体现在 device-originated data path 的处理方式上。Smart Data Cache Injection 值得关注之处, 并不只在于其作为功能点的存在, 更在于它表明平台试图减少 I/O 侧

producer 与核心侧 consumer 之间原本可以避免的数据搬运。对于快速消费报文、消息或加速器生成缓冲区的工作负载而言, 这种设计带来的体系结构收益并不限于狭义的延迟下降, 也可能表现为更少的无效主存流量以及更低的整体层次结构压力。[48]

CXL 支持则将同一问题进一步扩展到异构内存组织之中。当处理器开始参与内存扩展、资源池化与分层内存的软件策略时, 体系结构问题便不再局限于本地执行效率, 还涉及处理器能否在更大的 memory fabric 中充当高效节点。这会显著提高 cache policy、地址翻译、fabric 带宽与软件放置之间相互作用的重要性。因此, Zen 5 的平台含义已经超出了传统意义上的本地 DRAM 访问优化。[48]

安全性也应以同样的整合视角来理解。专用安全处理器, 以及围绕其构建的验证启动、加密、证明与虚拟化隔离机制, 说明可信建立过程已被纳入处理器常规部署模型之中。对当代数据中心系统而言, 这一点具有明确的体系结构相关性, 因为可真正利用的性能越来越受租户隔离、设备可信性以及虚拟化环境下受保护执行状态维持能力的制约。[48]

总体而言, 从 SoC 层面解读 Zen 5, 有助于说明纯粹的 core-centric 分析为何存在边界。在 EPYC 9005 中, 体系结构竞争力并不只取决于更宽、更深的 CPU core, 也取决于 compute、memory、I/O、fabric 与 trust services 在服务器处理器封装中的协同设计。因此, 平台级特征更适合被视为该架构性能叙事的主体组成部分, 而非次级实现细节。

16.2.13 Summary

项目	Zen 4		Zen 5
L1/L2 BTB	1.5K/7K		16K/8K
Return Address Stack	32		52
ITLB L1/L2	64/512		64/2048
Fetch/Decoded instruction bytes per cycle	32		64
Op cache associativity	12-way		16-way
Op cache bandwidth	9 macro-ops	12 instructions (or fused instructions)	
Dispatch bandwidth (macro-ops/cycle)	6		8
ALU/AGU	4/3		6/4
Integer PRF (reg/flag)	224/126		240/192
Vector registers	192		384
FP pre-scheduling queue	64		96
FP scheduler	2×32		3×38
主要 FP/vector 计算路径	3		4
Vector width	256 b		256 b/512 b
顺序跟踪/retire window	320		448
LS memory pipelines (load/store)	3/1		4/2
DTLB L1/L2	72/3072		96/4096
L1 data cache	32 KB/8-way		48 KB/12-way
L2 per core	1 MB/8-way		1 MB/16-way
L2 bandwidth	32 B/clock		64 B/clock
Load/store queue	136/64		202/104

Table 16.4: Zen 4 与 Zen 5 的主要结构参数对比。表中各项分别描述不同流水阶段, 不能直接相加为统一 IPC 上限。

从 AMD 自身代际对比来看, Zen 5 相较 Zen 4 在多项微架构参数上呈现较为激进的扩展: 例如, TLB 容量的提升幅度可类比为部分 ARM 设计跨多个代际的累积改进; 若干带宽相关资源在 AMD 优化手册中已明确体现为约 2× 的扩展, 而 L1 BTB 规模则在公开分析中常被描述为达到约

10× 的提升。这一设计方向与 AMD 长期以来“通过更大资源配置提升吞吐”的偏好相一致。不过, 实证测试亦表明 Zen 5 在某些特定场景下可能出现相较 Zen 4 的性能回退。总体而言, 若后续代际能够在继承这些设计思路的同时, 进一步修正现阶段暴露的前端瓶颈与实现约束, Zen 系列仍具有较强的演进潜力。从 Zen 5 当前暴露的实现约束来看, 若后续代际继续沿用双解码流水线思路, 则线程间资源共享效率仍将是值得重点关注的演进方向。[47], [48], [50]

Chapter 17

Intel Microarchitectures

17.1 Skymont

17.1.1 Overview

Skymont 是 Intel 面向能效核心 (E-core) 设计的乱序 x86 微架构, 用于 Lunar Lake 与 Arrow Lake 客户端处理器。它延续 Tremont 以来的分簇译码路线, 但不再把 E-core 局限在狭窄前端和小型后端: 三个译码簇、扩大的在途窗口、更多执行入口和更强的缓存层次, 使其单线程能力接近早期性能核心, 同时保持单核单线程和较小面积的设计目标。[54]

Intel 的代际评估称, Lunar Lake 中的 Skymont 相较低功耗 Crestmont, 在固定频率条件下可获得约 1.38 倍整数 IPC 和 1.68 倍浮点 IPC。该提升不是由单一结构产生, 而是来自控制流供给、译码并行度、调度容量、执行覆盖和数据带宽的同步扩张。[55]

结构	微架构特征
前端译码	分级分支预测; 64 KB、8 路 L1I; 不设置独立 micro-op cache
乱序执行	三个 3-wide x86 译码簇, 峰值合计 9 条指令/cycle
执行后端	扩大的重命名、调度与在途窗口; 单核单线程
缓存组织	26 个功能分化的执行入口, 覆盖整数、分支、访存、浮点和向量操作
	每核独立 L1, 客户端四核簇共享 4 MB L2

Table 17.1: Skymont 的主要组织特征。译码簇、dispatch、执行入口和退休分别处于不同阶段, 不能把峰值宽度直接相加。

17.1.2 Design Positioning: Scaling by Clustering

变长 x86 指令给超宽直接译码带来边界识别、跨块拼接、微码分流和布线压力。Skymont 没有采用单个 9-wide 译码阵列, 而是把工作拆给三个较窄簇。每个簇只需在较局部的数据通路内完成长度识别和语义转换, 前端再把各簇输出重组为程序顺序。这样做以额外的 steering 和合并逻辑, 换取更容易扩展的译码吞吐。

相同思路也延伸到执行后端。大量端口并非构成一个完全对称的统一执行池, 而是把常用能力分布到多个较小域中, 使整数、分支、地址生成、store data 和向量操作减少彼此争用。Skymont 的面积效率来自局部结构的复制和专门化, 而不是让每个入口都支持所有操作。

这种设计的风险是资源碎片化。若指令流无法均匀使用三个译码簇, 或某一类操作集中占用少数执行路径, 其余资源即使空闲也不能自动消除瓶颈。因此, Skymont 的峰值宽度只有在控制

流、指令组合和端口映射较均衡时才能转化为持续吞吐。

17.1.3 Front End

17.1.3.1 Branch Prediction and Instruction Supply

分支预测器需要同时解决方向、目标和供给时序。较小的低延迟结构优先覆盖循环和高频局部分支，容量更大的后级结构负责较大的代码 footprint；返回地址栈和间接目标预测器则分别处理 call/return 与多目标控制流。宽译码器并不只要求预测准确，还要求目标足够早地生成，否则三个译码簇会同时缺少输入。

Skymont 配置 64 KB、8 路组相联的 L1I。较大指令缓存降低大型应用在前端反复进入共享 L2 的频率，但组相联度和容量不能替代目标预测：taken branch 仍可能丢弃当前取指块尾部字节，跨 cache line、跨页和 ITLB miss 也会降低有效字节利用率。[54]

Skymont 不依赖独立 micro-op cache。热代码仍可从 L1I 相关结构中的预解码信息获益，避免每次重新发现全部指令边界；真正的语义译码则由三个 cluster 并行完成。这种选择节省了一套保存内部操作的容量阵列，但让 L1I、预解码元数据和译码簇长期处于更核心的供给路径。

17.1.3.2 Three Decode Clusters

三个 3-wide cluster 的峰值合计为每周期 9 条指令。前端首先把预测路径切分为可交给不同簇的片段，各簇完成边界识别、简单融合、复杂指令检测和内部操作生成，随后由合并队列恢复全局程序顺序。某一簇遇到微码指令或不利边界时，其它簇可以继续产生操作，但合并端仍必须保证年轻片段不能越过较老片段。[54]

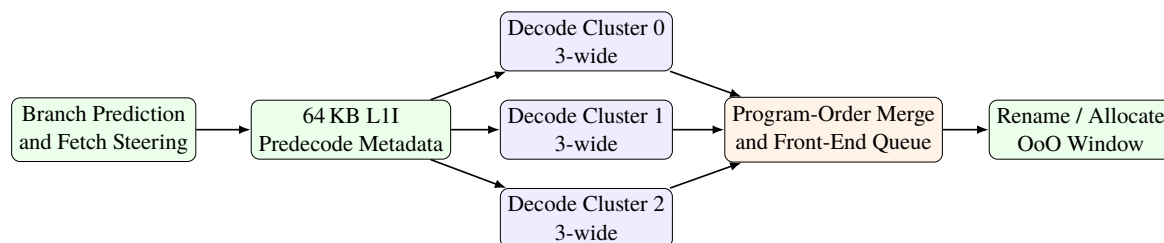


Figure 17.1: Skymont 分簇前端的逻辑数据流。三个译码簇扩大峰值吞吐，合并结构负责恢复程序顺序并吸收各簇短期不均衡。

分簇前端最重要的性能变量不是三个宽度的简单总和，而是 cluster utilization。长直线代码、短基本块、频繁跳转和复杂指令会形成不同的片段分布；若一个周期只有一个簇获得可译码片段，另外两个簇的存在不会提高吞吐。steering 策略需要在尽量填满簇与避免过度切分之间折中。

17.1.4 Rename, In-Flight State, and Recovery

来自三个译码簇的操作在进入后端前汇合为单一程序顺序，并统一分配 ROB、物理寄存器、调度项与 load/store queue。dispatch 宽度低于译码峰值并不矛盾：前端队列可以把 9-wide 译码产生的突发输出平滑成后端可持续接收的流量，译码簇也不必每周期同时满载。

扩大的在途窗口让 Skymont 能够越过 branch resolution、乘除法和缓存未命中寻找独立工作。ROB 只保存程序顺序与完成状态，真正可参与选择的操作分布在多个调度域；物理寄存器、scheduler 或 LSQ 中任一资源先耗尽，都会使名义窗口无法继续增长。

分支误预测恢复必须同时撤销三个前端簇产生的年轻操作,并恢复重命名映射、free list 和预测历史。更宽前端意味着同样的恢复周期会丢弃更多工作,因此预测准确率和检查点恢复效率对 Skymont 的收益尤为重要。较宽退休端则在长延迟阻塞解除后快速释放连续完成的在途状态,但不能长期突破 dispatch 对新工作的供给上限。

17.1.5 Distributed Execution Back End

Skymont 的 26 个执行入口体现的是功能覆盖和调度连接数量,而不是每周期能够完成 26 条任意指令。某些入口只接受简单整数或分支,某些用于地址生成和 store data,浮点、向量、乘除和加密操作也具有各自兼容集合。指令延迟与吞吐数据需要按具体指令读取,因为相同 ISA 类别中的不同操作仍可能使用不同端口和启动间隔。[56], [57]

17.1.5.1 Scalar and Branch Paths

整数后端需要同时承担地址形成、循环计数、比较、位操作和一般算术。把简单 ALU 能力复制更多路径,可以减少这些常见操作之间的结构冲突;分支执行资源则负责尽早验证前端预测。分支解析越晚,错误路径在扩大的窗口中占据的 ROB、寄存器和缓存请求越多。

端口专门化降低了全局选择器和旁路网络的复杂度,却使代码生成质量更加重要。若编译器反复选择只能进入少数复杂路径的指令形式,核心可能在总端口利用率不高时仍然受局部冲突限制。等价的移位、LEA、乘加或条件选择序列,可能因端口覆盖不同而表现出不同吞吐。

17.1.5.2 Floating-Point and Vector Paths

Skymont 支持现代客户端所需的标量浮点、Advanced Vector Extensions 与 AVX2 指令。体系结构指令宽度不能直接代表物理数据通路宽度;较宽操作可以占用多个内部片段或多个周期,置换、转换、乘法和 FMA 也不一定共享相同吞吐。分析向量性能时,应同时观察执行端口、load/store 带宽和频率限制,而不是只计算理论 lane 数。

浮点 IPC 提升较高,说明前代限制不只来自算术单元数量,也来自前端供给、调度窗口和数据路径。增加向量入口若没有更强的寄存器读取与缓存供给,只会把瓶颈转移到操作数侧;Skymont 的后端扩张必须与访存和共享 L2 一起理解。

17.1.6 Load/Store Pipeline and Translation

访存操作依次经过地址生成、DTLB 查询、load/store queue 顺序检查、L1D bank 访问和结果转发。地址生成入口数量、缓存读写端口和结果总线位于不同阶段,不能用单一“访存端口数”概括。跨行访问、未对齐操作、store forwarding 失败与 bank conflict 都可能降低完成率。

年轻 load 可以在预测为独立时绕过地址尚未解析的旧 store;若后续发现别名,则必须 replay load 及其依赖操作。更大的乱序窗口会提高并行 load 数量,也放大 memory dependence predictor、DTLB 和未命中队列的压力。对指针链而言,额外通道无法突破真实地址依赖;对数组和多流访问而言,更多在途 miss 才可能转化为更高 memory-level parallelism。

ITLB、DTLB 和 page walk 共享更低层缓存与内存带宽。大型客户端程序的代码、堆和共享库会同时扩大翻译 footprint,因此 L2 的作用不仅是保存普通数据,也包括承接指令取值、页表项和预取流量。翻译缺失与数据缺失在外部层级相互竞争,是四核簇满载时的重要压力来源。

17.1.7 Shared L2 and Cluster Behavior

客户端实现中,四个 Skymont 核心共享 4 MB L2。每个核心保留独立前端、L1 与乱序后端,而共享 L2 负责扩大总容量、减少同簇共享数据的重复副本,并汇聚向更远层级的请求。[54]

共享容量提高利用率,却使核心之间形成新的竞争关系。一个线程的 streaming miss 或低准确率预取可能占用 L2 bank、未命中队列和外部带宽,增加其它三个核心的命中延迟;热点共享行和原子操作还会引入权限迁移与仲裁。Skymont 单核的访存能力因此不能脱离四核簇的并发行为评价。

单核单线程避免了同一核心内部的 SMT 资源竞争,使 ROB、调度器和 L1 带宽均由一个线程使用。吞吐扩展主要依赖增加物理核心,而不是让两个线程共享同一后端;代价是单线程发生长延迟停顿时,核心内部没有第二线程填补空闲槽位。

17.1.8 Summary

Skymont 的设计主线是以分簇方式扩大 x86 核心,而不是复制一个小型 P-core。三个译码簇提高冷代码和大代码 footprint 下的操作供给,扩大的在途状态保存更多独立工作,功能分化的执行入口降低常见端口冲突,共享 L2 则为四核并发提供容量与数据复用。

这种组织的性能取决于多个平衡点:预测器能否持续生成可分配给多个簇的路径,合并与 dispatch 能否吸收译码突发,指令组合能否均匀使用专用端口,LSU 和共享 L2 又能否承受四核产生的请求。Skymont 的价值不在于某个峰值数字,而在于它以较小核心面积建立了完整的宽乱序数据流。

17.2 Lion Cove

17.2.1 Overview

Lion Cove 是 Intel 于 2024 年推出的性能核心(P-core)微架构,用于 Lunar Lake 与 Arrow Lake 客户端平台。其设计目标不是单纯扩大最高频率,而是在取消 SMT 后把面积、功耗和调度资源集中到单线程路径,并同步扩展分支预测、指令供给、乱序窗口、执行端口与数据缓存层次。

Intel 的固定频率综合评估称,Lion Cove 相较 Redwood Cove 获得约 14% 的 IPC 提升。该增长来自多个局部改进累积:8-wide 主译码降低操作缓存未命中时的前端瓶颈,整数与向量调度分离以扩展后端,更多执行入口提高常见操作并行度,48 KB 近端数据缓存与 192 KB 后备数据缓存则在低延迟和容量之间增加一级折中。[55], [58]

结构	主要特征
线程组织	单核单线程,不启用 SMT
前端	64 KB L1I; 8-wide 主译码;micro-op cache 热代码路径
乱序后端	扩大的在途窗口;整数与向量调度域分离
执行	更多整数、分支、浮点、向量和访存入口,端口功能不完全对称
数据缓存	48 KB 低延迟近端缓存、192 KB 后备数据缓存与可配置私有 L2

Table 17.2: Lion Cove 的核心组织。缓存命名在产品材料中可能采用 L0D/L1D/L2 口径,本章按“近端—后备—私有 L2”描述访问距离。

17.2.2 Design Positioning: Reinvesting the SMT Budget

传统 SMT 通过让两个线程共享前端、窗口和执行端口, 提高一个线程停顿时的资源利用率; 代价是每个结构都要处理线程标识、配额、公平性和跨线程冲突。Lion Cove 取消 SMT 后, 只需维持一个体系结构上下文, 单线程可以独占 ROB、物理寄存器、scheduler、LSQ 和一级缓存带宽。

取消 SMT 不会自动提高 IPC。若核心只是删除第二套架构状态, 面积节省可能无法直接缩短关键路径。Lion Cove 把这部分预算重新投入更宽译码、更深窗口、更多端口、细粒度时钟控制和新的数据缓存层, 使“单线程独占”与“单线程可消费更多资源”同时成立。

这种选择适合由 P-core 与 E-core 共同承担线程吞吐的客户端平台。操作系统可以把延迟敏感线程放到 Lion Cove, 把高并发后台工作交给 Skymont; 核心内部不再依赖 sibling thread 填补空洞, 而由更积极的预测、预取和乱序执行维持利用率。

17.2.3 Front End

17.2.3.1 Branch Prediction as Instruction-Supply Control

Lion Cove 提高分支预测带宽和前瞻距离, 使预测路径能够先于实际译码准备多个基本块。方向准确率决定错误路径频率, 目标预测容量决定大型代码工作集中的覆盖率, 而预测带宽决定短基本块能否持续填满 8-wide 译码器。三者缺一不可。[58]

分级 BTB 把低延迟与大容量目标查找分开: 近端结构在关键路径上快速给出常见目标, 后级结构覆盖较大的函数和共享库 footprint。间接分支、返回与普通直接分支具有不同历史规律, 因而通常由专门结构共同参与。Lion Cove 的前端改进应理解为一层控制流流水线, 而不是单个预测表容量增长。

错误预测的成本随窗口和供给宽度增加而上升。前端在一个未解析分支之后可以快速产生大量操作, 若恢复只依赖慢速逐项回滚, 宽度收益会被频繁清空抵消。因此, 预测器、重命名检查点、前端重定向和正确路径重新到达译码器的时间需要作为一个整体评价。

17.2.3.2 Instruction Cache, Decode, and Micro-Op Cache

Lion Cove 配置 64 KB L1I。新指令流经取指、预解码、对齐和 8-wide 主译码进入后端; 重复执行的热点路径可以由 micro-op cache 提供已经转换的内部操作, 绕过主译码器并降低能耗。两条路径最终都必须形成统一的程序顺序流。[58]

8-wide 主译码的重要性在于降低 micro-op cache 未命中时的落差。大型浏览器、运行时和游戏代码会频繁切换函数, 热点操作缓存不可能覆盖全部 footprint; 若常规译码明显更窄, 命中率波动会直接形成吞吐波动。更宽主译码让冷代码与热代码路径之间的供给差异缩小。

x86 指令仍可能被融合、拆分或交给微码序列。8 条体系结构指令并不必然产生 8 个内部操作, micro-op cache 的供给单位也不必与架构指令一一对应。比较前端宽度时, 必须确认计数单位和融合条件, 而不能把 decode、micro-op cache 与退休宽度当作同一指标。

17.2.4 Rename, Window, and Recovery

重命名阶段消除架构寄存器的写后写与读后写伪依赖, 并为每个操作分配程序顺序、物理寄存器、调度与访存资源。Lion Cove 扩大在途窗口后, 可以跨越更长的 cache miss 或执行延迟寻找独立工作; 但 ROB、scheduler、物理寄存器和 LSQ 的容量并不相同, 任何一项先耗尽都会阻止后续分配。

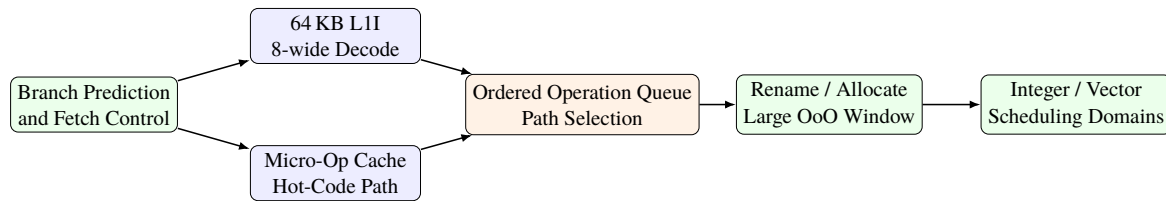


Figure 17.2: Lion Cove 的双路径指令供给。主译码处理新代码, micro-op cache 服务热点路径; 两者在重命名前恢复为统一顺序。

较大窗口增加恢复压力。分支误预测需要恢复寄存器映射和预测历史, 内存顺序违例可能触发选择性 replay 或更大范围 machine clear, 异常则必须在退休边界形成精确状态。检查点减少映射回滚距离, 选择性 replay 减少无关操作重做, 但都需要额外状态追踪。

取消 SMT 简化了恢复域。核心无需同时维护两个线程的投机映射, 也不会因一个线程的恢复直接占用另一个线程的前端带宽; 与此同时, 单线程错误会清空整个核心的有效工作, 没有 sibling thread 在恢复期间继续提供吞吐。因此 Lion Cove 更依赖低误预测率和快速重新填充。

17.2.5 Scheduling and Execution

17.2.5.1 Separated Integer and Vector Scheduling

Lion Cove 将整数与向量调度进一步分离。两类操作具有不同的源操作数宽度、执行延迟、旁路范围和端口兼容性, 若全部放入统一阵列, wakeup-select 与寄存器读取网络会随窗口和宽度迅速膨胀。独立调度域可以缩短局部连线, 并分别调整队列容量与选择策略。[58]

分离也会造成容量不可互换。纯整数代码可能填满整数 scheduler, 而向量队列仍有空位; 反之亦然。设计需要通过前端分配规则、队列大小和端口覆盖, 使常见工作负载不因静态边界过早阻塞。因而, “总调度项数”不如各域压力和指令组合更能解释性能。

17.2.5.2 Execution Ports and Operand Supply

新增执行入口降低简单 ALU、分支、地址生成和浮点/向量操作之间的结构冲突, 但端口并非完全同构。乘除、移位、条件选择、向量置换和转换通常只在部分路径执行; 即使总端口空闲, 某类专用操作仍可能达到局部吞吐上限。[57], [58]

更宽后端还要求足够的寄存器读取、旁路和写回能力。多条操作同时发射会产生大量源操作数请求, 长距离广播则增加功耗和周期时间。分布式 scheduler、局部旁路与时钟门控把全局网络拆成较小区域, 使执行宽度可以扩展而不必让所有结果在一个周期内广播到所有端口。

Lion Cove 采用更细粒度的时钟控制, 使暂时不用的局部结构减少动态功耗。细粒度 gating 必须足够快, 否则结构重新唤醒会在突发代码阶段产生气泡; 它与无 SMT 设计相互配合, 因为单线程工作负载更容易出现某些执行域长期空闲的阶段。

17.2.6 Load/Store Unit and Data Cache Ladder

Lion Cove 的数据侧不是简单的“L1 加大”。48 KB 近端数据缓存维持低延迟命中路径, 192 KB 后备数据缓存承接更大的工作集, 再由私有 L2 提供更远容量。该阶梯把传统两级私有数据层次拆成更多命中点, 使频繁访问数据保持较短路径, 同时降低中等 footprint 直接进入大 L2 的概率。[58]

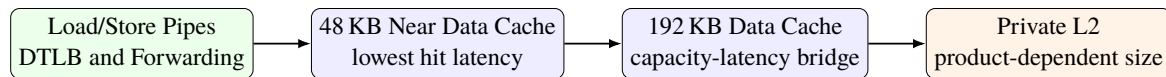


Figure 17.3: Lion Cove 的私有数据缓存阶梯。更近层级优化命中延迟, 后备层级减少访问大容量 L2 的频率; 实际请求可能采用并行标签查询和提前转发。

增加一级缓存会引入新的标签、替换和包含关系管理。核心需要决定预取数据放入哪个层级, 近端替换是否保留到后备缓存, 以及一次 miss 是串行查询还是并行发往下一层。若所有请求都严格逐级等待, 额外层级可能增加未命中延迟; 实际设计通常通过预测、并行标签查找或早期请求降低该代价。

LSU 仍需处理 memory dependence prediction、store-to-load forwarding 和顺序验证。年轻 load 可以越过地址未知的旧 store, 但别名出现时必须 replay; 三层私有缓存只改变数据从何处返回, 不改变精确异常和内存模型要求。缓存带宽、DTLB 端口、load/store queue 和未命中资源必须同步扩展, 才能支撑新增执行入口。

17.2.7 Memory Management Unit

Lion Cove 采用分离 L1 ITLB/DTLB 和共享后级翻译缓存。更宽前端会提高指令翻译请求率, 更多 load 管线会提高数据翻译并发度; 大型代码、堆、共享库和虚拟机还会扩大 translation footprint。TLB 容量、查找带宽与 page-walk 并发度因此同时影响核心供给。

页表遍历会与普通指令和数据访问争用缓存及内存带宽。后级 TLB 命中虽然比 L1 TLB 慢, 却远低于完整多级 page walk; 大页扩大 TLB reach, 但可能增加内存碎片和部署限制。评价 MMU 时应同时观察页面大小分布、嵌套翻译和 page-walk cache, 而不是把不同 TLB 项数简单相加。

17.2.8 Retirement, Power, and Platform Context

退休端把乱序完成的操作重新整理成精确程序历史。宽退休可以在 ROB 头部长延迟操作完成后快速释放连续已完成项, 但无法越过最老未完成指令。更大的窗口提高了隐藏延迟的机会, 也可能在 miss 过长时积累更多无法退休的年轻工作。

Lunar Lake 与 Arrow Lake 使用相同核心家族, 但缓存容量、频率、电源管理和 SoC 互连并不完全相同。Lunar Lake 强调低功耗移动平台, Arrow Lake 覆盖更高功率桌面和移动配置; 同一 Lion Cove 在不同产品中的 L2、频率和功耗曲线不能被视为核心固定常数。

混合架构还要求操作系统理解 P-core 与 E-core 的能力差异。Lion Cove 的无 SMT 单线程资源适合延迟敏感任务, Skymont 簇适合并发吞吐和较低功耗; Thread Director 与调度器的决策会影响核心微架构能力是否被正确使用。[54]

17.2.9 Summary

Lion Cove 的设计可以概括为: 取消 SMT 后把资源集中到一条更宽、更深的单线程数据流。分支预测与双路径前端准备操作, 扩大窗口和分离调度暴露并行性, 更多执行入口消费工作, 48 KB、192 KB 与私有 L2 组成的数据缓存阶梯则降低操作数等待。

其关键限制也来自这些结构的耦合。micro-op cache 未命中会把压力交给 8-wide 主译码, 整数或向量队列可能因分离而局部饱和, 专用端口可能在总资源空闲时形成冲突, 新增缓存层级也必须通过预测和并行访问控制延迟。Lion Cove 的 IPC 提升不是某个宽度的结果, 而是单线程资源重新分配后的整体平衡。

Chapter 18

Arm Microarchitectures

18.1 Cortex-X925

18.1.1 Overview

Cortex-X925 是 Arm 面向旗舰移动设备与高性能客户端场景设计的 Cortex-X 系列核心, 采用 Armv9.2 体系结构。相较 Cortex-X4, 它同时扩大控制流供给、在途窗口、向量执行和访存并行度, 设计重点从“提高局部峰值”转向“让宽流水线持续获得正确路径与及时数据”。

Arm 的参考评估显示, X925 在 Geekbench 6.2 上获得约 15% 的 IPC 提升; 前端分支预测和指令取值能力约提高一倍, 最大指令窗口容量约提高一倍, load 管线由 3 条增加到 4 条, 部分后端工作负载的可处理规模提高约 25–40%。这些变化共同降低控制流、窗口容量和数据供给形成串行瓶颈的概率。[59]

结构	主要特征
ISA	Armv9.2-A; SVE/SVE2; MTE 与 RAS 扩展
前端	约两倍分支预测与指令取值能力; 10-wide Decode
在途状态	最大指令窗口容量约为前代两倍
向量配置	128-bit 或 256-bit 向量长度; 后端可配置 2 或 4 条 128-bit 向量数据通路
访存	4 条 load 管线; 64 KB L1D
缓存	64 KB L1I; 每核 2 MB 8-way 或 3 MB 12-way 私有 L2
翻译	128 项 L1 ITLB、96 项 L1 DTLB、2048 项 8-way 共享 L2 TLB

Table 18.1: Cortex-X925 的主要微架构参数。向量长度、后端数据通路和 L2 容量可由具体实现选择。

18.1.2 Design Positioning: A Wider Mobile Core

移动核心的面积和功耗预算比服务器核心更受约束, 但高端应用的代码与数据 footprint 已接近桌面负载。浏览器 JIT、游戏引擎、相机处理和本地 AI 同时要求高分支预测质量、大窗口、宽向量与高访存带宽。X925 的扩张因而不是简单增加 ALU, 而是把前端、窗口和数据侧按相近比例扩大。

宽度只有在各阶段平衡时才有效。10-wide Decode 若长期受 taken branch 或 L1I miss 限制, 后端窗口不会被填满; 更大的窗口若缺少足够物理寄存器、调度项与 load queue, 只会把瓶颈移到分配端; 四条 load 管线若受 DTLB、L1D bank 或未命中队列限制, 也无法维持峰值。

移动平台还要考虑持续功耗。宽核心可以在短时间内快速完成前台任务并回到低功耗状态,但若向量、缓存和预测结构长期满载,频率会受到热设计与 SoC 功耗分配约束。X925 的峰值能力必须与具体芯片的电源管理、内存系统和散热实现一起评价。

18.1.3 Front End

18.1.3.1 Branch Prediction and Fetch Bandwidth

X925 把分支预测与指令取值能力提高到前代约两倍,目标是在控制流密集代码中跨越更多基本块,并为 10-wide Decode 提供持续输入。[59] 预测准确率决定错误路径频率,预测带宽决定每周可以推进多远,目标命中率则决定 taken branch 是否需要等待后级结构;三者共同形成有效前端吞吐。

宽前端尤其容易受短基本块影响。即使每条分支都较容易预测,若目标生成只能逐块串行推进,译码器仍会缺少足够指令。多目标或多块预测可以在一个周期内准备更远路径,但需要更大的 BTB 读取带宽、分支边界信息和前端队列。

错误路径的代价也随宽度增长。预测错误不仅丢弃更多已译码操作,还会产生无用 L1I、TLB 和数据预取请求。X925 因而需要让预测、检查点、重定向和前端重新填充协同工作,而不能只追求更高 taken-branch 吞吐。

18.1.3.2 Instruction Cache and Decode

X925 配置 64 KB L1I,缓存行为 64 B,并使用预解码与前端缓冲向 10-wide Decode 供给指令。[60] AArch64 固定 4 B 编码降低指令边界识别复杂度,但 cache-line 尾部浪费、跨页访问、taken branch 和 ITLB miss 仍会使原始字节带宽无法全部转化为有效指令。

不设置独立 MOP cache 也可以形成宽前端。固定长度编码、较宽 L1I 读取、预解码元数据和并行 decoder 可以直接向后端供给;代价是热点循环仍需经过译码能耗路径。是否加入操作缓存取决于重复译码成本、面积、命中率和前端时序,而不是判断核心先进程度的单一标准。

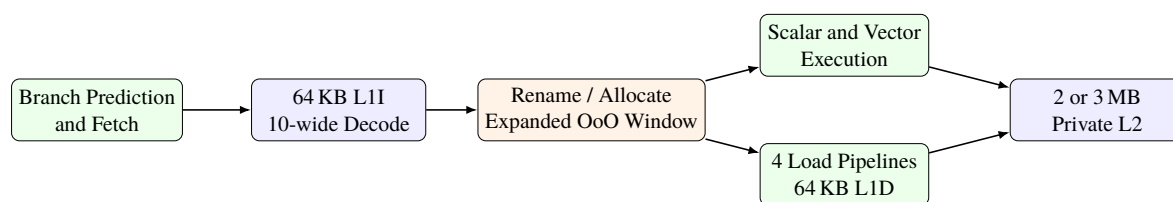


Figure 18.1: Cortex-X925 的分析性数据流。前端、在途窗口、执行与访存侧同步扩张;分支恢复和重定向路径未在图中展开。

18.1.4 Rename, In-Flight Window, and Recovery

X925 的最大指令窗口容量约为 Cortex-X4 的两倍。更大的窗口允许核心跨越长延迟 load、向量操作或分支依赖,寻找更远处的独立工作;它提高的是可观察范围,而不是消除真实数据依赖。

“指令窗口”不等同于单个 ROB 数字。ROB 维护程序顺序和精确状态, scheduler 保存可选择操作,物理寄存器保留未退休结果,LSQ 跟踪访存顺序。它们可能按不同幅度扩展,任何结构满载都会把反压传回 Rename/Allocate。

窗口扩大后, 恢复成本更重要。分支误预测需要恢复重命名映射、free list、预测历史和前端队列; 内存顺序违例可能只 replay 受影响数据流, 也可能触发更大范围清除。选择性恢复减少无关工作丢失, 却增加依赖追踪和状态复杂度。

退休仍按程序顺序进行。最老 cache miss 可以阻塞大量已经完成的年轻操作, 较大窗口只能推迟资源耗尽; 及时预取、更多并行 miss 和更早分支解析之所以重要, 最终都体现为 ROB 头部停顿时间缩短。

18.1.5 Execution Back End

18.1.5.1 Scalar and Branch Execution

标量后端承担整数算术、地址计算、比较、分支和系统操作。宽 dispatch 需要多个低延迟 ALU 与分支解析路径, 否则前端生成的控制流和地址操作会集中争用少数端口。执行单元数量仍不能直接等同于 issue 宽度, 因为 scheduler、寄存器读取与旁路网络可能在更早位置形成限制。

分支执行带宽决定预测验证速度。前端可以快速提出多个未来目标, 但条件值常依赖更早 load 或比较; 若分支长期等待操作数, 错误路径会继续占用窗口。缩短 producer-to-branch 依赖链和减少分支端口冲突, 与提高预测准确率同样重要。

18.1.5.2 SVE2 and Configurable Vector Width

X925 支持 SVE 与 SVE2, 体系结构向量长度可配置为 128 bit 或 256 bit; 后端配置还可表现为 2 条或 4 条 128-bit 向量数据通路。[60] 因而, 软件看到的向量长度、内部 lane 数量和每周期可启动的操作数量是三个不同概念。

256-bit 配置提高单条指令处理的数据量, 但也增加寄存器读取、旁路、写回和峰值功耗压力。多条 128-bit 路径则更容易并行处理独立操作, 并允许 SoC 实现者在面积与吞吐之间选择。乘法、FMA、置换、归约和密码操作仍可能具有不同启动间隔, 不能由总数据通路数直接推出统一峰值。

Arm 报告 X925 的 INT8 吞吐提高约 50%, 反映向量后端对图像、视频和本地 AI 预处理的优化。[59] 真实性能还取决于 load 带宽、数据布局 and 量化内核是否能避免频繁置换; 若操作数不能及时到达, 更多向量 lane 只会增加空闲容量。

18.1.6 Load/Store Pipeline and Memory Ordering

X925 把 load 管线由 3 条增加到 4 条, 并提高 L1D 数据带宽。[59] 一条 load 至少经历地址生成、DTLB、load queue、memory dependence prediction、L1D bank 和结果返回; 四条管线只有在这些阶段均具备足够并行度时, 才能形成接近峰值的持续读取。

64 KB L1D 在容量与访问延迟之间取舍。多 bank 组织可以支持并行 load, 但同 bank、跨 cache line 和未对齐访问仍会产生冲突。store 还需要分别处理地址和数据, 并在退休后等待缓存端口、一致性权限和内存排序条件; load 扩宽不意味着 store 路径自动具有相同带宽。

年轻 load 可以在预测为独立时越过地址未知的旧 store。若随后发现地址重叠, 核心必须重放 load 及其依赖操作; 若地址和数据满足条件, 则可以通过 store-to-load forwarding 避免访问缓存。更大的窗口和四条 load 管线会同时提高 memory dependence predictor、LSQ 比较和 replay 机制的压力。

单条指针链仍受 load-to-use 延迟限制。四条管线主要提高多个独立数组、对象或预取流的并行度; 若下一个地址依赖当前 load 返回, 再多端口也无法并行生成后继请求。间接访问性能因此同时依赖预取器、窗口深度和编译器能否暴露独立工作。

18.1.7 Cache and Prefetch Hierarchy

X925 每核具有 64 KB L1I、64 KB L1D, 以及可配置的 2 MB 或 3 MB 私有统一 L2。2 MB 配置为 8 路组相联, 3 MB 配置为 12 路组相联; 更大容量通过增加 way 数维持相同 set 数量尺度, 降低工作集冲突但增加标签与数据选择成本。[60]

私有 L2 减少访问系统级缓存和 DRAM 的频率, 也为指令、数据、页表遍历和预取请求提供共同后备层。3 MB 配置对大型应用和游戏资产更有利, 但不同核心之间不能共享其空闲容量; 多核共享数据仍需经过一致性互连或系统级缓存。

硬件预取需要在及时性和污染之间折中。流式与步长访问可以较稳定地提前填充, 复杂间接访问则更难预测。预取目标放得太近会挤占 L1 容量, 放得太远又可能来不及覆盖 demand miss; 移动 SoC 的带宽与功耗限制还要求预取器在拥塞时主动节流。

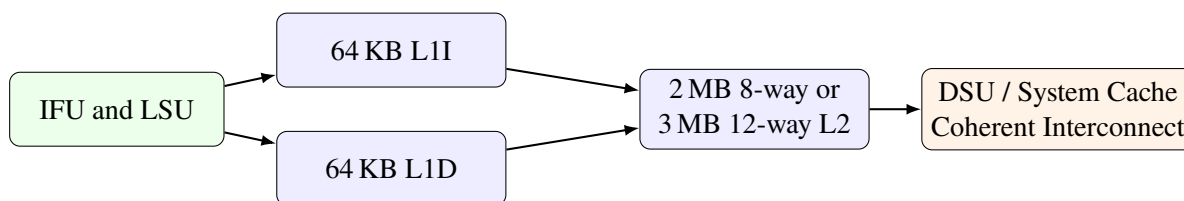


Figure 18.2: Cortex-X925 的核心可见缓存层次。私有 L2 容量由 SoC 实现选择, 系统级缓存与互连属于集群和平台配置。

18.1.8 Memory Management Unit

X925 使用两级 TLB: 128 项 L1 ITLB 服务前端, 96 项 L1 DTLB 服务 LSU, 2048 项、8 路组相联的共享 L2 TLB 承接两侧未命中。[60] 一级结构优化低延迟和并行访问, 二级结构扩大 translation reach; 指令与数据在 L2 TLB 共享容量, 也会相互竞争。

页表遍历请求可以缓存于 L2、系统级缓存和外部缓存, 并与普通数据请求共同占用带宽。X925 还支持面向 table walk 的预取控制, 使连续翻译描述符更早进入缓存层次。[60] 该机制减少的是 page walk 中读取页表项的等待, 与程序数据预取器处理的地址流不同。

大页可以显著扩大 TLB reach, 但不能覆盖所有场景。浏览器和移动运行时常包含大量小映射、共享库和保护边界, 虚拟化还可能引入两阶段翻译。评价 X925 的翻译能力时, 需要同时观察页大小分布、L2 TLB 命中率、page-walk 并发度和缓存中的页表数据竞争。

18.1.9 Reliability, MTE, and Array Protection

移动旗舰核心同样需要服务器式的错误检测。X925 对 L1D tag、L2 tag/data 和关键事务状态使用 SECDED ECC, 对可重新取得的部分结构使用 parity 或较轻保护。[60] 单比特纠错避免瞬态错误立即转化为软件故障, 双比特检测则允许系统通过 RAS 记录和异常路径采取恢复措施。

Memory Tagging Extension (MTE) 在指针与内存分配之间加入标签检查, 用于发现 use-after-free 和越界访问。标签检查需要与地址翻译、缓存元数据和异常报告协同; 同步模式提高错误定位精度, 异步模式允许更低前台开销, 但错误可能稍后报告。MTE 是体系结构安全能力, 其性能取决于具体缓存和流水线实现。

错误检测也受投机执行约束。奇偶校验或 ECC 错误可以在数据返回后才被发现, 核心必须保证错误值不会作为可退休结果; 若年轻操作已经投机消费, 相关数据流需要撤销并在精确边界报告。RAS 因而不仅是 SRAM 编码问题, 也与 replay、退休和系统错误处理共同构成完整路径。

18.1.10 System Integration

X925 通常与 Cortex-A 系列核心、DSU、系统级缓存、GPU、NPU 和内存控制器组成异构 SoC。每核 3 MB L2 可以降低远端流量, 却会增加芯片面积; 较小 L2 配合更大系统缓存则有利于多核共享。实现者需要在单线程命中率、总容量和互连带宽之间选择。

四条 load 管线和更积极预取会提高单核请求生成率。若系统级缓存、互连或 DRAM 无法同步扩展, 核心内部增加的并行度会转化为排队而不是更低延迟。频率、向量配置和 L2 容量也会改变功耗分配, 从而影响 GPU/NPU 同时工作时的持续性能。

18.1.11 Summary

Cortex-X925 的核心变化是前端、窗口与数据侧同步扩张。更高分支预测和取指能力为 10-wide Decode 准备正确路径, 约两次的最大窗口扩大乱序观察范围, 四条 load 管线和 2/3 MB 私有 L2 提高操作数到达率, 可配置的 128/256-bit SVE2 后端则覆盖从移动应用到本地 AI 的不同面积与吞吐目标。

其性能上限不由单一参数决定。短基本块会限制前端, 真实依赖会限制窗口, bank conflict 和 TLB miss 会限制四条 load 管线, 向量数据通路也受寄存器与缓存供给约束。X925 的意义在于为 SoC 设计者提供一套更宽的高性能核心模板, 而最终产品仍需通过缓存、频率、互连和功耗配置把这些能力转化为持续性能。

18.2 Neoverse N2

18.2.1 Overview

Neoverse N2 是 Arm 面向云计算、边缘基础设施与通信系统设计的 Armv9.0-A 处理器核心。它在 Neoverse N1 的能效取向上引入 SVE2、Memory Tagging Extension (MTE) 等体系结构能力, 并通过前端、乱序窗口和访存后端的同步扩展提高每线程性能。Arm 给出的代际比较称, N2 的单线程性能较 N1 提升约 40%; 这一结果来自厂商的参考配置和工作负载集合, 不能直接视为固定的 IPC 增幅。[61], [62]

从微架构参数看, N2 并不是依靠单一的大型结构获得性能提升。它保持 64 KB 一级指令和数据缓存, 将常规指令路径维持在每周周期 4 条, 同时增加能够提供每周周期 5 个宏操作的 MOP Cache; 重命名宽度扩展到每周周期 5 条, ROB 增至 160 项以上; 访存侧则由两条 load/store 通道和一条额外 load 通道提高数据供给能力。这些改动共同指向一个较为明确的设计重点: 减少前端中断和分支恢复损失, 并提高服务器代码中的访存并行度。[62]

18.2.2 Front End

18.2.2.1 Branch Prediction Bandwidth

N2 前端的重要变化不仅是提高分支预测准确率, 还包括提高单位周期内能够推进的控制流距离。预测器每周周期可以处理两个各覆盖 8 条指令的预测区域, 并在条件允许时跨越两个 taken branch。对于基本块较短、分支密度较高的服务器代码, 这种能力能够避免前端只能逐块生成取指地址, 从而使 4-wide 或 5-wide 的后续供给路径更容易保持忙碌。这里的两个 taken branch 是预测目标生成能力, 而不是保证每周周期从任意两个缓存位置取得完整指令块。[62]

结构	主要参数
分支预测	每周期预测两个 8 指令区域, 最高处理两个 taken branch; 64 项 Nano BTB、8K 项主 BTB
指令供给	64 KB、4 路 L1I; 1536 项、4 路 skewed MOP Cache; L1I 路径每周 期 4 条指令, MOP 路径每周期 5 个宏操作
在途状态 标量与分支 向量	5-wide rename、160 项以上 ROB、rename checkpointing 4 条整数 ALU 路径, 每周期最多解析两个分支 128-bit SVE/SVE2 向量长度, 官方比较材料标为两条 128-bit 路 径
访存	两条 load/store 通道加一条 load 通道; L1 命中读取带宽为 3 × 16 B/cycle, store data 带宽为 32 B/cycle
缓存与翻译	64 KB、4 路 L1D; 512 KB 或 1 MB、8 路私有 L2; 48 项 ITLB、44 项 DTLB、1280 项共享 L2 TLB

Table 18.2: Neoverse N2 的主要公开微架构参数。预测、取指、重命名和访存宽度描述的是不同流水阶段, 不能直接相加为统一的发射宽度。

目标预测采用容量和时序不同的结构。64 项 Nano BTB 位于快速路径, Arm 将其描述为可以避免命中分支产生 taken-branch bubble; 8K 项主 BTB 则覆盖更大的代码工作集。与 N1 相比, N2 的方向预测状态增加约 50%, 主 BTB 从 6K 项增至 8K 项, 并新增 Alt-Path Branch Prediction。Alt-Path 的状态组织和选择算法未被定义为某种特定学术预测器, 更适合把它理解为主预测结果之外的补充候选路径。[62]

N2 还可以对未命中 BTB 的条件分支进行较早的重新定向, 并把 Fetch Queue 从 N1 的 12 项扩大到 16 项。更深的队列使分支预测驱动指令预取能够维持更多在途请求, 从而降低大型代码 footprint 下的 L1I 未命中暴露时间。分支目标同时可以使用软件上下文号进行标记, 以减少不同执行上下文对目标预测状态的直接混用。[62]

18.2.2.2 Instruction Cache and MOP Cache

N2 配置 64 KB、4 路组相联的 L1 指令缓存, 缓存行为 64 B。常规 L1I 路径每周期提供 4 条指令; 新加入的 Macro-operation Cache (MOP Cache) 可以提供每周期 5 个宏操作。MOP Cache 容纳 1536 个已经译码和优化的宏操作, 采用 4 路 skewed associative 组织, 并作为取指流水线中的 L0 指令缓存工作。[62], [63]

MOP Cache 的作用不只是绕过重复译码。Arm 的材料还将更多指令融合与 MOP 路径联系起来, 因此一个 MOP 不一定对应一条原始体系结构指令。命中时, 前端可以用更高的操作供给率向重命名阶段交付热代码; 未命中时则退回 L1I 和 4-wide 常规译码路径。由此形成的双路径解释了 N2 为何同时保留较宽的一级指令缓存和容量不大的 MOP Cache: 前者承担较大代码工作集, 后者提高热点区域的吞吐和能效。

18.2.3 Rename, In-Flight Window, and Recovery

N2 的重命名宽度由 N1 的每周期 4 条提高到 5 条, ROB 从 128 项扩大到 160 项以上。更宽的重命名路径用于吸收 MOP Cache 的 5-wide 供给, 而更大的 ROB 则允许核心跨越较长的执行或缓存停顿寻找独立工作。官方材料没有公开物理寄存器数量、调度队列容量以及整数和向量操作的具体分配方式, 因此 160 项以上 ROB 不能直接等同于同等规模的统一调度窗口。[62]

N2 新增 rename checkpointing。遇到分支预测错误时, 处理器可以恢复到较早保存的重命名映射, 而不必仅依赖从 ROB 尾部逐项重建状态。这项机制与前端更积极的多分支预测相互配合:

预测器扩大了错误路径可能覆盖的范围, 检查点则控制错误发生后的映射恢复成本。Arm 同时将整体流水线深度从 N1 的 11 级缩短为 N2 的 10 级, 但这一比较值不能直接解释为所有分支误预测都具有固定的 10 周期代价。[62]

N2 每周期最多可以解析两个分支, 高于 N1 的一个。预测带宽决定前端能够多快地提出未来路径, 分支解析带宽则决定后端能够多快地验证这些路径。若解析速度长期低于预测推进速度, 错误路径会在 ROB 中积累并占用执行资源; 因此, 多分支预测、rename checkpointing 和双分支解析应被视为同一套控制流流水线的不同环节。[62]

18.2.4 Execution Back End

N2 配置四条整数 ALU 路径, 相比 N1 增加一条。增加常用整数执行能力可以降低地址计算、循环控制、比较和普通算术之间的结构冲突, 但执行单元数量并不自动给出总 issue 宽度。整数乘除、分支、访存和向量操作仍可能使用不同的专用路径, 调度器划分、端口兼容性和操作数读取带宽也没有在公开材料中完整披露。[62]

向量侧实现 SVE 与 SVE2, 并将体系结构向量长度固定为 128 bit。Hot Chips 的路线图将其描述为两条 128-bit SVE2 路径, 同时支持 bfloat16; TRM 还列出了 SVE2 AES、SHA-3、SM4 和 bit-permute 等扩展。这里需要区分 SVE 的向量长度无关编程模型与具体核心的数据通路宽度: 软件可以使用可扩展向量语义编写代码, 但 N2 每条体系结构向量的实现长度为 128 bit。[62], [63]

这种配置更侧重在有限面积和功耗内提高通用向量覆盖率, 而不是通过超宽向量单元追求最高浮点密度。对网络处理、压缩、加密、字符串和整数数据处理而言, 两条中等宽度路径可以并行执行多个独立操作; 真实性能仍取决于指令混合、置换与归约吞吐, 以及访存流水线能否持续提供数据。

18.2.5 Load/Store Pipeline

N2 的地址生成资源由两条 load/store 共用通道和一条额外 load 通道组成。三条通道允许每周生成更多读取地址, 但不能被简单理解为三条完全相同的访存流水线: 第三条只承担 load, 而 store 还需要地址和数据路径共同完成。官方比较材料给出的 L1 load-hit 带宽为每周三次 16 B 读取, store data 带宽则为 32 B/cycle。[62]

TRM 从缓存接口角度给出了另一组数据通路描述: 整数执行侧具有三条 64-bit 读取和四条 64-bit 写入路径, 向量侧具有三条 128-bit 读取和两条 128-bit 写入路径。这些接口宽度描述缓存与执行管线之间的数据交换能力, 而 AGU 数量描述地址生成能力; 两组数字位于不同阶段, 不应互相替代。跨缓存行、未对齐访问、地址翻译缺失和 bank 冲突仍可能使实际完成率低于峰值。[63]

三条 load 路径还会增加 load queue、DTLB、L1D bank 和未命中处理资源的压力。只有当程序存在足够多互不依赖的地址, 且 cache miss 可以并行在途时, 增加的地址生成和读取带宽才能转化为更高的 memory-level parallelism; 指针依赖链上的单个未命中仍然受 load-to-use 延迟限制。

18.2.6 Cache Hierarchy and Prefetching

N2 配置 64 KB、4 路组相联的 L1 数据缓存, 以及每核 512 KB 或 1 MB、8 路组相联的私有 L2。L2 内部读带宽和写带宽分别提高到 64 B/cycle。L1D 与 L2 采用严格包含关系, 因此 L1D 中的数据也占用 L2 容量, L2 数据被替换时可能反向失效对应的 L1D 缓存行; L1I 与 L2 在默认配置下则为弱包含关系。[62], [63]

严格包含关系简化了部分一致性和探测路径,但也意味着私有 L2 的标称容量并非完全独立于 L1D。另一方面,较大的 L2 和翻倍的内部带宽为三条 load 路径、指令取指以及页表遍历提供共同的后备层。评价 N2 的缓存层次时,容量、带宽、包含关系和未命中并行度需要同时考虑。

N2 保留 stride、spatial/region 和 stream 等预取方式,并加入 Correlated Miss Caching (CMC) 所代表的时间相关预取。传统步长预取器根据相邻地址之间的算术关系生成请求;CMC 则记录反复出现的 cache-miss 序列,在相似上下文再次出现时提前请求后续地址。因此,它可以覆盖部分缺少固定步长、但历史访问顺序重复的数据结构遍历。该机制并不是直接预测指针的数值,而是利用过去观察到的未命中相关性。[62]

硬件预取器可以把 load 侧请求送入 L1 或 L2, store 侧预取则面向 L2。预取越积极,越能及时覆盖长延迟访问,同时也越可能占用缓存、未命中队列和外部带宽。N2 支持根据系统返回的 CBusy 拥塞信息调整预取激进程度,官方材料将其划分为四级。这样的反馈路径使核心在带宽充足时扩大覆盖率,并在共享内存系统拥塞时减少低准确率请求。[62], [63]

18.2.7 Memory Management Unit

N2 的一级指令 TLB 为 48 项全相联结构,一级数据 TLB 为 44 项全相联结构。两者命中时均可在一个时钟周期内提供翻译。共享 L2 TLB 采用 5 路组相联组织,共有 1280 项,同时服务指令、数据以及虚拟化中的中间物理地址翻译。L1 DTLB 未命中但 L2 TLB 命中时,相比 L1 DTLB 命中增加约三个周期,实际延迟还可能受到请求仲裁影响。[63]

L2 TLB 支持从 4 KB 到 1 GB 的多种页与块粒度,并保存部分两阶段翻译所需的信息。较大的共享翻译结构对服务器核心尤其重要,因为代码、堆、共享库和虚拟机可以同时扩大 translation footprint。L2 TLB 的容量虽然明显高于一级结构,但指令和数据请求也会竞争同一资源,因此页大小分布与访问局部性仍然会影响有效覆盖范围。

MMU 还包含 translation-table prefetcher,可以识别连续页表访问并预取下一项页表数据。它与数据预取器解决的是不同问题:前者减少页表遍历过程中读取翻译描述符的等待,后者提前请求程序数据。对于大地址空间、频繁 TLB miss 或嵌套翻译场景,页表预取和 L2 TLB 覆盖率共同决定 page walk 有多少延迟暴露给前端和访存流水线。

18.2.8 Summary

Neoverse N2 的主要微架构变化可以归纳为三条相互关联的路径。前端通过双区域分支预测、两 taken-branch 目标生成、分级 BTB 和 MOP Cache 提高正确路径供给;中段通过 5-wide rename、160 项以上 ROB、检查点和双分支解析扩大并行窗口并控制恢复成本;访存侧通过三条地址生成通道、更高 L1/L2 带宽、CMC 预取和较大的 L2 TLB 减少数据与翻译等待。

这些结构说明, N2 并不是对 N1 的简单频率升级,也不是依靠极大乱序窗口追求单点峰值。它把有限的面积投入到服务器代码中经常出现的控制流、代码 footprint、数据访问和地址翻译瓶颈,并通过适度扩宽的后端消费新增供给。其实际性能仍取决于 MOP Cache 覆盖率、分支预测质量、独立访存数量以及共享内存系统是否能够承受预取和多核心并发流量。

18.3 Neoverse V2

18.3.1 Overview

Neoverse V2 是 Arm 面向云计算、高性能计算和机器学习场景设计的 V 系列服务器核心,也是 V 系列中首个采用 Armv9 的产品。Arm 将其定位为较 Neoverse V1 更强调每线程性能的实现,并宣称在部分 HPC 与机器学习负载中可达到最高约两倍性能。该结果包含核心、缓存、频率、编译器和平台配置的共同作用,不能直接解释为固定的 IPC 增幅。[64], [65]

V2 的微架构扩张集中在四个相互连接的位置:run-ahead 分支预测提前生成控制流,6-wide 常规译码和 8-wide MOP 路径提高操作供给,320 项以上乱序窗口与分布式调度器扩大可见并行性,三条访存通道、四 bank 私有 L2 和多种预取器提高数据到达速度。架构框图还给出 8-wide dispatch、8-wide retirement、6 条整数 ALU 路径、2 条分支路径与四组 128-bit 向量数据通路。这里的 320 项以上描述整体乱序窗口,不能在缺乏进一步说明时直接写成 ROB 容量。[66]

18.3.2 Front End

18.3.2.1 Run-Ahead Branch Prediction

V2 将分支预测流水线与实际指令获取进一步解耦。预测器可以在 fetch 之前运行,把未来路径写入更深的前端队列;这样不仅可以提前确定控制流,还能把预测地址作为指令缓存预取请求,以覆盖 ITLB、L1I 和更低缓存层的等待。其峰值为每周期预测两个分支,但这一数字表示预测路径推进能力,并不保证每周期都能从两个任意缓存位置取得完整指令块。[66]

方向预测器采用具有分阶段输出的 8-table TAGE 组织。与 V1 相比,V2 将 TAGE 表容量扩大约一倍,并改为 2 路组相联,同时使用更长的历史信息;间接分支则具有专用预测器。目标预测采用两级 BTB,Nano BTB 扩大约十倍,主 BTB 被拆分为两个层级,总表项数增加约 50%。这些比例能够说明容量和时序组织发生了变化;各级 BTB 的绝对表项数并未披露,因而不能进一步确定其索引、标签或替换方式。

前端缓冲也随之扩展。Fetch Queue 从 16 项增加到 32 项,Fill Buffer 从 12 项增加到 16 项,指令 TLB 与缓存读取带宽则提高到 V1 的两倍。更深的队列使预测器能够在实际译码之前维持更多候选路径和在途取指请求,但错误路径也可能占据更多前端状态,因此它必须与后文的重命名检查点和恢复流程共同理解。[66]

18.3.2.2 Instruction Cache and MOP Cache

V2 配置 64 KB、4 路组相联的 L1 指令缓存,缓存行为 64 B。L1I 路径最高提供每周期 6 条部分译码指令;并行的 Macro-operation Cache(MOP Cache)保存已经译码和优化的操作,命中时最高提供每周期 8 个操作。MOP Cache 容纳 1536 个宏操作,采用 4 路 skewed associative 组织,并作为 L0 指令缓存工作。[66], [67]

V2 相比 V1 缩小了 MOP Cache,以换取更好的面积和能耗效率。这个选择说明操作缓存并非越大越好:较大的 L1I、翻倍的取指带宽和更深的 Fetch Queue 可以承担大型代码工作集,而容量较小但带宽较高的 MOP Cache 负责热点路径。MOP 命中还可以利用更高的 8-wide 通路绕过常规译码,因此容量、命中率和供给宽度需要共同评价。[66]

两级结构也采用不同的替换策略。L1I 使用近似最近最少使用的 PLRU,而 MOP Cache 使用 Not Recently Used(NRU)。PLRU 在较大的指令工作集上保留一定的时间局部性信息;NRU 只区分近期是否使用,状态和更新成本较低,更适合位于高带宽关键路径上的小型操作缓存。替换策

略的差异说明, V2 并未把 MOP Cache 当作 L1I 的简单缩小版, 而是把它作为延迟、吞吐和能耗目标不同的前端层级。[67]

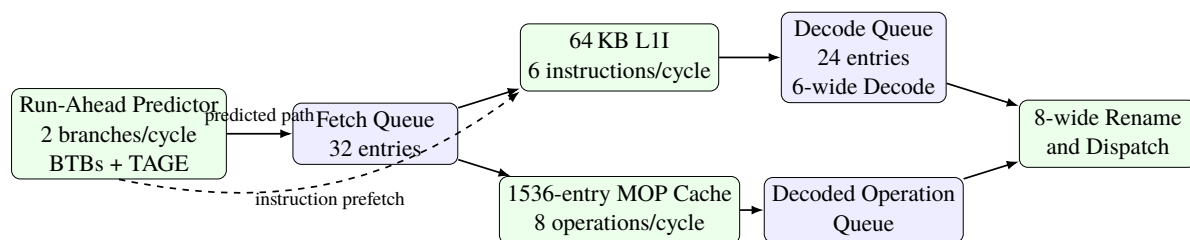


Figure 18.3: 依据架构框图重绘的 V2 前端数据流。预测先于实际取指推进, L1I 与 MOP Cache 形成带宽不同的两条供给路径; 该图不表示物理布局。

18.3.3 Decode, Rename, Dispatch, and Retirement

常规译码器由 V1 的 5 路扩展到 6 路, Decode Queue 从 16 项增加到 24 项。来自 L1I 的部分译码指令经过并行 decoder, 而 MOP Cache 提供的完整操作以更高带宽绕过常规译码; 两条路径最终都进入 8-wide rename/dispatch。译码器还支持简单指令融合, 使若干相邻体系结构指令在进入后端前减少为更少的内部操作。[66]

V2 具有 320 项以上的乱序窗口, 并可以每周期退休最多 8 个操作。窗口规模决定处理器能够同时保留多少在途工作, 但不等于所有操作都进入一个统一调度阵列; V2 实际采用多个独立 issue queue, ROB、物理寄存器、访存队列和各调度器的具体容量也不能由 320 这一数字直接拆分得到。

重命名侧的检查点总数由 5 个增加到 6 个, 其中向量检查点由 3 个增加到 5 个。V2 同时改进 rename rebuild flow, 使流水线 flush 后能够更快重建体系结构与投机映射状态。增加检查点可以减少频繁分支恢复时的映射回滚距离, 更多向量检查点则避免宽向量数据流使恢复状态过早耗尽。各类错误的恢复周期并未披露, 因此这些改动应被理解为恢复路径优化, 而不是固定的误预测延迟承诺。[66]

18.3.4 Distributed Issue and Execution

V2 后端由多个彼此独立的 issue queue 组成, 部分队列可以在同一周期选择两个操作。操作数不随指令一起保存在 issue queue 中, 而是在发射之后才从物理寄存器文件或近期结果缓存读取。这种 late read after issue 可以减小调度项宽度, 但要求端口选择、寄存器读取和旁路网络在发射后继续保持正确对应。

Result Cache 保存尚未写回或近期产生的值, 并配合 lazy writeback 减少物理寄存器文件写端口压力。若消费者很快到达, 它可以从结果缓存或旁路获得数据; 只有需要长期保存的值才必须及时进入物理寄存器文件。该组织用额外的局部状态换取较低的全局寄存器带宽, 是 V2 扩大执行宽度而不使寄存器文件成为唯一瓶颈的重要手段。[66]

整数侧配置 6 条 ALU 路径与 2 条分支路径, 其中部分路径还承担 shift、multiply、integer MAC、divide、CRC 和系统操作。与 V1 相比, V2 新增两条单周期 ALU, 相关标量/乘法 issue queue 从 20 项增加到 22 项。向量 issue queue 则从 20 项增加到 28 项, 谓词操作带宽提高一倍。不同路径的能力并不相同, 因此“6 条 ALU”不能被解释为任意六条整数指令均可持续并行执行。

V2 支持部分寄存器到寄存器和立即数 move 的零延迟处理, 并增加包括 CMP+CSEL/CSET 在内的融合形式。这些优化不会提高算术单元的峰值数量, 却可以缩短依赖链并减少调度器与执行端口中的辅助操作。向量侧实现 SVE/SVE2, 体系结构向量长度为 128 bit; 后端框图将其概括为四

组 128-bit 低延迟数据通路, 但具体指令仍可能因乘法、置换、归约或密码功能而具有不同吞吐。
[66], [67]

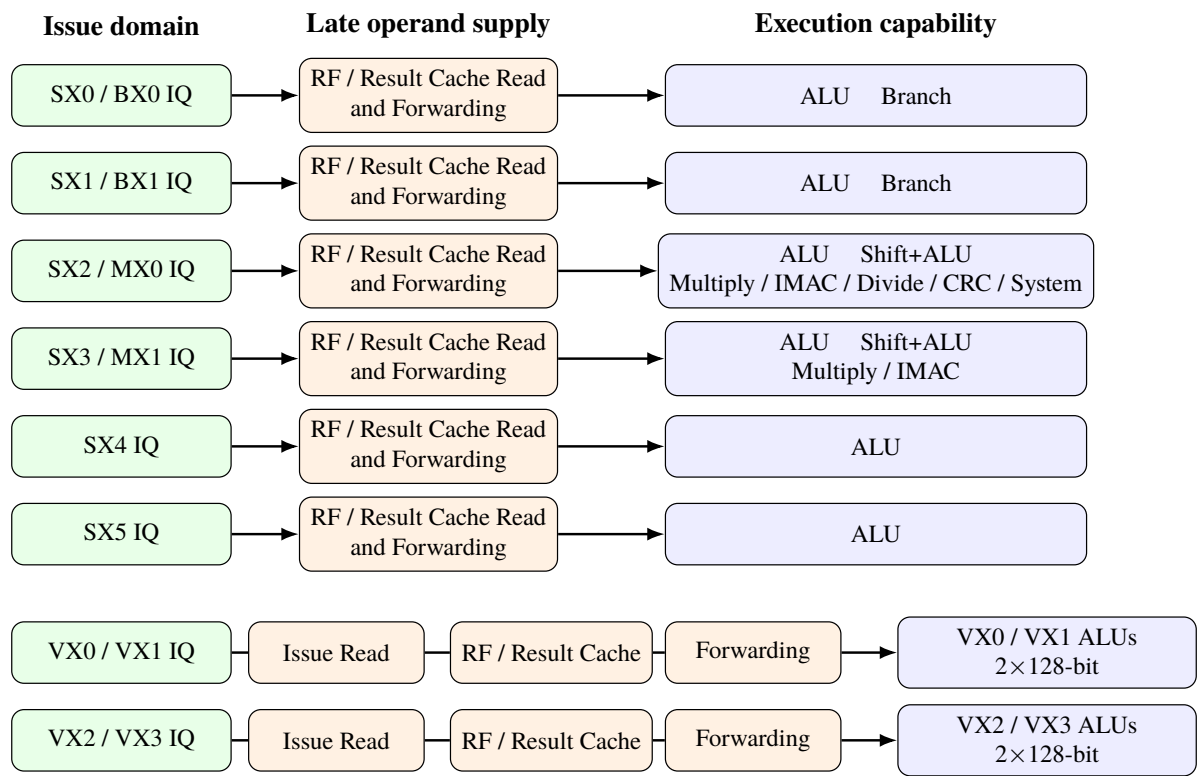


Figure 18.4: 根据官方执行框图重绘的 V2 标量与向量端口拓扑。六条标量路径并非同构, 分支、移位、乘除、CRC 和系统操作只出现在部分路径; 图中连接表示逻辑供给关系, 不表示物理布局或所有指令的固定端口绑定。

18.3.5 Load/Store Pipeline and Memory Ordering

V2 的访存后端包含两条 load/store 共用通道和一条额外 load 通道。整数侧配置四条 8 B 结果总线, 浮点、SVE 与 Neon 侧配置三条 16 B 结果总线。地址生成通道和结果总线位于访存流水线的不同阶段: 前者决定可以启动多少地址访问, 后者决定命中数据能够以多大带宽返回执行后端, 二者均可能受到缓存 bank、对齐和翻译缺失限制。[66]

Store-to-load forwarding 可以在接近 L1 命中的延迟下把更老 store 的数据交给年轻 load。为了降低全相联比较的能耗和时序压力, V2 使用基于虚拟地址哈希的转发辅助结构; 最终仍需验证地址、访问大小与字节覆盖关系, 以保证不会遗漏更老的别名 store。Store Buffer 以及 Read-After-Read、Read-After-Write 跟踪结构也得到扩展, 但其绝对容量并未披露。

V2 将部分 read-after-read hazard 的跟踪延长到 L2 cache line 的生命周期, 以减少一致性或地址冲突较晚暴露时触发的流水线 flush。L1 DTLB 从 V1 的 40 项增至 48 项, 并保持全相联和多页大小支持; L1D 替换策略则由 PLRU 改为动态 RRIP。这里的 RRIP 用于预测缓存行的再引用距离, 与内存相关性预测解决的是不同问题。[66]

三条访存通道分别从 LS0、LS1 和专用 LD issue queue 接收操作, 经过地址生成和各自的 DTLB 查询后, 共享 L1D 的标签与数据阵列。命中数据在返回执行端之前还要经过对齐与转发选择, 因此三条地址通道、缓存阵列端口和结果总线并不是可以彼此替代的同一个宽度。Store Buffer 位于写入路径的汇合位置, 并与 RST、MB 等辅助结构协同; 这些名称在 Arm 的框图中用于标示

内部数据流, 但其完整组织和算法并未公开。

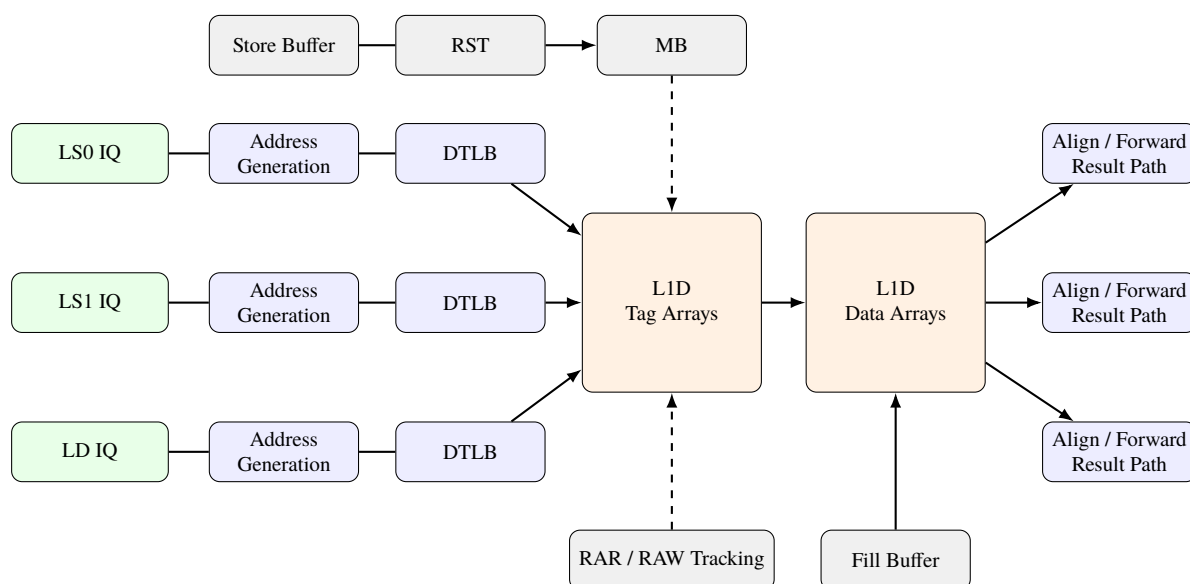


Figure 18.5: 根据 V2 访存框图重绘的 LSU 数据流。LS0、LS1 与 LD 路径分别完成调度、地址生成和翻译, 再共享 L1D 标签与数据阵列; 虚线表示辅助控制或跟踪关系, 不表示固定的单周期旁路。

18.3.5.1 Write Streaming Mode

普通 write miss 会像 read miss 一样分配缓存行并启动 linefill, 但连续覆盖大块内存时, 这一步可能先读入整行、随后又被新数据完全覆盖。V2 因而能够识别连续的 full-cache-line write: 当这种模式达到可配置阈值后, L1 和 L2 进入 write streaming mode, 后续写缺失不再为该行启动 linefill, 而是把数据直接送往 L2 或更低层。该机制尤其适合 memset 一类整行覆盖操作, 可以减少无用读流量、缓存污染和 linefill 能耗。[67]

Write streaming 并不改变 load 的正常分配行为。若出现非整行写入, 或者处理器读取正在流式写出的同一缓存行, 核心会退出该模式, 以恢复部分写合并和读后写所需的缓存语义。V2 还可以把连续整行写的观察结果向系统级缓存与 DRAM 方向传递, 并分别设置阈值; 因此这不是单个 L1D 的局部优化, 而是一条可以贯穿私有缓存与外部内存系统的写流提示。

18.3.5.2 Atomics and Exclusive Accesses

对可缓存内存的原子操作, V2 会根据缓存行所在位置选择 near atomic 或 far atomic。L1D 命中时核心优先尝试在近端完成; 若请求在集群内均未命中且互连支持远端原子, 则可将运算交给互连执行。反之, 命中集群内后级缓存或互连不支持该能力时, 操作由集群的后级缓存与内存系统完成。软件可以用 PLDW、PRFM PSTL1KEEP 或控制寄存器设置提高 near atomic 的倾向, 但最终路径仍取决于缓存行位置和一致性状态。[67]

Load-Exclusive、Store-Exclusive 与 CLREX 则由核心内部的两状态 exclusive monitor 管理。该监视器只记录 open 与 exclusive 状态; 它与缓存一致性和外部监视机制共同决定条件存储是否成功。原子读改写与 exclusive 序列都可能执行同步操作, 但前者可以被内存系统直接实现, 后者则依赖显式的独占加载、条件存储和监视状态, 二者不应视为同一条微架构路径。

18.3.6 Hardware Prefetching

V2 同时使用在 L1 和 L2 访问流上训练的多预取引擎, 并可以把结果放入 L1 或 L2。部分引擎使用虚拟地址训练, 从而跨越页面并协同触发 TLB 预取; 预取距离会根据准确率和及时性动态调整。这种组织的目标不是始终生成最多请求, 而是在数据真正被消费前完成填充, 同时控制缓存污染和未命中资源占用。[66]

在规则访问之外, V2 增加或强化了三类机制。Global Spatial Memory Streaming (GSMS) 覆盖比普通 SMS 更大的空间偏移; Sampling Indirect Prefetch (SIP) 针对指针解引用形成的间接地址; TableWalk 预取则提前请求页表项。L2 GSMS 还把指令 PC 纳入训练, 以减少不同访问流之间的混淆。它们分别处理空间 footprint、间接地址和翻译描述符, 不能简单归为同一种“高级预取器”。

预取请求使用低于 demand request 的 QoS 等级, 目的是在互连或缓存端拥塞时优先保证真正阻塞执行的访问。错误预取仍会占用 L1/L2、Transaction Queue 和外部带宽, 因此 V2 对训练事务进行更严格过滤, 并依靠准确率、及时性与系统反馈限制激进程度。

18.3.7 Private L2 Cache

V2 支持每核 1 MB 或 2 MB 私有统一 L2, 采用 8 路组相联和四个独立 bank。每个 bank 每两个周期可以完成一次 64 B 读取或写入, 四 bank 聚合带宽最高为 128 B/cycle。该数字要求请求能够分散到不同 bank, 不能解释为单条 load 或同一 bank 能够每周期传输 128 B。[66], [67]

L2 的典型 load-to-use 延迟为 10 个周期, 2 MB 配置仍维持与 1 MB 配置相同的这一标称路径。L2 配置 96 项 Transaction Queue, 以容纳来自指令、数据、页表遍历、预取和一致性请求的在途事务。较高聚合带宽只有在 bank 分布与队列容量均未成为瓶颈时才能持续实现。[66]

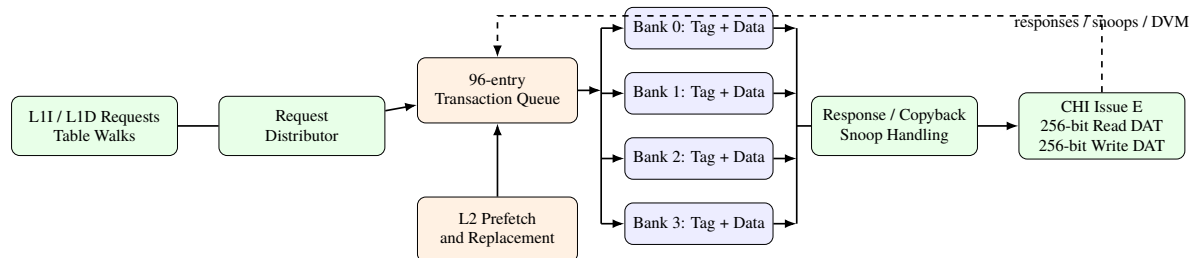


Figure 18.6: V2 私有 L2 的请求与事务组织。四个 bank 提供聚合内部带宽, Transaction Queue 维持未完成事务, CHI 数据通道连接集群互连; 图中没有展开 bank 内部流水级与仲裁细节。

L1D 与 L2 采用严格包含关系, 因此 L2 替换可能反向失效对应的 L1D 缓存行。L1I 与 L2 的关系取决于可配置的 instruction-cache coherency 选项: 默认可以采用弱包含关系, 启用该选项后则变为严格包含。因此, 不能笼统地把指令侧和数据侧都写成无条件严格包含。[67]

替换策略使用 6-state RRIP, 并通过 dead-block prediction 区分只使用一次和会被多次使用的数据。L2 copyback 还可以向系统级缓存传递替换提示, 使私有缓存观察到的局部复用信息影响更远层级的放置决策。这些机制的价值在于提高 2 MB 容量的有效利用率, 而不是单纯延长缓存行驻留时间。[66]

18.3.7.1 Transaction and Write Paths

96 项 Transaction Queue 并不意味着对外可以同时维持 96 个任意类型事务。CHI 接口的总体未完成事务上限为 92; read、write 和 Distributed Virtual Memory (DVM) 事务各自的单类上限也都是 92, 能够接收的未完成 snoop 上限为 53。这些数值描述不同类别单独达到的能力, 不可相加为

层级	替换策略	微架构侧重点
L1I	PLRU	用较低状态成本近似保留指令时间局部性
MOP Cache	NRU	减少热点操作缓存的状态更新与关键路径压力
L1D	Dynamic RRIP	按访问行为调整再引用预测, 降低一次性数据污染
L2	6-state RRIP; dead-block prediction	细分驻留优先级, 并区分一次使用与多次复用的数据

Table 18.3: V2 各私有缓存层级采用的替换策略。不同策略反映了容量、访问时延和数据复用模式的差异。

92 次读取、92 次写入和 92 次 DVM 同时在途。TQ 项数与 92 项接口上限之间的差额属于内部资源安排, 不能据此推断具体保留用途。[67]

L2 通过一个 CHI Issue E 接口连接集群, 读、写 DAT 通道宽度均为 256 bit, 即每个数据拍 32 B。它与四 bank 的 128 B/cycle 聚合带宽位于不同边界: 前者约束核心私有 L2 与互连之间的数据传输, 后者描述缓存 bank 内部并行访问能力。大量 L2 命中可以利用内部 bank 并行, 而持续未命中或 copyback 最终仍会受到 CHI 通道、互连信用与远端接收能力限制。[67]

写事务侧还针对所有权获取和整行覆盖做了专门优化。对于共享态写命中, V2 可以使用 CHI 的 MakeReadUnique, 以缩短取得可写权限的流程。普通写回与缓存维护操作可以合并为更少的事务; Write*Zero 类事务则允许 memset 式零填充使用专门的整行语义。这些优化减少的是一致性往返、事务数或数据传输, 而不是提高 L1D 的地址生成宽度。[66]

18.3.7.2 Array Protection

V2 对可能保存脏数据或关键事务状态的阵列使用 SECDED ECC, 包括 L1D 的标签与数据、L2 的标签与数据以及 L2 Transaction Queue; 单比特错误可被纠正, 双比特错误可被检测。L1I、MOP Cache 和 MMU RAM 保存的数据可以从更低层重新取得, 因此采用奇偶校验保护, 发现单比特错误后通过重新取数恢复。这样的分层保护避免为所有阵列支付相同的 ECC 面积与访问代价, 同时保证不可简单丢弃的写数据具有更强的纠错能力。[67]

18.3.8 Memory Management Unit

V2 的 L1 ITLB 和 L1 DTLB 均为 48 项全相联结构, 命中时可在一个时钟周期内提供翻译。共享 L2 TLB 采用 8 路组相联组织, 共有 2048 项, 并同时保存指令、数据、两阶段翻译和部分页表遍历中间结果。L1 DTLB 未命中但 L2 TLB 命中时, 相比 L1 DTLB 命中增加约 6 个周期, 实际延迟还会受到请求仲裁影响。[67]

L2 TLB 支持从 4 KB 到 1 GB 的多种页与块粒度, 并包含 translation-table prefetcher, 用于识别连续页表访问并提前请求下一项。较大的 2048 项共享结构提高了服务器程序和虚拟机的 translation reach, 但指令和数据请求仍会竞争相同容量与访问带宽, 页大小分布也会改变其有效覆盖范围。

18.3.9 Summary

Neoverse V2 的设计可以概括为“预测先行、双路供给、分布式调度和高带宽数据层次”。run-ahead 预测器与更大的 BTB、TAGE 和前端队列负责提前准备正确路径; 6-wide Decode 与

8-wide MOP、Rename、Dispatch 和 Retirement 路径提高操作吞吐; late read、Result Cache 和 lazy writeback 则降低宽后端对物理寄存器文件的集中压力。

访存侧的三条通道、低延迟 store-to-load forwarding、整行写流识别、多引擎预取、四 bank L2 和 2048 项 L2 TLB 共同服务于更大的乱序窗口; near/far atomic、92 项对外事务能力与 CHI 写事务优化则缩短同步和一致性路径。V2 的关键不只是把单个结构做宽, 而是让控制流、操作数和数据能够足够早地到达执行端。其限制同样来自这些结构之间的耦合: MOP Cache 未命中、bank conflict、转发失败、预取污染或翻译缺失, 都可能使名义上的 8-wide 后端无法持续获得有效工作。

Chapter 19

NVIDIA Microarchitectures

19.1 Vera and Olympus

19.1.1 Overview

Vera 是 NVIDIA 首款采用自研 Olympus 核心的服务器处理器。从微架构角度看, Olympus 属于一颗宽发射、深乱序、高访存并行度的 Armv9.2 核心。其最值得关注的并不是“面向 Agent”这一产品标签, 而是它如何处理三类长期限制单线程性能的问题: 控制流难以提前确定、真实数据依赖链难以并行化, 以及间接访存难以及时进入缓存层次。[68], [69]

每个 Olympus 核心支持两个硬件线程, 并具有 10-wide decode、10-wide rename/allocate、较多整数和向量执行路径、四条 load 路径、两条 store 路径以及 2 MB 私有 L2。芯片级的分布式 SLC 与高带宽内存只在它们影响核心可见延迟、未命中吞吐和共享资源竞争时讨论。[68], [70]

项目	主要参数
核心与线程	88 个 Olympus 核心, 176 个硬件线程
ISA	Armv9.2, 支持 SVE2 与 FP8
前端	最高每周期两个 taken branch, 128 B/cycle 指令读取, 10-wide decode
执行资源	8 个简单整数 ALU、2 个复杂整数 ALU、4 个分支单元、6 个 128-bit SVE 单元、4 条 load 路径与 2 条 store 路径
私有缓存	64 KB L1I、96 KB L1D、每核 2 MB L2
共享层级	164 MB 分布式 System-Level Cache / L3, 最高 3.4 TB/s SCF 剖分带宽
内存接口	最高 1.2 TB/s SOCAMM2 LPDDR5X

Table 19.1: 与 Olympus 核心供给和执行能力直接相关的主要参数。资源数量并不自动等价于可持续吞吐。

理解 Olympus 时, 最重要的是避免被单个宽度数字牵引。十路解码并不意味着每周期始终可以退休十条有效指令; 大量执行单元也不意味着它们拥有完全独立的调度端口; 四条 load 路径也不意味着 L1D、DTLB 与 load queue 能够在所有地址分布下保持同等吞吐。真正决定性能的, 是各阶段宽度是否平衡、操作数能否及时到达、投机结果能否低成本验证, 以及错误恢复是否会形成长时间的前后端空洞。

19.1.2 Design Positioning: A Wide Core Sustained by Speculation

Olympus 的整体思路可以概括为:先提供足够宽的取指、重命名和执行资源,再通过多类预测机制扩大这些资源能够看到的未来工作。分支预测推进控制流, value prediction 推进真实数据依赖, memory renaming 与图预取推进内存通信和地址依赖。相比单纯扩大 ROB, 这种设计试图直接改变依赖何时变为可执行的问题。

这类设计的难点在于, 宽度本身会制造新的瓶颈。前端若无法每周期给出足够多的正确目标, 十路解码便会空转; 调度器若不能及时唤醒操作数, 执行单元数量只会增加闲置容量; load/store queue、TLB、L1D banks 或 MSHR 若不足, 四条 load 路径会迅速把压力传递到更低层。因而, Olympus 的关键不在于某一级“很宽”, 而在于不同阶段能否持续保持相近的有效吞吐。

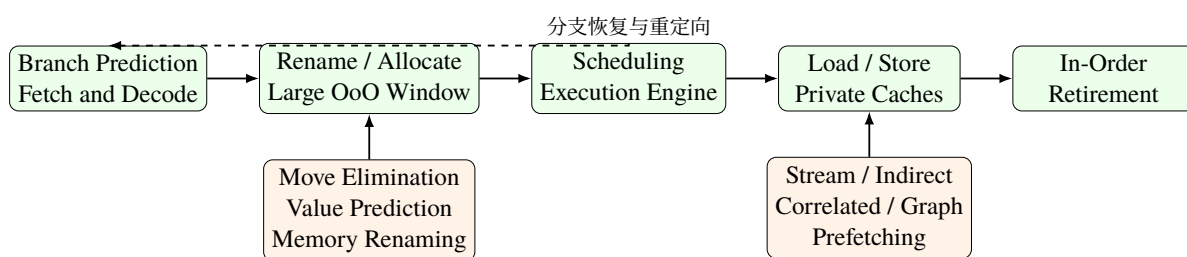


Figure 19.1: Olympus 的分析性数据流。辅助预测结构的作用, 是在前端、数据依赖和访存依赖三个位置扩大可提前执行的范围。

19.1.3 Pipeline Balance and In-Flight State

Olympus 的 decode、rename/allocate 与 retirement 名义宽度均为每周期 10 个操作。这种对称宽度有利于避免某个固定阶段在理想路径上过早成为上限, 但实际吞吐还取决于指令融合、微操作展开、分支密度和执行延迟。若一条架构指令被拆成多个内部操作, 或者某些指令需要进入微码或专用序列, 前端的十条指令并不会等价转化为十个可独立发射的后端操作。[68]

宽流水线还要求足够大的在途状态。ROB 决定处理器能够跨越多长的停顿寻找独立工作, scheduler 决定有多少就绪候选能够同时参与选择, 物理寄存器文件决定未退休值能够保留多久, load/store queue 则限制在飞访存数量。这些结构的容量尚未披露, 因此名义宽度不足以单独说明 Olympus 的延迟隐藏能力。

另一个关键约束是反压传播。decode queue 可以吸收短暂的前端抖动, ROB 和 scheduler 可以吸收执行延迟, store queue 与 MSHR 可以吸收缓存端延迟; 但其中任一结构耗尽后, 阻塞都会逐级向前传播。对 Olympus 这样的宽核心而言, 资源耗尽往往不是缓慢降速, 而是从高吞吐迅速转为前端停顿。因此, 分析其真实性能时, 应把队列容量、分配失败和 replay 占用视为与名义宽度同等重要的参数。

19.1.4 Front End

19.1.4.1 Branch Prediction as a Bandwidth Problem

Olympus 的分支预测器由 neural predictor 与若干 special-purpose predictors 组成, 并可在特定条件下每周期处理两个 taken branch。这里的关键不只是预测准确率, 而是预测带宽。当基本块较短、控制流密集时, 即使每个分支单独看都不难预测, 前端也可能因为目标生成不足而无法持续填

满宽解码器。两个 taken branch/cycle 的意义,是在同一周期内跨越更多基本块,使取指路径能够向更远处推进。[68]

“神经预测器”这一名称本身不能确定具体算法。它可能使用加权历史相关性,也可能采用多表组合或其它硬件学习结构;实际效果仍取决于历史长度、表容量、索引、置信度和访问延迟。Olympus 通过 ahead pipelining 让预测先于实际取指推进,以避免较复杂预测结构直接阻塞每周期的 PC 生成。

每周期两个 taken branch 仍会受到基本块对齐、BTB 命中、目标类型、cache-line 边界、间接分支和返回指令等条件约束。若第二个目标依赖第一个目标对应块中的信息,前端还需要多块预测或分层预测机制才能在同一周期确定更远路径。因此,该数字表示满足条件时的目标生成能力,而不是所有控制流上的固定吞吐。

19.1.4.2 Instruction Supply and Decode

Olympus 配置 64 KB、4 路组相联的 L1 指令缓存,指令读取带宽为 128 B/cycle,并在取指后配置可容纳 48 条指令的 decode queue。Arm 指令通常具有固定 4 B 编码,因此 128 B 理论上对应 32 条原始指令;有效指令取指能力则为每周期 16 条,说明供给还受到取指块组织、对齐、分支边界和内部传输粒度约束。最终解码宽度为每周期 10 条指令或融合后的操作。[68]

Olympus 的前端框图没有独立的 decoded-op cache。固定长度 Arm 指令降低了解码边界识别的复杂度,使较宽的直接解码路径更容易实现;但大型代码 footprint 下的 L1I、ITLB、目标预测和指令预取仍然重要。若 10-wide decode 经常因为 L1I miss 或目标生成不足而得不到输入,后端增加的宽度就无法转化为 IPC。

另一方面,128 B/cycle 的 L1I 读取能力不能被简单等价于 32 instructions/cycle 的持续供给。取指地址可能位于缓存行中部,taken branch 可能使行尾的大量字节失去价值,多个目标也可能落在不同 cache line 或页内。因而,前端宽度评价必须同时观察有效字节比例、分支密度和取指重定向频率,而不能只比较单一的峰值带宽。

19.1.5 Mid Core and Critical-Path Acceleration

Olympus 把 rename/allocate 到 scheduler 之间的区域称为 Mid Core,并将其用于 critical-path acceleration。重命名、分配和退休路径具有 10 micro-ops/cycle 的名义宽度,同时配置较大的 reorder buffer、整数与向量物理寄存器文件以及若干依赖缩短机制。ROB、物理寄存器和调度器的具体容量尚未披露,因此不能直接比较其有效乱序窗口与其它服务器核心。[68]

Mid Core 真正值得关注的地方,不是窗口“很大”这一笼统描述,而是 NVIDIA 明确披露了 move elimination、value prediction 和 memory renaming。传统乱序执行只能绕过尚未就绪的旧指令,寻找与其无关的年轻工作;一旦程序中形成单一而连续的真实数据依赖链,窗口再大也无法突破数据流本身的顺序。值预测与内存重命名试图改变的,正是这个限制。

19.1.5.1 Move Elimination

Move elimination 处理的是最简单的一类伪工作。若一条寄存器 move 只是把某个已存在的值赋予新的体系结构寄存器,重命名阶段可以直接让目标寄存器映射到同一物理值,而无需占用执行端口产生完全相同的数据。这样既减少调度器压力,也缩短依赖链,使后续消费者更早获得正确的物理寄存器标识。

这类优化看似局部, 却对宽发射核心十分重要。十路解码和大量 ALU 会快速放大无效操作带来的前端、调度和功耗成本; 若 move、零值生成、常量传播或其它可在重命名阶段完成的操作仍进入后端, 就会挤占真正有计算价值的执行槽。Olympus 将 move elimination 与 value optimization 并列, 说明重命名阶段还承担部分值传播职责, 具体覆盖范围尚不明确。

19.1.5.2 Value Prediction

值预测利用“某条指令或某个上下文产生的值经常重复”这一现象, 在真实结果生成之前给出预测值。依赖于该值的年轻指令可以立即进入调度与执行, 从而把原本串行的数据依赖链转换为一段可投机并行推进的执行。经典研究通常把这种现象称为 value locality。[71]

其基本过程可以分为四步。第一, 预测器根据指令 PC、历史值、路径或其它上下文生成候选值; 第二, 只有置信度足够高的预测才允许消费者提前执行; 第三, 生产者最终计算出真实值后进行验证; 第四, 若预测错误, 处理器必须使依赖该值的操作失效并重新执行。与分支预测类似, 值预测的收益来自提前推进, 代价则是错误恢复、验证带宽、额外状态和潜在功耗。

值预测比方向预测更难落地, 原因在于预测空间通常远大于“taken/not-taken”。一个 64-bit 结果理论上具有巨大的取值范围, 而且错误值可能沿依赖链传播到地址生成、分支条件和存储数据。实现必须精确追踪哪些操作消费了预测值, 并在验证失败时限制恢复范围。若采用全流水线 flush, 错误代价可能过高; 若采用选择性 replay, 则调度器、旁路网络和依赖追踪又会显著复杂化。

因此, Olympus 披露 value prediction 的意义, 不应被理解为“所有 load 或 ALU 都会猜值”, 而更可能是只覆盖具有高值稳定性、恢复成本可控且位于关键链上的操作。它对解释器状态、对象类型标签、循环中的重复边界值、虚函数元数据或某些稳定指针可能尤其有效, 但是否真正产生广泛收益, 仍取决于预测覆盖率、置信度策略和错误恢复实现。

19.1.5.3 Memory Renaming

Memory renaming 主要针对 store-to-load 依赖链: 若硬件能够预测或确认某个 load 将从更老 store 获得数据, 就可以在常规缓存访问完成之前建立更直接的通信关系。其思想与寄存器重命名不同, 却具有相似目标: 识别真正的数据生产者, 避免消费者被表面上的存储顺序或地址解析延迟阻塞。[18], [68]

典型场景是 store 地址或数据较早可得, 而后续 load 的完整地址尚未完成生成。传统 load/store queue 必须进行地址比较, 并在匹配后执行 store-to-load forwarding; 如果相关性可以由历史预测, 处理器便可能提前把 load 与特定 store 关联起来, 减少内容寻址比较和等待时间。更激进的实现还可能预测转发值, 使 load 的消费者先行执行, 再在地址和数据最终确认后验证。

这类机制同时具有明显风险。若预测了错误的生产者, 或忽略了中间存在的别名 store, 处理器就会违反内存顺序并必须重放相关操作。Memory renaming 因而需要与 memory disambiguation、store queue、地址生成和 replay 机制协同。可以确定的是 Olympus 试图缩短 store-load 通信链; 其具体算法和恢复粒度尚未披露。

19.1.5.4 Speculation Validation and Recovery

Olympus 同时使用分支预测、值预测和内存相关性预测, 因而恢复机制不是附属功能, 而是决定这些投机是否具有净收益的核心部分。三类预测虽然都在“提前推进”执行, 却具有不同的验证时刻: 分支在条件与目标计算完成时验证, 值预测在真实生产者产生结果时验证, memory

renaming 则要等相关 load 与更老 store 的地址、顺序和转发关系得到确认。验证越晚, 错误结果能够影响的年轻指令越多, 回收的在途工作也越大。

分支误预测通常需要重定向前端, 并清除错误路径上的年轻操作; 值预测错误和内存相关性错误则可能采用更局部的 replay。若硬件能够精确标记消费预测值的指令, 或追踪某次投机 load 的依赖后继, 就可以只使受污染的数据流重新进入调度器, 而保留不相关的在途工作。相反, 若依赖追踪粒度较粗, 设计可能退化为检查点恢复或大范围流水线 flush。前者降低单次错误代价, 却增加调度器、ROB、物理寄存器和旁路网络中的状态复杂度; 后者实现简单, 但会削弱激进预测在低置信度场景中的价值。

恢复还具有带宽问题。一次错误不只需要标记失效操作, 还可能重新读取寄存器、重新进行地址翻译和重新占用执行端口。若多个预测机制同时作用于同一依赖链, 错误可能形成 replay storm, 使局部恢复反复触发并长期占据调度资源。Value prediction 的收益因而同时取决于准确率、验证带宽、依赖标记和选择性 replay 的范围。

19.1.6 Execution Engine

Olympus 包含 8 个简单整数 ALU、2 个复杂整数 ALU、额外的 CRC/shift 能力、4 个分支执行单元、6 个 128-bit SVE 向量单元、4 条 load 路径与 2 条 store 路径, 其中两条向量路径还支持密码学操作。FP8 支持使部分低精度数据处理可以直接在 CPU 向量单元中完成。[68]

这些数字不能直接相加为“每周周期可发射多少条指令”。框图列出的可能是功能能力、执行单元数量或可接受特定操作的路径, 而非完全独立的 issue ports。多个功能单元可能共享 scheduler、寄存器读端口、旁路网络或前级管线; 复杂 ALU 也可能只在部分周期接受新操作。因此, 评价执行后端时必须区分资源数量、启动间隔、操作延迟和端口兼容性。

宽后端首先受到 wakeup-select 与操作数供给能力约束。调度器必须在每个周期识别大量刚刚就绪的操作, 并把它们分配到兼容端口; 物理寄存器文件则要提供相应的读写带宽。分簇调度、端口绑定、寄存器文件分区或延迟旁路可以降低复杂度, 却会产生新的冲突和额外周期。由于端口映射尚不明确, 执行单元数量只能说明能力集合, 不能说明任意指令组合下的持续发射率。

八个简单 ALU 与四个分支单元表明, Olympus 为控制流密集且包含大量地址、计数与状态判断的代码配置了较高的标量处理能力。四个分支单元的意义不一定是每周周期退休四条分支, 而可能是减少分支解析与普通整数操作之间的端口冲突, 使预测结果能够更早得到验证。对于拥有激进前端的核心, 分支执行带宽同样重要: 预测器负责提出未来路径, 执行单元则必须及时确认路径是否正确, 否则 ROB 中的错误路径会持续扩大。

因此, 分支解析延迟与预测带宽需要成对考察。前端能够每周周期跨越两个 taken branch, 只表示它可以快速生成候选路径; 若条件计算需要等待较长依赖链, 或分支单元之前存在拥塞, 错误路径仍会在 ROB 中累积。四个分支单元更可能提高并行验证能力并减少端口竞争, 而不是单独决定误预测代价。真正的恢复周期还包括结果产生、误预测检测、检查点恢复、前端重定向以及正确目标重新到达解码器的时间。

六条 128-bit SVE 路径代表另一种取舍。相比少量超宽向量单元, 更多中等宽度流水线有利于同时处理多个独立向量操作, 也更容易与 Spatial Multithreading 的资源分区对应。另一方面, 向量宽度只有 128 bit, 单条指令处理的数据量不及 256-bit 或 512-bit 实现; 实际吞吐取决于 SVE 指令如何被拆分、六条路径支持哪些操作, 以及 load/store 带宽能否持续供给。NVIDIA 将该设计描述为兼顾 HPC、数据处理和 AI 预处理, 但它更像高并发的通用向量后端, 而不是替代 GPU 的大规模矩阵计算单元。

这些资源被描述为 128-bit SVE execution units, 但该数字可能指内部数据通路宽度, 也可能对

应实现选择的体系结构向量长度。两者需要严格区分：若体系结构向量长度为 128 bit，六条路径主要提供并行操作数量；若指令需要跨多个片段执行，吞吐还会受到分解方式与占用周期约束。缺少指令延迟和启动间隔时，只能确定 Olympus 配置了较多中等宽度向量流水线。

四条 load 与两条 store 路径体现了明显的访存侧扩张。不过，访存路径数量只表示核心接收地址生成或缓存访问请求的潜力，持续吞吐仍受到 L1D bank、TLB 端口、load/store queue、缓存行边界和未命中资源限制。若四条 load 同时访问同一 bank，或者其中多数成为 L1 miss，瓶颈会迅速下移到 MSHR、L2 或 SCF。Olympus 的执行宽度必须与后文讨论的缓存和内存系统结合起来理解。

19.1.7 Load/Store Pipeline and Memory Ordering

四条 load 路径与两条 store 路径不能直接解释为六套彼此独立的地址生成、DTLB 查询和 L1D 访问通路。一条访存操作至少要经历有效地址生成、地址翻译、load/store queue 分配、缓存 bank 访问、相关性检查与结果转发；store 还需要分别处理地址和数据，并在退休后按照内存模型要求对外可见。不同阶段可能具有不同宽度，因此峰值地址生成率、L1D 访问率和最终完成率未必相同。

对于 load，最关键的低延迟路径通常是地址生成、DTLB 命中、L1D 命中和旁路返回。96 KB L1D 若采用多个 bank 维持并行访问，同一 bank、跨 cache line 或未对齐访问便可能降低吞吐；若 DTLB 端口少于 load 路径，地址翻译也会成为更早的结构冲突。一次 L1 miss 则需要占用 miss status holding register (MSHR) 或等价结构，并把请求送往私有 L2。四条 load 路径只有在 bank 分布、翻译命中和未命中资源均足够时，才可能形成接近峰值的持续带宽。

load/store queue 承担的任务不仅是保存尚未退休的访存操作，还要维护年龄关系、检查地址别名并完成 store-to-load forwarding。为了让年轻 load 绕过地址尚未解析的更老 store，处理器需要进行 memory dependence prediction；若预测为独立，load 可以提前访问缓存，但当更老 store 随后解析到同一地址时，该 load 及其消费者必须 replay。Olympus 的 memory renaming 可以被视为这条路径上的进一步优化：它不仅预测“是否相关”，还可能预测“与哪一个 store 相关”，从而更早建立数据转发关系。

实现上的难点在于转发条件十分严格。store 与 load 可能只部分重叠，数据可能尚未全部生成，访问也可能跨越缓存行或页边界。即使地址匹配，硬件仍要组合正确字节并保持异常与内存排序语义。错误转发、遗漏别名或过早执行都会触发重放；若依赖链很长，重放代价可能远大于一次普通缓存未命中。因此，store queue 的比较带宽、转发网络和 replay 选择机制，是判断四 load、两 store 是否真正有效的关键。

内存模型进一步约束 store 的完成方式。体系结构允许的重排序可以帮助 load 提前执行，但已退休 store 仍需经过 store queue 或 store buffer，按照一致性与屏障语义逐步获得全局可见性。原子操作、屏障、设备访问和一致性冲突会压缩可重排空间，并可能迫使流水线等待较老访存完成。Load/store queue、MSHR、DTLB 端口和 replay 组织的容量尚不明确，因此访存路径数量不能直接代表持续完成率。

19.1.8 Cache and Translation Hierarchy

Olympus 配置 96 KB、6 路组相联的 L1 数据缓存和 64 KB、4 路组相联的 L1 指令缓存，每个核心拥有 2 MB、8 路组相联的私有 L2。一级翻译结构包括 64-entry 全相联 ITLB 与 112-entry 全相联 DTLB，并由更大的统一二级 TLB 提供后备。[68]

96 KB L1D 能够容纳更多对象字段、解释器状态与短生命周期工作集,从而降低对 L2 的压力;但 6 路相联和更大容量也会增加标签比较、布线和功耗。其命中延迟尚未披露,因此不能仅凭容量判断关键路径代价。

每核 2 MB 私有 L2 则体现出 Olympus 对大代码和不规则数据工作集的重视。较大的私有 L2 可以减少线程之间的共享缓存竞争,并为图遍历、数据库和运行时元数据提供更稳定的局部驻留空间。其标称访问延迟约为 10 个周期,但实际 load-to-use 延迟仍会随地址、冲突和在途请求变化。[68]

全相联一级 TLB 的优势是避免组冲突,使有限表项能够更稳定地覆盖不规则虚拟地址工作集;代价则是更高的比较能耗和更复杂的多端口实现。对于四 load、两 store 的宽访存后端,TLB 查询带宽、page-walk 并行度和后级 TLB 访问率可能比单纯容量更关键。

19.1.9 Prefetching and the Graph Prefetcher

Olympus 同时配置 stream、indirect、correlated-miss、spatial 与 graph-oriented 等多类预取器。多预取器并存意味着硬件并不假设所有工作负载遵循同一种地址规律,而是分别针对连续流、固定或重复关联、空间 footprint 以及数据依赖型地址生成。真正困难的问题不是生成更多候选地址,而是避免多个预取器互相重复、争用带宽和污染缓存。

Graph prefetcher 处理形如

```
result += A[B[i]]
```

的数组间接访问。对处理器而言, $B[i]$ 是 producer load, 其返回值参与计算 consumer 地址 $A[B[i]]$ 。常规预取器在 producer 结果返回之前无法知道 consumer 地址, 因此两次 cache miss 容易串行暴露。Olympus 试图学习 producer 与 consumer 的对应关系, 在观察到 producer 流后, 提前生成后续 consumer 请求。[68]

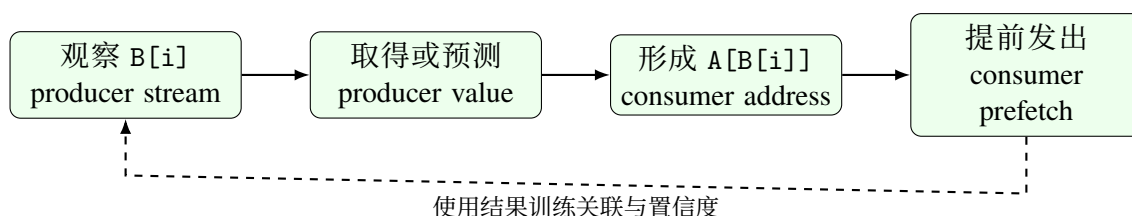


Figure 19.2: 间接访问预取的基本 producer-consumer 链。核心问题是把原本串行的两个 miss 部分重叠。

这一思想延续了依赖型链表预取、indirect memory prefetching 和相关性地址生成等研究路线。Olympus 的特点在于把这类机制与较大私有缓存、高带宽内存和图工作负载支持放入同一产品设计。[29], [68]

实现此类预取器需要解决至少四个问题。第一, 如何识别 producer 与 consumer 的 PC 或地址关系; 第二, 如何保证 producer value 足够早地获得, 否则预取仍然来不及; 第三, 如何控制错误关联和越界地址带来的无效流量; 第四, 预取结果应放入 L1、L2 还是独立缓冲。若 consumer 很快到达, 放在 L2 可能仍然太远; 若直接填入 L1, 则错误预取会污染最宝贵的近端容量。

Graph prefetching、value prediction 与 memory renaming 分别作用于地址依赖、寄存器数据依赖和 store-load 通信。它们都需要置信度与恢复机制, 但现有信息不足以说明三者是否形成直接串联的数据通路。

19.1.10 Spatial Multithreading

Vera 为每个 Olympus 核心提供两个硬件线程, 并将其称为 Spatial Multithreading。单线程模式允许一个线程使用完整核心, 双线程模式则把部分资源划分给两个上下文, 以降低相互阻塞和吞吐波动。其行为取决于前端、乱序窗口、执行端口与访存队列采用静态分区、动态共享还是带保留份额的混合策略。[68]

传统 simultaneous multithreading 并不是把两个线程简单地按时间轮流送入各流水阶段, 而是允许多个线程在同一周期竞争共享发射槽。Spatial Multithreading 的区别在于更强调资源分区和性能隔离, 而不是首次允许两个上下文并行使用执行资源。[53], [68]

Spatial Multithreading 更准确的定位, 应是在 SMT 资源池中引入更强的空间分区与性能隔离。其优点是减少 noisy neighbor: 一个线程的分支密度、cache miss 或向量操作不容易完全占据另一线程所需的关键资源。其代价则是资源碎片化: 若线程 A 的整数执行配额空闲, 而线程 B 的整数队列拥塞, 严格分区可能阻止 B 使用本可闲置的能力。因此, Spatial Multithreading 的效果取决于分区是否固定、是否可按阶段调整, 以及哪些结构仍然保持共享。

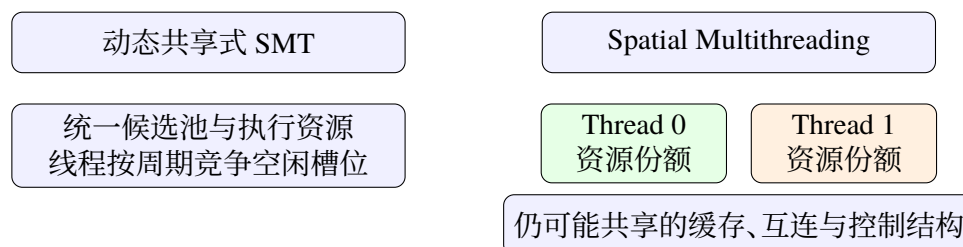


Figure 19.3: 动态资源池与空间分区概念差异。Olympus 的具体资源边界取决于各结构的共享与保留策略。

尚不明确的资源边界包括分支历史、ROB、scheduler、物理寄存器、load/store queue、MSHR 与 L1 cache ways。单线程和双线程模式切换还可能需重新配置内部配额, 因此其成本不能等同于普通软件线程切换。

从核心利用率看, Spatial Multithreading 在资源碎片与线程干扰之间重新取舍。静态分区能够阻止一个线程耗尽 ROB、scheduler 或 LSQ, 却会在另一线程停顿时留下不可借用的空闲份额; 完全动态共享能够提高平均利用率, 却可能使长延迟线程占据大量在途状态。带保留份额的动态共享可以在两者之间折中, 但其具体范围尚未披露。

19.1.11 Scalable Coherency Fabric and System-Level Cache

Vera 的 88 个核心通过第二代 NVIDIA Scalable Coherency Fabric(SCF)连接, SCF 剖分带宽为 3.4 TB/s。互连由分布式 Coherency Switch Node(CSN)构成, 每个 CSN 最多连接两个核心, 并通过 Memory Switch Node(MSN)连接内存控制器。对 Olympus 核心而言, SCF 决定私有 L2 未命中后的路由距离、共享缓存访问延迟与一致性流量承载能力。[68], [69]

SCF 同时集成 164 MB 分布式 System-Level Cache。称其为“统一 L3”描述的是地址与一致性语义, 而不意味着任意核心访问任意 slice 都具有完全相同的物理距离。请求仍需经过片上路由、目录查询和目标 slice; 核心附近的访问与远端 slice 访问可能具有不同延迟。分布式组织提高了总带宽, 却没有消除片上距离、拥塞和 bank 冲突。

分布式 SLC 的主要作用有三类。第一, 吸收私有 L2 之外的共享数据, 降低 DRAM 流量; 第二, 为跨核心共享提供一致性汇合点; 第三, 在 88 核并发下把目录与数据带宽分散到多个 slice。与此

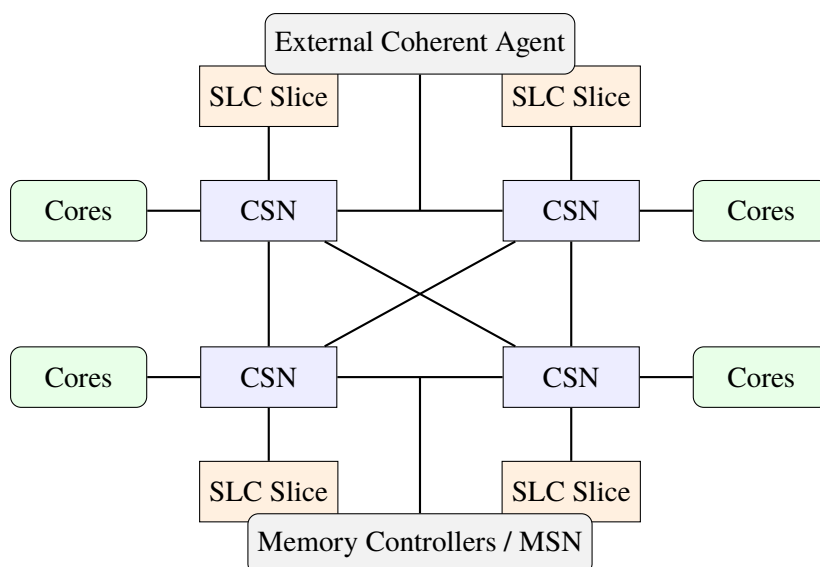


Figure 19.4: SCF 的概念化组织。共享地址空间由分布式路由、目录与 SLC slices 共同实现, 而非单一集中式缓存。

同时, 地址到 slice 的映射、热点页、共享写入和原子操作仍可能造成局部拥塞。评价 SCF 时应关注的不只是总带宽, 还包括不同核心对不同 slice 的延迟分布、双向流量公平性和满载下的排队行为。

若 SLC 容量或内存带宽被分区, 核心可见的未命中延迟和有效带宽也会随之改变; 但这属于共享层级的资源控制, 不改变 Olympus 私有流水线的基本组织。

19.1.12 Core-Visible Memory Bandwidth and Latency

Vera 的 LPDDR5X 内存系统提供最高 1.2 TB/s 总带宽。对单个 Olympus 核心而言, 这一数字的主要意义是允许更多 L2 与 SLC 未命中同时在途, 并在多核并发时降低内存通道过早饱和的概率。高带宽只有在 load queue、MSHR、SCF、内存控制器和 DRAM banks 能够维持足够的并行请求时才可利用; 若核心无法生成足够多的独立 miss, 理论带宽不会改善单条依赖链的等待时间。[68], [69]

带宽不等于访问延迟。一次随机 load miss 仍需经过私有 L2、SLC 查询、片上路由、控制器排队和 DRAM 时序; 当地址依赖限制 memory-level parallelism 时, 这些阶段会直接暴露在关键路径上。相反, streaming 或高度并发的 miss 可以利用 bank-level parallelism 与预取, 将多个等待重叠起来。因而, 评价 Olympus 的内存侧能力应同时测量空载延迟、不同并发度下的带宽爬升、MSHR 饱和点和多核压力下的排队延迟, 而不能只引用峰值传输率。

19.1.13 Summary

从核心设计看, Olympus 最有辨识度的部分并不是 10-wide decode 或 96 KB L1D 这些单一参数, 而是它同时在三条关键路径上使用预测: 前端通过高带宽分支预测推进控制流, Mid Core 通过 value prediction 与 memory renaming 推进数据依赖, 访存系统通过 indirect 和 graph prefetching 推进地址生成。三者共同体现了一种明确的设计取向: 当传统乱序窗口无法再从已知依赖图中找到足够并行性时, 硬件开始预测依赖图尚未产生的未来信息。

这种设计同时意味着较高的验证与恢复复杂度。分支预测错误需要重定向前端, 值预测错误需要重放消费者, 内存依赖预测错误需要恢复 load/store 顺序, 预取错误则消耗缓存和带宽。

Olympus 的真正水平将取决于这些机制是否具有足够高的置信度、是否能够限制错误恢复范围, 以及多种预测器是否会在同一工作负载中相互放大资源压力。

核心频率、ROB 与 scheduler 容量、物理寄存器数量、预测器层级、issue port 映射、缓存延迟、load/store queue、MSHR 与线程资源分区仍会显著影响实际吞吐。名义宽度描述结构上限, 持续性能则取决于这些资源是否形成平衡的数据流。

Olympus 没有脱离现代乱序超标量处理器的基本范式, 却把值预测、内存通信预测和面向间接访问的预取置于核心设计的显著位置。其性能取决于这些预测能否覆盖关键依赖, 同时把验证、replay 与资源占用成本限制在可控范围内。

References

- [1] J. L. Hennessy, D. A. Patterson, and C. Kozyrakis, *Computer Architecture: A Quantitative Approach*, 7th ed. Morgan Kaufmann, 2025.
- [2] S. Mittal, “A survey of techniques for dynamic branch prediction,” *Concurrency and Computation: Practice and Experience*, vol. 31, no. 8, e4666, 2019. doi: [10.1002/cpe.4666](https://doi.org/10.1002/cpe.4666)
- [3] A. Perais and R. Sheikh, “Branch target buffer organizations,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’23, Toronto, ON, Canada: Association for Computing Machinery, 2023, pp. 240–253. doi: [10.1145/3613424.3623774](https://doi.org/10.1145/3613424.3623774) [Online]. Available: <https://doi.org/10.1145/3613424.3623774>
- [4] N. K. Soundararajan, P. Braun, T. A. Khan, B. Kasikci, H. Litz, and S. Subramoney, “Pdede: Partitioned, deduplicated, delta branch target buffer,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’21, Virtual Event, Greece: Association for Computing Machinery, 2021, pp. 779–791. doi: [10.1145/3466752.3480046](https://doi.org/10.1145/3466752.3480046) [Online]. Available: <https://doi.org/10.1145/3466752.3480046>
- [5] A. Sez nec, S. Jourdan, P. Sainrat, and P. Michaud, “Multiple-block ahead branch predictors,” in *Proceedings of the Seventh International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS VII, Cambridge, Massachusetts, USA: Association for Computing Machinery, 1996, pp. 116–127. doi: [10.1145/237090.237169](https://doi.org/10.1145/237090.237169) [Online]. Available: <https://doi.org/10.1145/237090.237169>
- [6] B. Pham, A. Bhattacharjee, Y. Eckert, and G. H. Loh, “Increasing tlb reach by exploiting clustering in page translations,” in *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*, IEEE Computer Society, 2014, pp. 558–567. doi: [10.1109/HPCA.2014.6835964](https://doi.org/10.1109/HPCA.2014.6835964)
- [7] C. H. Park, T. Heo, J. Jeong, and J. Huh, “Hybrid tlb coalescing: Improving tlb translation coverage under diverse fragmented memory allocations,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, ser. ISCA ’17, Toronto, ON, Canada: Association for Computing Machinery, 2017, pp. 444–456. doi: [10.1145/3079856.3080217](https://doi.org/10.1145/3079856.3080217) [Online]. Available: <https://doi.org/10.1145/3079856.3080217>
- [8] A. Bhattacharjee and M. R. Martonosi, “Inter-core cooperative tlb prefetchers for chip multiprocessors,” *ACM SIGPLAN Notices*, vol. 45, no. 3, pp. 359–370, 2010. doi: [10.1145/1736020.1736060](https://doi.org/10.1145/1736020.1736060)
- [9] G. Vavouliotis, “Advanced hardware prefetching in virtual memory systems,” Ph.D. dissertation, Polytechnic University of Catalonia, 2024. doi: [10.5821/dissertation-2117-403959](https://doi.org/10.5821/dissertation-2117-403959)

- [10] D. Lustig, A. Bhattacharjee, and M. Martonosi, “Tlb improvements for chip multiprocessors: Inter-core cooperative prefetchers and shared last-level tlbs,” *ACM Transactions on Architecture and Code Optimization*, vol. 10, no. 1, 2:1–2:22, 2013. doi: [10.1145/2445572.2445574](https://doi.org/10.1145/2445572.2445574)
- [11] A. Kurth, P. Vogel, A. Marongiu, and L. Benini, “Scalable and efficient virtual memory sharing in heterogeneous socs with tlb prefetching and mmu-aware dma engine,” in *Proceedings of the 36th IEEE International Conference on Computer Design*, 2018, pp. 292–300. doi: [10.1109/ICCD.2018.000052](https://doi.org/10.1109/ICCD.2018.000052)
- [12] T. D. Duong, Y. S. Kim, and J. Y. Hur, “Tlb coalescing with range compressed page table for embedded i/o devices,” *IEEE Access*, vol. 13, pp. 12 623–12 633, 2025. doi: [10.1109/ACCESS.2025.3528945](https://doi.org/10.1109/ACCESS.2025.3528945)
- [13] E. Rotenberg, S. Bennett, and J. E. Smith, “Trace cache: A low latency approach to high bandwidth instruction fetching,” in *Proceedings of the 29th Annual ACM/IEEE International Symposium on Microarchitecture*, ser. MICRO 29, Paris, France: IEEE Computer Society, 1996, pp. 24–35.
- [14] D. Sima, “The design space of register renaming techniques,” *IEEE Micro*, vol. 20, no. 5, pp. 70–83, 2000. doi: [10.1109/40.877855](https://doi.org/10.1109/40.877855)
- [15] I. Jeong, J. Lee, M. K. Yoon, and W. W. Ro, “Reconstructing out-of-order issue queue,” in *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’22, Chicago, Illinois, USA: IEEE Press, 2023, pp. 144–161. doi: [10.1109/MICRO56248.2022.000023](https://doi.org/10.1109/MICRO56248.2022.000023) [Online]. Available: <https://doi.org/10.1109/MICRO56248.2022.000023>
- [16] R. M. Tomasulo, “An efficient algorithm for exploiting multiple arithmetic units,” *IBM Journal of Research and Development*, vol. 11, no. 1, pp. 25–33, 1967. doi: [10.1147/rd.111.0025](https://doi.org/10.1147/rd.111.0025)
- [17] A. Perais and A. Sez nec, “Storage-free memory dependency prediction,” *IEEE Computer Architecture Letters*, vol. 16, no. 2, pp. 129–132, 2017. doi: [10.1109/LCA.2016.2628379](https://doi.org/10.1109/LCA.2016.2628379)
- [18] A. Moshovos and G. S. Sohi, “Streamlining inter-operation memory communication via data dependence prediction,” in *Proceedings of the 30th Annual ACM/IEEE International Symposium on Microarchitecture*, 1997, pp. 235–245. doi: [10.5555/266800.266823](https://doi.org/10.5555/266800.266823)
- [19] J. E. Smith and A. R. Pleszkun, “Implementing precise interrupts in pipelined processors,” *IEEE Transactions on Computers*, vol. 37, no. 5, pp. 562–573, 1988. doi: [10.1109/12.4607](https://doi.org/10.1109/12.4607)
- [20] R. Balasubramonian, N. P. Jouppi, and N. Muralimanohar, *Multi-Core Cache Hierarchies*. Morgan & Claypool, 2011. doi: [10.2200/S00365ED1V01Y201105CAC017](https://doi.org/10.2200/S00365ED1V01Y201105CAC017)
- [21] V. Nagarajan, D. J. Sorin, M. D. Hill, and D. A. Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd ed. Morgan & Claypool, 2020. doi: [10.2200/S00962ED2V01Y201910CAC049](https://doi.org/10.2200/S00962ED2V01Y201910CAC049)
- [22] L. Lamport, “How to make a multiprocessor computer that correctly executes multiprocess programs,” *IEEE Transactions on Computers*, vol. C-28, no. 9, pp. 690–691, 1979. doi: [10.1109/TC.1979.1675439](https://doi.org/10.1109/TC.1979.1675439)
- [23] A. Jain and C. Lin, *Cache Replacement Policies*. Morgan & Claypool, 2019. doi: [10.2200/S00922ED1V01Y201905CAC047](https://doi.org/10.2200/S00922ED1V01Y201905CAC047)

- [24] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely, and J. Emer, “Adaptive insertion policies for high performance caching,” in *Proceedings of the 34th Annual International Symposium on Computer Architecture*, 2007, pp. 381–391. doi: [10.1145/1250662.1250709](https://doi.org/10.1145/1250662.1250709)
- [25] A. Jaleel, K. B. Theobald, S. C. Steely, and J. Emer, “High performance cache replacement using re-reference interval prediction,” in *Proceedings of the 37th Annual International Symposium on Computer Architecture*, 2010, pp. 60–71. doi: [10.1145/1815961.1815971](https://doi.org/10.1145/1815961.1815971)
- [26] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely, and J. Emer, “Ship: Signature-based hit predictor for high performance caching,” in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, 2011, pp. 430–441. doi: [10.1145/2155620.2155671](https://doi.org/10.1145/2155620.2155671)
- [27] A. Jain and C. Lin, “Back to the future: Leveraging belady’s algorithm for improved cache replacement,” in *Proceedings of the 43rd Annual International Symposium on Computer Architecture*, 2016, pp. 78–89. doi: [10.1109/ISCA.2016.17](https://doi.org/10.1109/ISCA.2016.17)
- [28] B. Falsafi and T. F. Wenisch, *A Primer on Hardware Prefetching*. Morgan & Claypool, 2014. doi: [10.2200/S00581ED1V01Y201405CAC028](https://doi.org/10.2200/S00581ED1V01Y201405CAC028)
- [29] A. Roth, A. Moshovos, and G. S. Sohi, “Dependence based prefetching for linked data structures,” in *Proceedings of the Eighth International Conference on Architectural Support for Programming Languages and Operating Systems*, 1998, pp. 115–126. doi: [10.1145/291069.291034](https://doi.org/10.1145/291069.291034)
- [30] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, “Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers,” in *Proceedings of the 13th IEEE International Symposium on High Performance Computer Architecture*, 2007, pp. 63–74. doi: [10.1109/HPCA.2007.346185](https://doi.org/10.1109/HPCA.2007.346185)
- [31] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, “Spectre attacks: Exploiting speculative execution,” in *2019 IEEE Symposium on Security and Privacy*, 2019, pp. 1–19. doi: [10.1109/SP.2019.00002](https://doi.org/10.1109/SP.2019.00002)
- [32] E. M. Koruyeh, K. N. Khasawneh, C. Song, and N. Abu-Ghazaleh, “Spectre returns! speculation attacks using the return stack buffer,” in *12th USENIX Workshop on Offensive Technologies*, 2018.
- [33] E. Barberis, P. Frigo, M. Muench, H. Bos, and C. Giuffrida, “Branch history injection: On the effectiveness of hardware mitigations against cross-privilege spectre-v2 attacks,” in *31st USENIX Security Symposium*, 2022, pp. 1357–1374.
- [34] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, “Meltdown: Reading kernel memory from user space,” in *27th USENIX Security Symposium*, 2018, pp. 973–990.
- [35] Intel Corporation. “Speculative store bypass / cve-2018-3639 / intel-sa-00115,” Accessed: Aug. 24, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/speculative-store-bypass.html>

- [36] Intel Corporation. “Microarchitectural data sampling,” Accessed: Aug. 24, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/intel-analysis-microarchitectural-data-sampling.html>
- [37] Intel Corporation. “Gather data sampling,” Accessed: Aug. 24, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/gather-data-sampling.html>
- [38] D. Evtvyushkin, D. Ponomarev, and N. Abu-Ghazaleh, “Jump over aslr: Attacking branch predictors to bypass aslr,” in *49th Annual IEEE/ACM International Symposium on Microarchitecture*, 2016, pp. 1–13. DOI: 10.1109/MICRO.2016.7783733
- [39] M. Lipp, V. Hadžić, M. Schwarz, A. Perais, C. Maurice, and D. Gruss, “Take a way: Exploring the security implications of amd’s cache way predictors,” in *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, 2020, pp. 813–825. DOI: 10.1145/3320269.3384746
- [40] K. Troester and R. Bhargava, “Amd next generation “zen 4” core and 4th gen amd epyc 9004 server cpu,” in *Hot Chips 35*, 2023. Accessed: Aug. 24, 2026. [Online]. Available: https://hc2023.hotchips.org/assets/program/conference/day1/CPU1/HC_Zen4_Epyc_Final_20230825%20-%20Embargoed%20until%20Aug%2029%202023.pdf
- [41] B. Munger et al., ““zen 4”: The amd 5nm 5.7ghz x86-64 microprocessor core,” in *2023 IEEE International Solid-State Circuits Conference*, 2023, pp. 24–26. DOI: 10.1109/ISSCC42615.2023.10067540
- [42] Advanced Micro Devices, Inc., *Software optimization guide for the amd zen4 microarchitecture*, Publication 57647, Revision 1.00, 2023.
- [43] LLVM Project. “X86scheduleznver4.td: Amd zen 4 scheduling model,” Accessed: Aug. 24, 2026. [Online]. Available: <https://github.com/llvm/llvm-project/blob/main/llvm/lib/Target/X86/X86ScheduleZnver4.td>
- [44] Advanced Micro Devices, Inc. “4th gen amd epyc processor architecture,” Accessed: Aug. 24, 2026. [Online]. Available: <https://www.amd.com/en/products/processors/server/epyc/4th-generation-architecture.html>
- [45] Advanced Micro Devices, Inc. “Amd “zen” core architecture,” Accessed: Aug. 22, 2026. [Online]. Available: <https://www.amd.com/en/technologies/zen-core.html>
- [46] Advanced Micro Devices, Inc. “Amd ryzen 9 9950x specifications,” Accessed: Aug. 22, 2026. [Online]. Available: <https://www.amd.com/en/products/processors/desktops/ryzen/9000-series/amd-ryzen-9-9950x.html>
- [47] B. Cohen, M. Subramony, and M. Clark, “Next generation “zen 5” core,” in *2024 IEEE Hot Chips 36 Symposium*, 2024. DOI: 10.1109/HCS61935.2024.10665102 Accessed: Aug. 24, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/10665102>
- [48] Advanced Micro Devices, Inc., *5th gen amd epyc processor architecture*, First Edition, 242937605-C, Mar. 2025.

- [49] Advanced Micro Devices, Inc. “5th generation amd epyc server cpus,” Accessed: Aug. 22, 2026. [Online]. Available: <https://www.amd.com/en/products/processors/server/epyc/9005-series.html>
- [50] Advanced Micro Devices, Inc., *Software optimization guide for the amd zen5 microarchitecture*, 58455, Aug. 15, 2024. Accessed: Aug. 24, 2026. [Online]. Available: https://docs.amd.com/v/u/en-US/58455_1.00
- [51] G. Cozma. “A video interview with mike clark, chief architect of zen at amd,” Accessed: Aug. 24, 2026. [Online]. Available: <https://chipsandcheese.com/p/a-video-interview-with-mike-clark-chief-architect-of-zen-at-amd>
- [52] Advanced Micro Devices, Inc., *Amd epyc 8005 processor architecture overview*, 72128, May 19, 2026. Accessed: Aug. 24, 2026. [Online]. Available: <https://docs.amd.com/v/u/en-US/72128-amd-epyc-8005-processor-architecture-overview>
- [53] D. M. Tullsen, S. J. Eggers, and H. M. Levy, “Simultaneous multithreading: Maximizing on-chip parallelism,” in *Proceedings of the 22nd Annual International Symposium on Computer Architecture*, 1995, pp. 392–403. doi: 10.1145/223982.224449
- [54] Intel Corporation. “Intel core ultra 200s series and arrow lake architecture press briefing,” Accessed: Aug. 22, 2026. [Online]. Available: <https://download.intel.com/newsroom/2024/client-computing/IFA-ARL-Press-Briefing.pdf>
- [55] Intel Corporation. “Computex 2024 performance index,” Accessed: Aug. 22, 2026. [Online]. Available: <https://edc.intel.com/content/www/us/en/products/performance/benchmarks/computex-2024/>
- [56] Intel Corporation. “Intel processors and processor cores based on skymont microarchitecture: Instruction throughput and latency,” Accessed: Aug. 22, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/content-details/837381/intel-processors-and-processor-cores-based-on-skymont-microarchitecture-instruction-throughput-and-latency.html>
- [57] Intel Corporation. “Intel 64 and ia-32 architectures optimization reference manual,” Accessed: Aug. 22, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/content-details/671488/intel-64-and-ia-32-architectures-optimization-reference-manual-volume-1.html>
- [58] Intel Corporation. “2024 intel tech tour: Next-generation p-core—the lion cove microarchitecture,” Accessed: Aug. 22, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/content-details/824430/2024-intel-tech-tour-next-gen-p-core-the-lion-cove-microarchitecture.html>
- [59] Arm Limited. “The ultimate cpu: Arm cortex-x925’s breakthrough with a 15 percent ipc improvement,” Accessed: Aug. 22, 2026. [Online]. Available: <https://newsroom.arm.com/blog/arm-v9-cortex-x925-cpu-performance>
- [60] Arm Limited, *Arm cortex-x925 core technical reference manual*, 2024. Accessed: Aug. 22, 2026. [Online]. Available: <https://developer.arm.com/documentation/102807/latest/>

- [61] Arm Limited. “Arm neoverse n2,” Accessed: Aug. 22, 2026. [Online]. Available: <https://www.arm.com/products/silicon-ip-cpu/neoverse/neoverse-n2>
- [62] A. Pellegrini, *Arm neoverse n2: Arm’s 2nd generation high performance infrastructure cpus and system ips*, Hot Chips 33 Conference Presentation, Aug. 2021.
- [63] Arm Limited, *Arm neoverse n2 core technical reference manual*, 2021. Accessed: Aug. 22, 2026. [Online]. Available: <https://developer.arm.com/documentation/102099/latest/>
- [64] Arm Limited. “Arm neoverse v2,” Accessed: Aug. 22, 2026. [Online]. Available: <https://www.arm.com/products/silicon-ip-cpu/neoverse/neoverse-v2>
- [65] S. Mondal. “Arm neoverse v2 platform: Best in class performance/tco for the cloud,” Accessed: Aug. 22, 2026. [Online]. Available: <https://developer.arm.com/community/arm-community-blogs/b/servers-and-cloud-computing-blog/posts/arm-neoverse-v2-platform-best-in-class-cloud-and-ai-ml-performance>
- [66] M. Bruce, *Arm neoverse v2 platform: Leadership performance and power efficiency for next-generation cloud computing, ml and hpc workloads*, Hot Chips 2023 Conference Presentation, Aug. 2023.
- [67] Arm Limited, *Arm neoverse v2 core technical reference manual*, Document ID 102375-0002-03-en, Issue 03, 2022.
- [68] NVIDIA Corporation, *Nvidia vera cpu architecture: Max single-threaded cpu at scale for agents*, Version 1.1, 2026. Accessed: Aug. 22, 2026. [Online]. Available: https://nvdam.widen.net/s/nmw5vblpqd/nvidia_vera_cpu_architecture_whitepaper?nvid=nv-tblg-543584
- [69] NVIDIA Corporation. “Inside nvidia vera cpu: Olympus cores built for maximum single-threaded performance in agentic ai,” Accessed: Aug. 22, 2026. [Online]. Available: <https://developer.nvidia.com/blog/inside-nvidia-vera-cpu-olympus-cores-built-for-maximum-single-threaded-performance-in-agentic-ai/>
- [70] NVIDIA Corporation. “Nvidia vera cpu,” Accessed: Aug. 22, 2026. [Online]. Available: <https://www.nvidia.com/en-us/data-center/vera-cpu/>
- [71] M. H. Lipasti, C. B. Wilkerson, and J. P. Shen, “Value locality and load value prediction,” in *Proceedings of the Seventh International Conference on Architectural Support for Programming Languages and Operating Systems*, 1996, pp. 138–147. DOI: [10.1145/237090.237173](https://doi.org/10.1145/237090.237173)